



SOICT

HUST

ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

Multimodal Machine Learning Lecture

3.1: Unimodal Representations

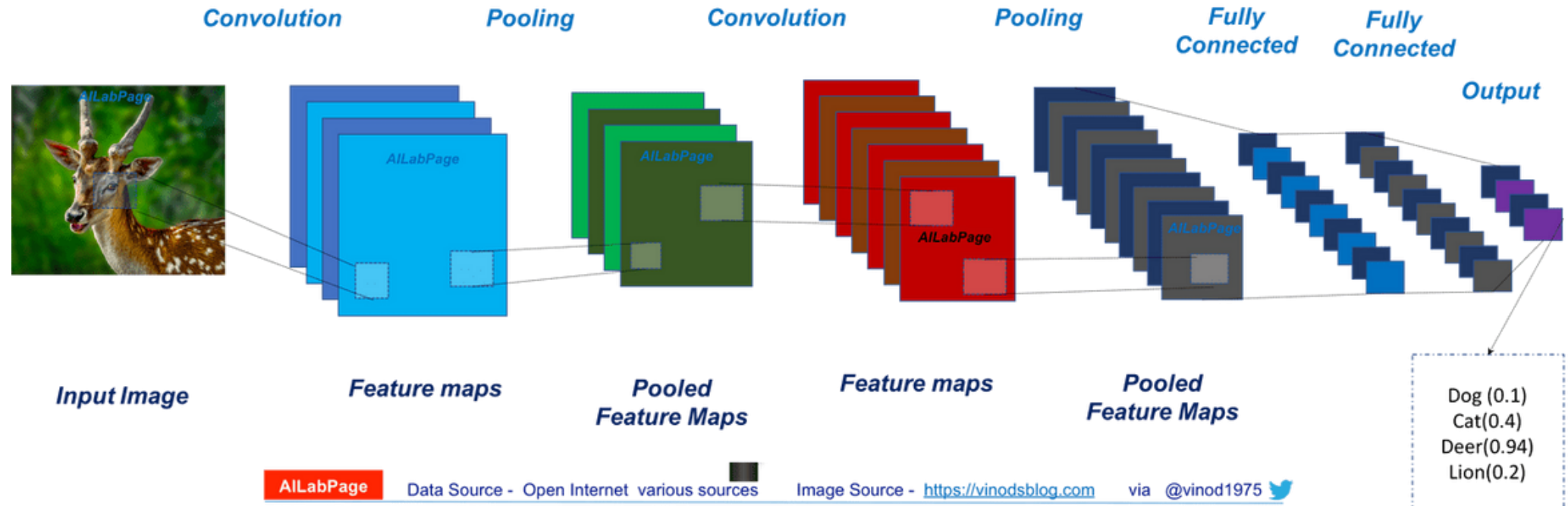
Đàm Quang Tuấn

Lecture Objectives

- Region-based CNNs
 - Object detection and recognition
- Word representations
 - Distributional hypothesis
 - Word vector space
- Sentence Modeling
 - Recurrent neural networks
- Language models and pretraining
- Syntax and language structure
 - Recursive neural networks

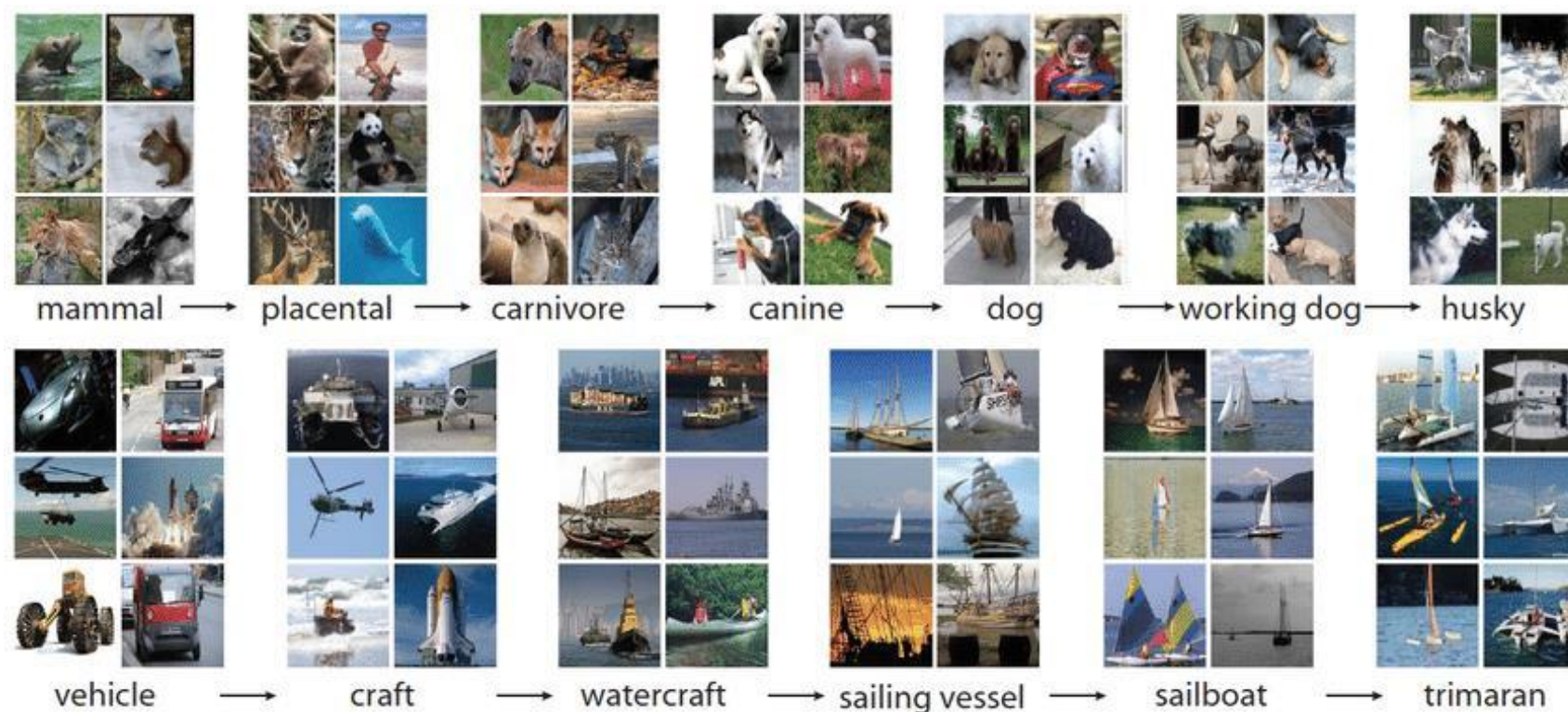
Region-based CNNs

Convolutional Neural Network



➡ Translation invariance is enabled by the pooling layer

ImageNet



- ➡ Objects already centered, ready for training Hierarchy
- ➡ similar to FrameNet(originally designed for words)

Object Detection (and Segmentation)



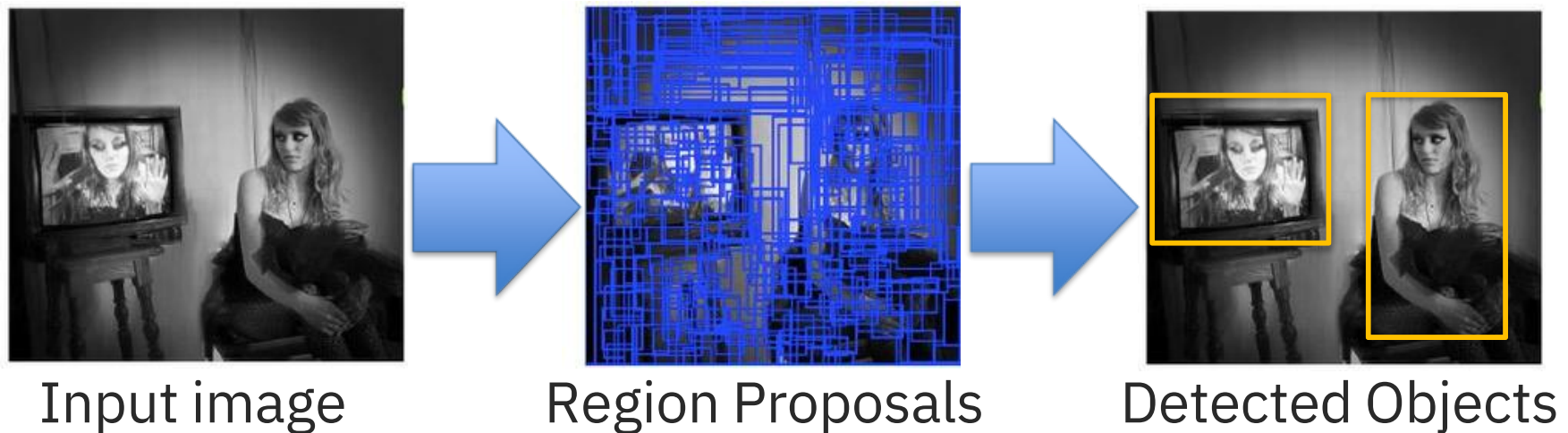
Input image



Detected Objects

One option: Sliding window

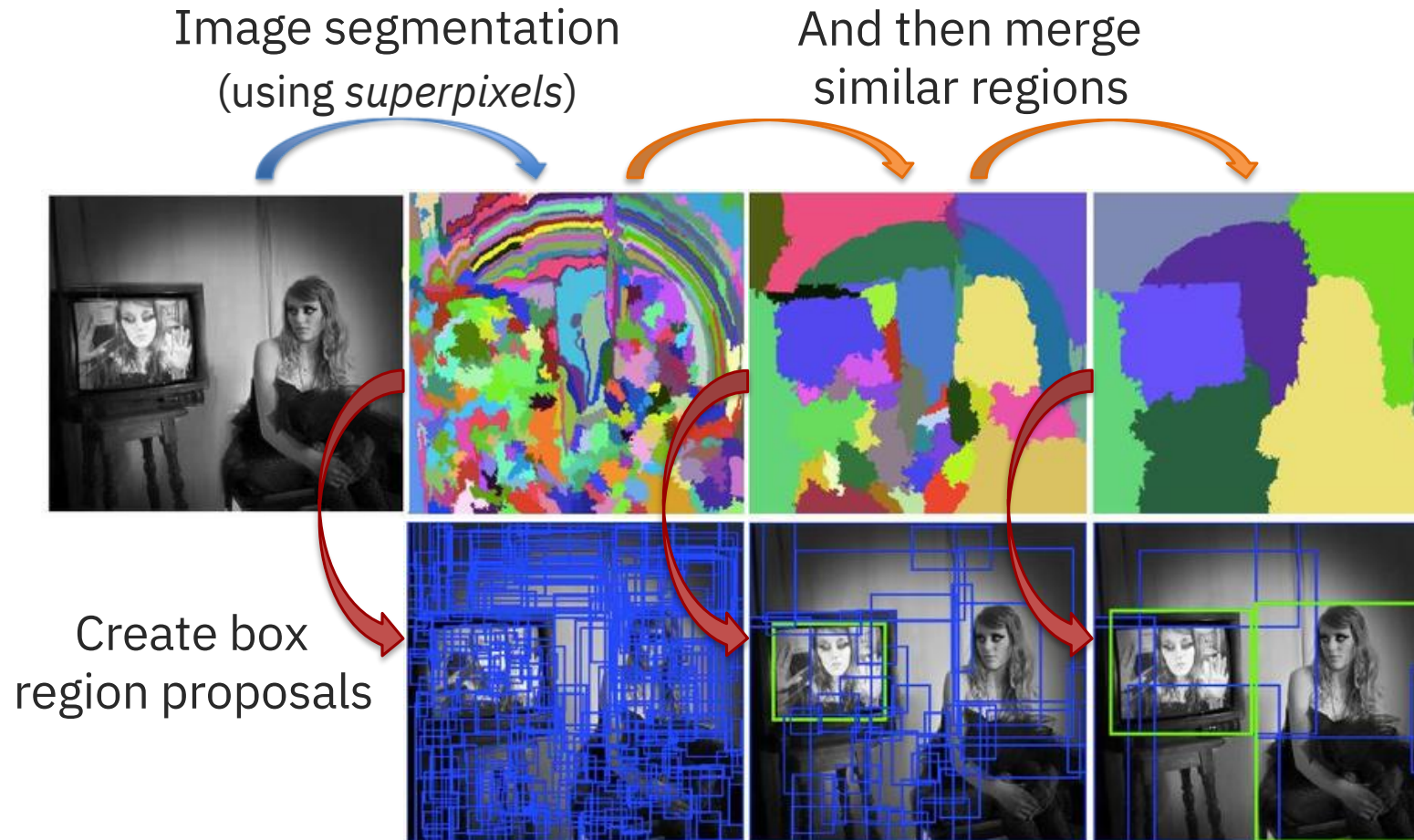
Object Detection (and Segmentation)



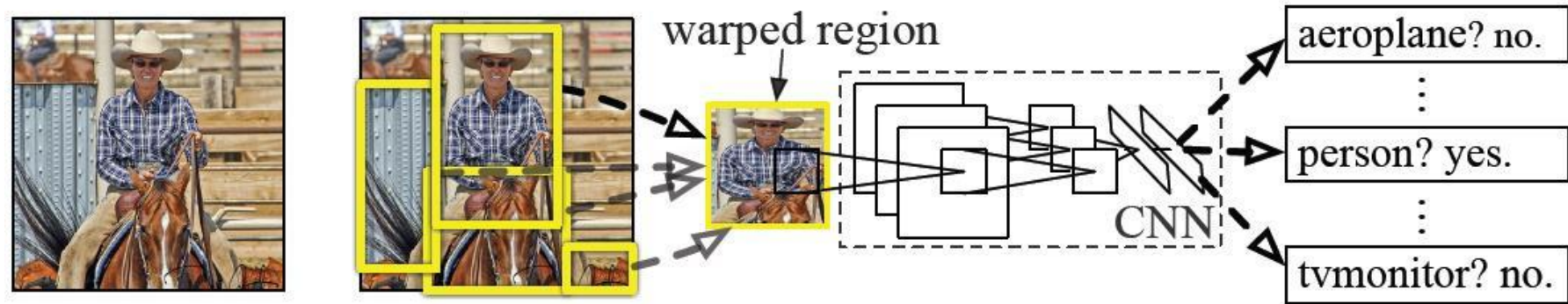
A better option: Start by Identifying hundreds of region proposals and then apply our CNN object detector

How to efficiently identify region proposals?

Selective Search [Uijlings et al., IJCV 2013]



R-CNN [Girshicket al., CVPR 2014]



- Select ~2000 region proposals ➡ Time consuming!
- Warp each region Apply CNN to
- each region ➡ Time consuming!

Fast R-CNN: Applies CNN only once, and then extracts regions

Faster R-CNN: Region selection on the Conv5 response map

Word Representations

Simple Word Representation

Written language

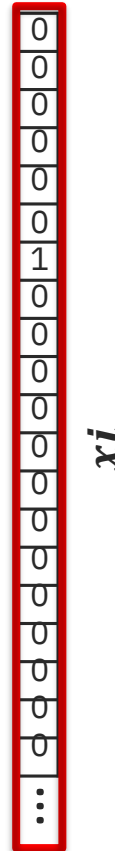
★★★★★ Masterful!

By Antony Witheyman - January 12, 2006

Ideal for anyone with an interest in disguises who likes to see the subject tackled in a **humourous** manner.

0 of 4 people found this review helpful

Input observation



x_i

“one-hot” vector

$|x_i|$ = number of words in dictionary

What is the meaning of “bardiwac”?

■ ■ ■ ■

He handed her glass of bardiwac.

Beef dishes are made to complement the bardiwacs. Nigel staggered to his feet, face flushed from too much bardiwac. Malbec, one of the lesser-known bardiwac grapes, responds well to Australia’s sunshine.

- I dined off bread and cheese and this excellent bardiwac.
- The drinks were delicious: blood-red bardiwac as well as light, sweet Rhenish.

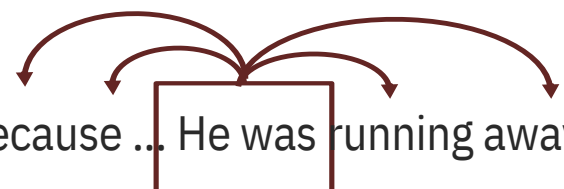
□ bardiwac is a heavy red alcoholic beverage made from grapes

How to learn (word) features/representations?

➡ Distribution hypothesis: Approximate the word meaning by its surrounding words

➡ Words used in a similar context will lie close together

He was walking away because ... He was running away because ...



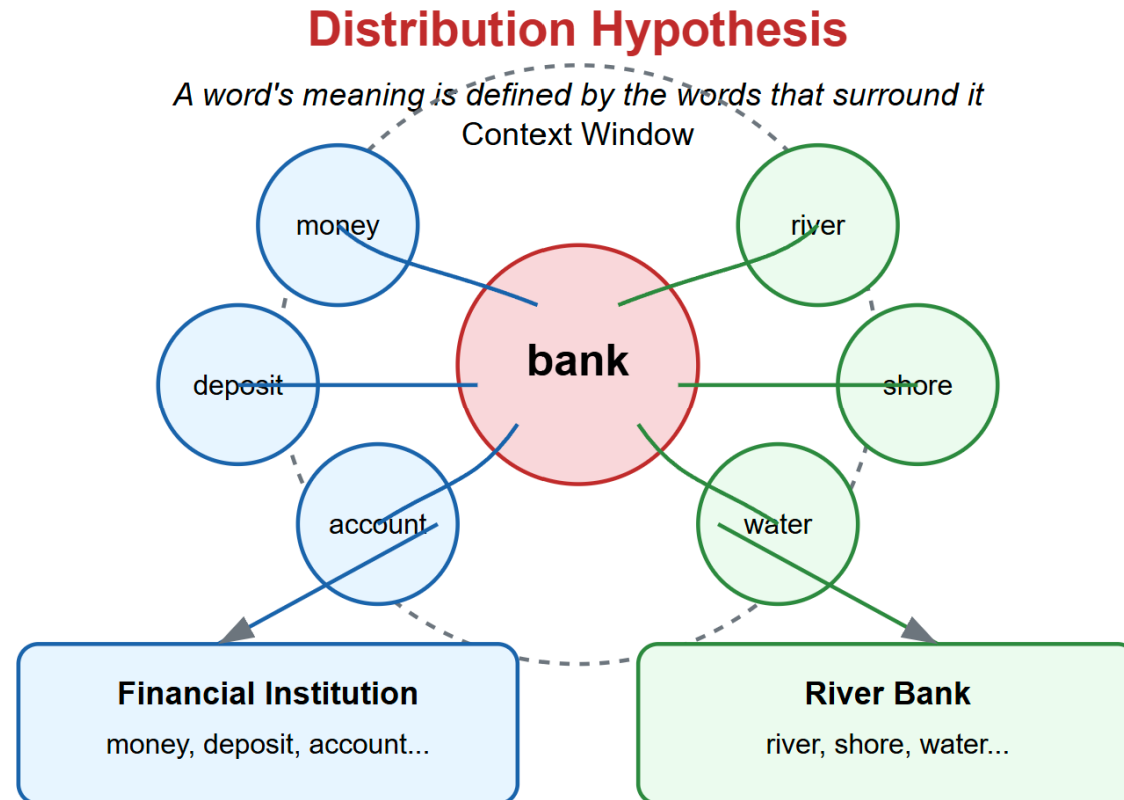
Instead of capturing co-occurrence counts directly,

➡ **predict surrounding words of every word**

$$\frac{1}{T} \sum_{t=1}^T \sum_{-c \leq j \leq c, j \neq 0} \log p(w_{t+j} | w_t)$$

How to learn (word) features/representations?

➡ Distribution hypothesis: Approximate the word meaning by its surrounding words



How to learn (word) features/representations?

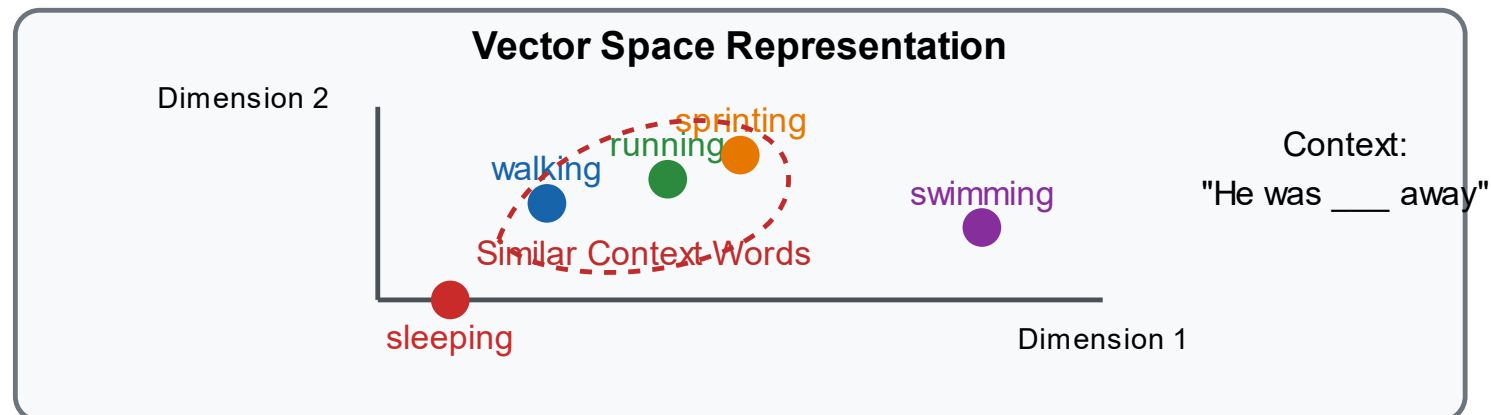
Words in Similar Context

Words used in a similar context will lie close together in vector space

He was **walking** away because the weather was bad.

He was **running** away because the police arrived.

He was **sprinting** away because he was late.



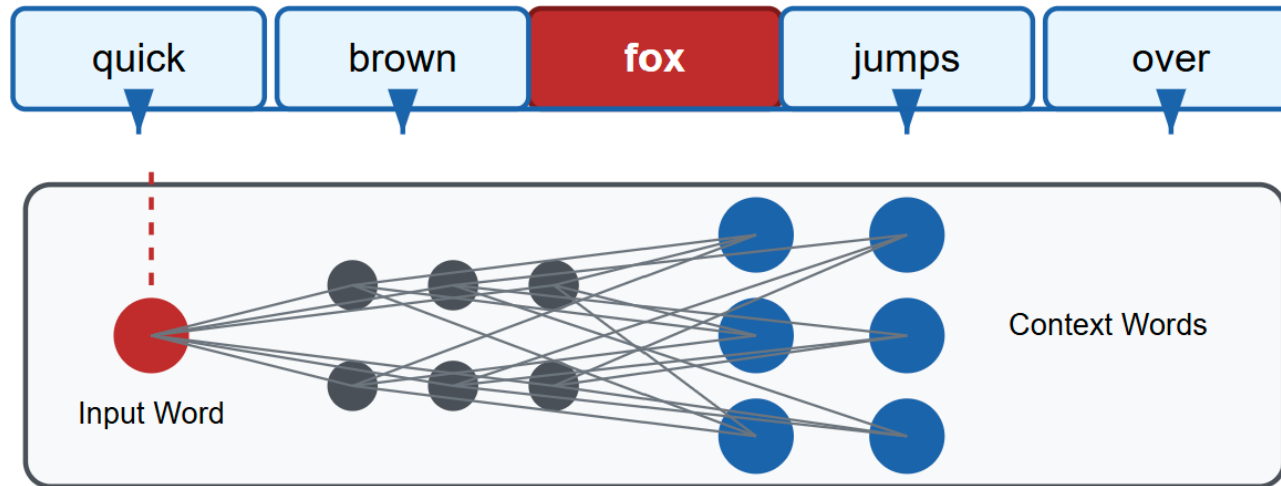
How to learn (word) features/representations?

Word2Vec: Predict Surrounding Words

Instead of counting co-occurrences, predict context from target word

"The quick brown fox jumps over the lazy dog"

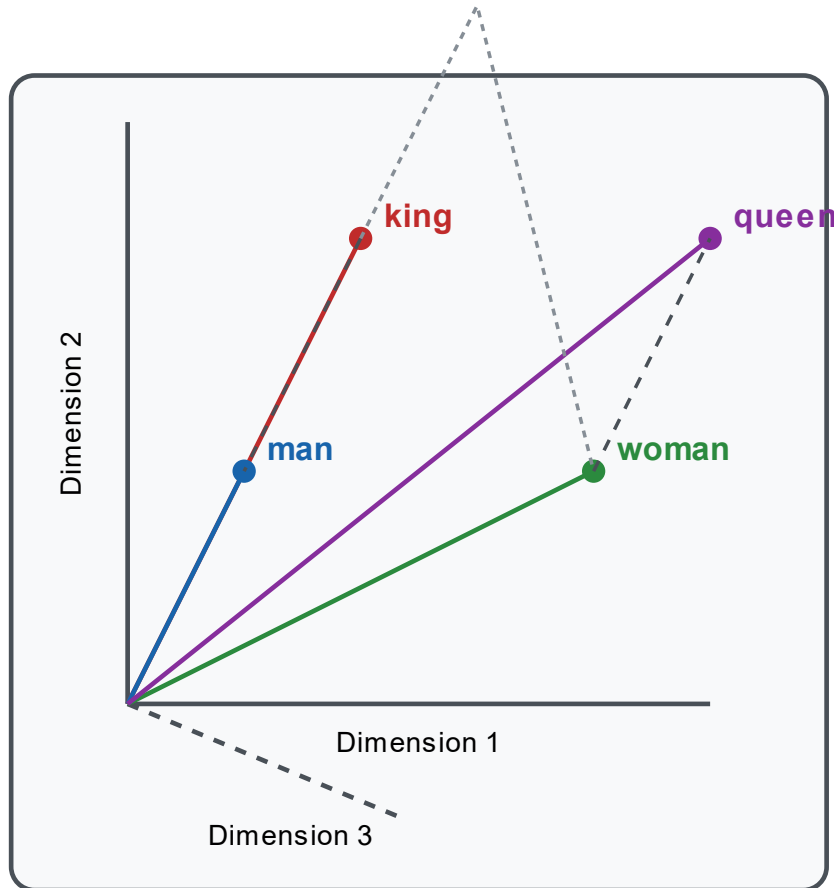
Skip-gram Model



$$\frac{1}{T} \sum_{t=1}^T \sum_{-c \leq j \leq c, j \neq 0} \log p(w_{t+j} | w_t)$$

How to learn (word) features/representations?

Word Embeddings and Vector Operations



Vector Analogies

king - man + woman $\approx$ queen

Examples of other analogies:

Paris - France + Italy $\approx$ Rome

car - road + water $\approx$ boat

Einstein - scientist + artist $\approx$ Picasso

Words with similar meanings have similar vector representations

Geometric interpretation

- row vector $\mathbf{x}_{\text{dog}}$ describes usage of Word *dog* in the corpus
- can be seen as Coordinates of point in n -dimensional Euclidean space R^n

	get	see	use	hear	eat	kill
knife	51	20	84	0	3	0
cat	52	58	4	4	6	26
dog	115	83	10	42	33	17
boat	59	39	23	4	0	0
cup	98	14	6	2	1	0
pig	12	17	3	2	9	27
banana	11	2	2	0	18	0

co-occurrence matrix M

Geometric interpretation

Geometric Interpretation of Word Vectors

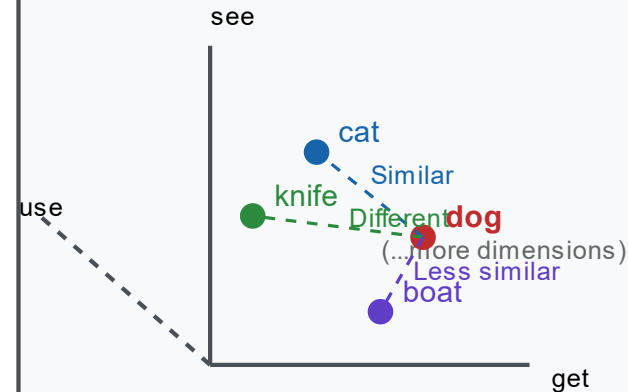
Co-occurrence Matrix M

Word	get	see	use	hear	eat
knife	51	20	84	0	3
cat	52	58	4	4	6
dog	115	83	10	42	33
boat	59	39	23	4	0
cup	98	14	6	2	1

$x_{\text{dog}} = [115, 83, 10, 42, 33]$

Row vector describes usage pattern

n-dimensional Euclidean Space



Geometric Interpretation:

- Each word is a point in n-dimensional space (n = number of context words)
 - Co-occurrence patterns determine the position in this space
 - Words with similar contexts are closer together in this space

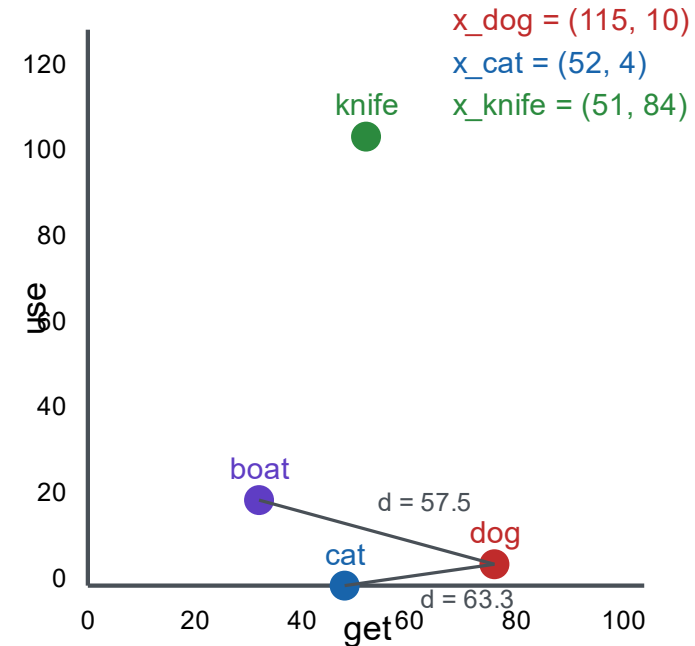
Distance and similarity

Distance and Similarity in Word Vector Space

Key Concepts:

- Illustrated for two dimensions:
get and
use : $\mathbf{x}_{\text{dog}} = (115, 10)$
- Similarity = spatial proximity
(Euclidean distance)
- Location depends on frequency
of noun: $f_{\text{dog}} \gg f_{\text{cat}}$
- Other similarity measures:
 - Cosine similarity
 - Dot product
 - Manhattan distance

Two dimensions of English V-Obj DSM



$$\text{Euclidean distance: } d(\mathbf{x}, \mathbf{y}) = \sqrt{\sum_i (\mathbf{x}_i - \mathbf{y}_i)^2}$$

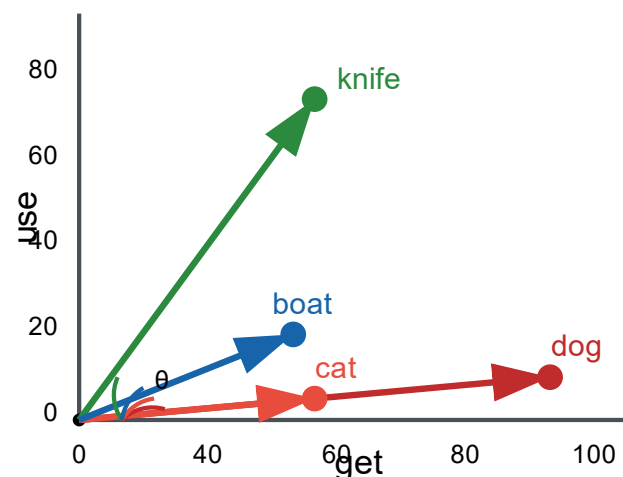
Angle and similarity

Angle and Similarity in Word Vectors

Key Concepts:

- Direction more important than location
- Normalize "length" (magnitude) of vector: $\| \mathbf{x}_{\text{dog}} \|$
- Use angle as distance measure:
Cosine similarity =
 $\cos(\theta) = \mathbf{A} \cdot \mathbf{B} / (\|\mathbf{A}\| \cdot \|\mathbf{B}\|)$
- Advantages:
 - Ignores magnitude differences
 - Works well for high dimensions
 - Focus on pattern similarity

Two dimensions of English V-Obj DSM



Normalized vectors have length 1 but preserve direction

Cosine Similarity

- $\text{sim}(\text{dog}, \text{cat}) = \cos(\theta) = 0.97$ (small angle)
- $\text{sim}(\text{dog}, \text{knife}) = \cos(\theta) = 0.34$ (large angle)

Skip-gram & CBOW

CONTENTS

1. $F = Wx$
 2. Skip-gram
 3. CBOW
-

$$F = Wx$$

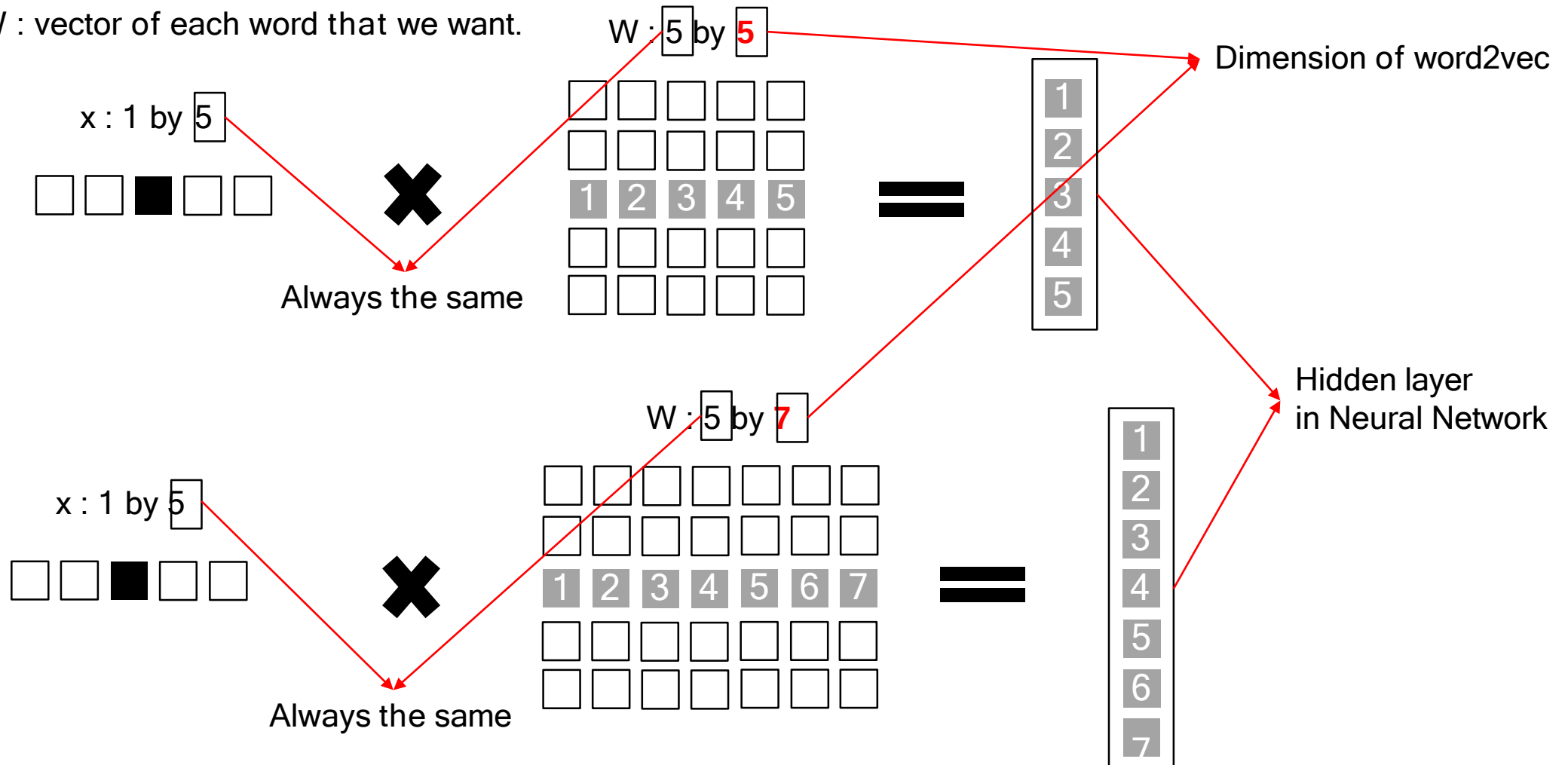
Skip-gram & CBOW

$F=Wx$

Skip-gram

CBOW

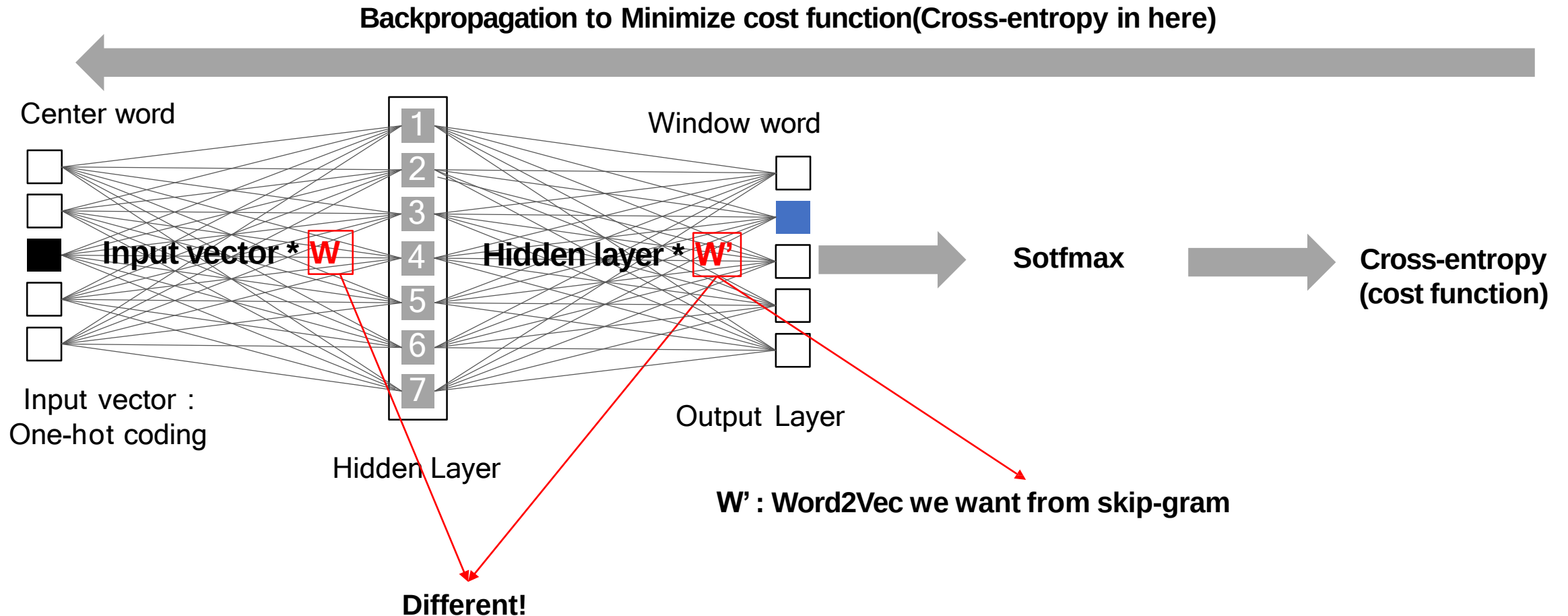
- $F = Wx$
- x : one-hot vector of Vocabularies.
- W : vector of each word that we want.



Skip-gram

Skip-gram & CBOW

- Let me explain the architecture of skip-gram.



Skip-gram & CBOW

- Let's say, our vocabulary is {I, like, the, natural, language, processing} from a sentence, "I like the natural language processing". and the size of windows is 1.
- a pair consists of {center word, window word skipped}

I	like	the	natural	language	processing	{I, like}
---	------	-----	---------	----------	------------	-----------

A sample for an
example of skip-gram

I	like	the	natural	language	processing	{like, I}, {like, the}
---	------	-----	---------	----------	------------	------------------------

I	like	the	natural	language	processing	{the, like}, {the, natural}
---	------	-----	---------	----------	------------	-----------------------------

I	like	the	natural	language	processing	{natural, the}, {natural, language}
---	------	-----	---------	----------	------------	-------------------------------------

I	like	the	natural	language	processing	{language, natural}, {language, processing}
---	------	-----	---------	----------	------------	---

I	like	the	natural	language	processing	{processing, language}
---	------	-----	---------	----------	------------	------------------------

F=Wx

Skip-gram

CBOW

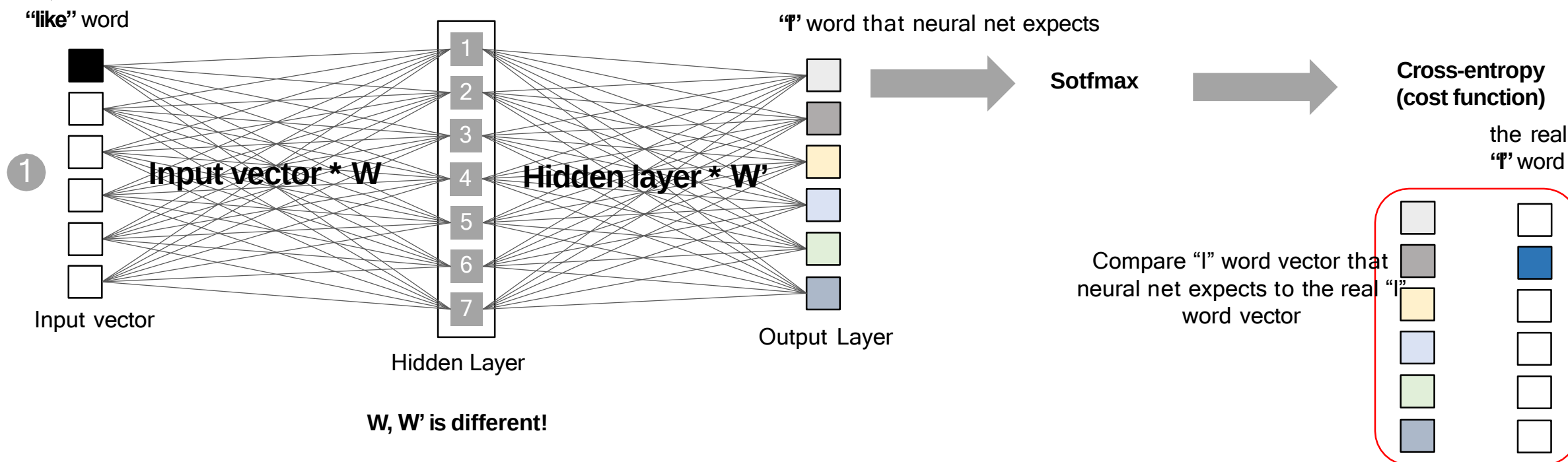
Skip-gram & CBOW

I	like	the	natural	language	processing	{like, I}, {like, the}
---	------	-----	---------	----------	------------	------------------------

A sample for an example of skip-gram

	I	like	the	natural	language	processing
One-hot vector of "I"	1	0	0	0	0	0
One-hot vector of "like"	0	1	0	0	0	0
One-hot vector of "the"	0	0	1	0	0	0

Backpropagation to Minimize cost function(Cross-entropy in here)



F=Wx

Skip-gram

CBOW

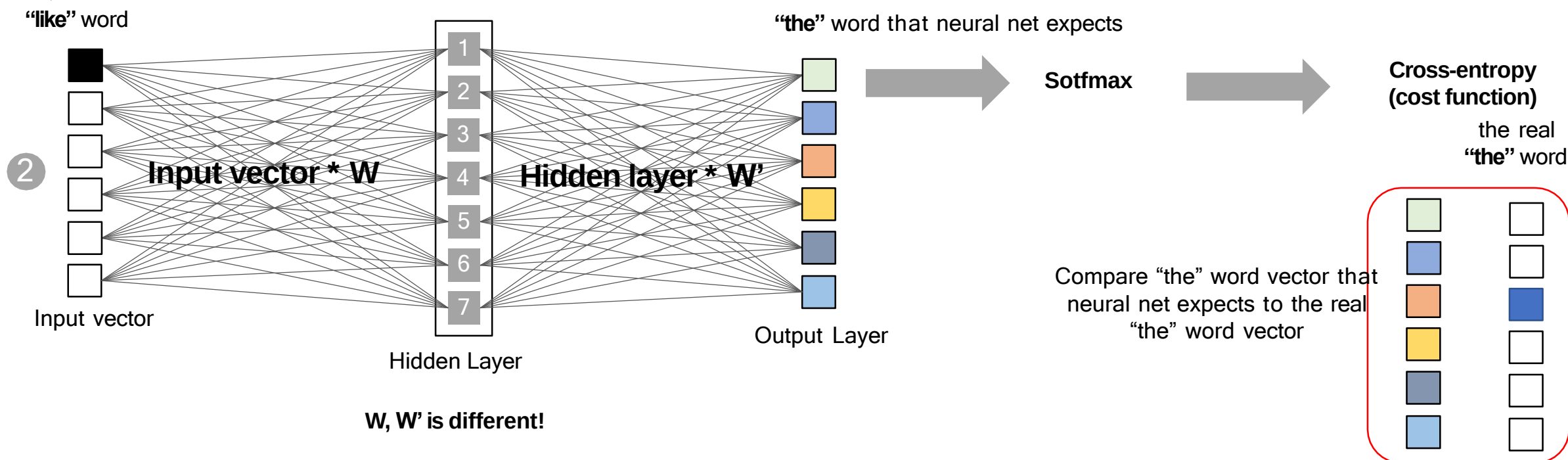
Skip-gram & CBOW

I	like	the	natural	language	processing	{like, I}, {like, the}
---	------	-----	---------	----------	------------	------------------------

A sample for an example of skip-gram

	I	like	the	natural	language	processing
One-hot vector of "I"	1	0	0	0	0	0
One-hot vector of "like"	0	1	0	0	0	0
One-hot vector of "the"	0	0	1	0	0	0

Backpropagation to Minimize cost function(Cross-entropy in here)

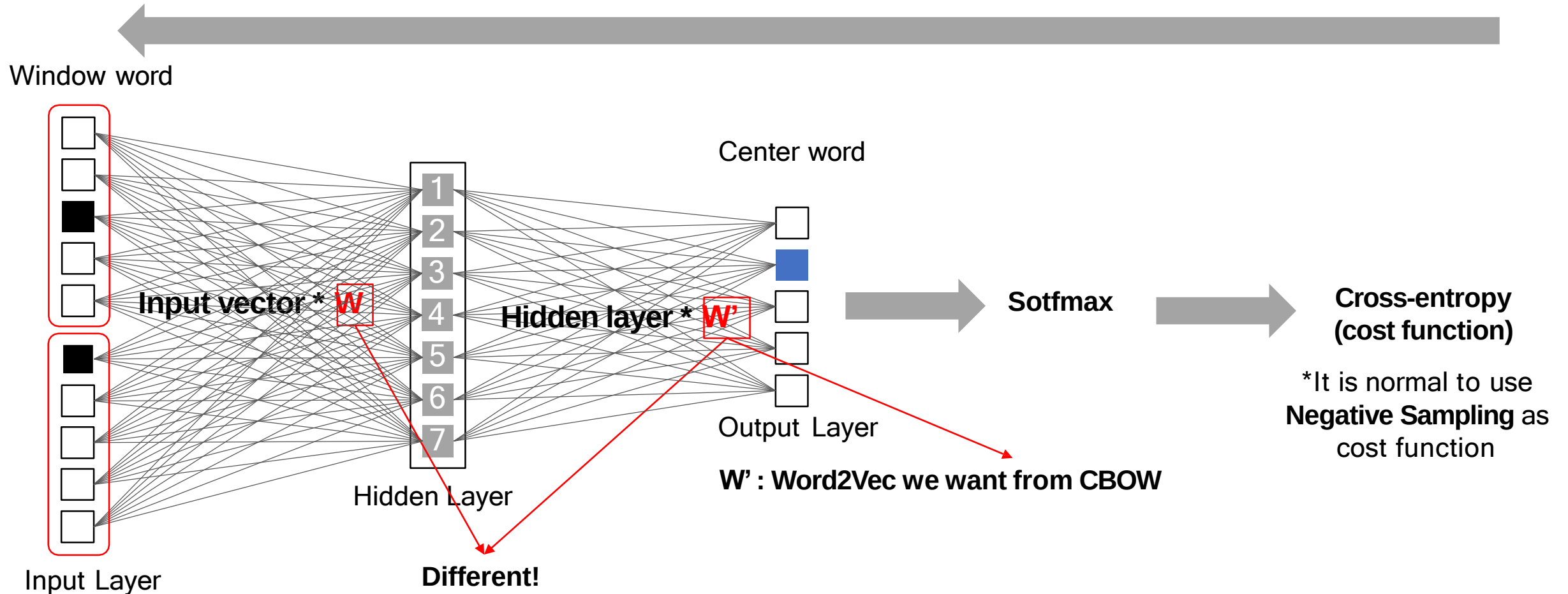


CBOW

Skip-gram & CBOW

- Let me explain the architecture of Continuous Bag-of-Words.

Backpropagation to Minimize cost function(Cross-entropy in here)



Skip-gram & CBOW

· Let's say, our vocabulary is {I, like, the, NLP, programming} from a sentence, "I like the NLP programming". and the size of windows is 1.

- a pair consists of {[window word], center word}

I	like	the	NLP	programming	{ [like], I }
---	------	-----	-----	-------------	---------------

I	like	the	NLP	programming	{ [I, the], like }	A sample for an example of CBOW
---	------	-----	-----	-------------	--------------------	---------------------------------

I	like	the	NLP	programming	{ [like, NLP], the }
---	------	-----	-----	-------------	----------------------

I	like	the	NLP	programming	{ [the, programming], natural }
---	------	-----	-----	-------------	---------------------------------

I	like	the	NLP	programming	{ [NLP], language }
---	------	-----	-----	-------------	---------------------

F=Wx

Skip-gram

CBOW

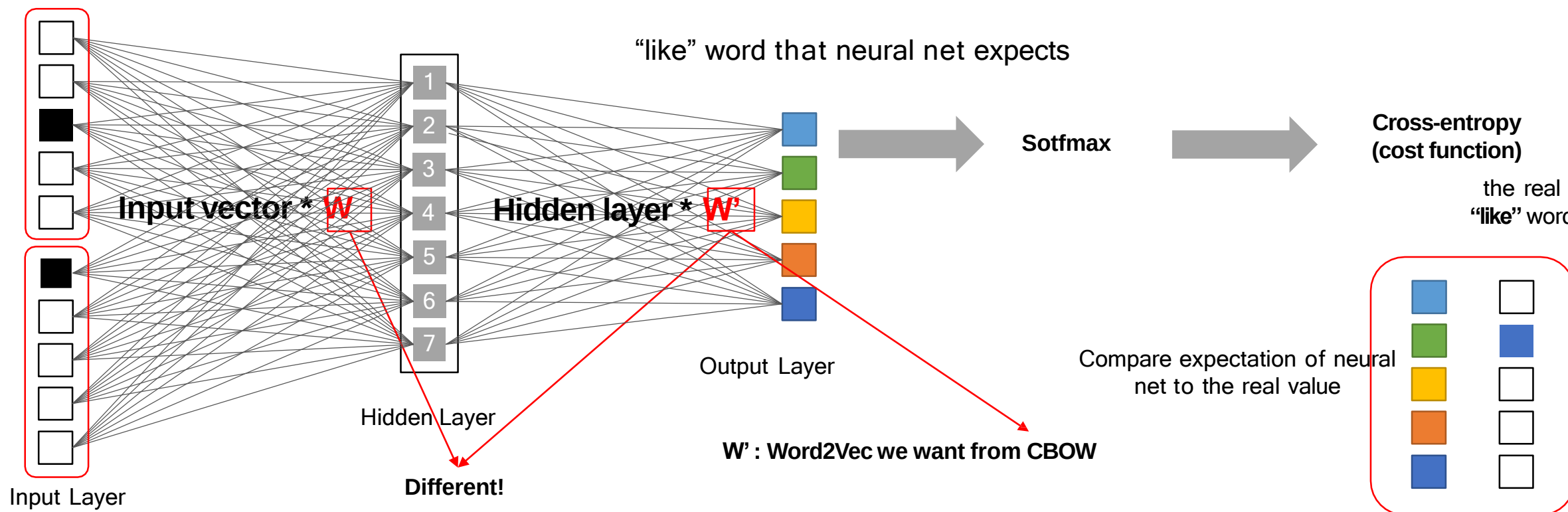
Skip-gram & CBOW



Backpropagation to Minimize cost function(Cross-entropy in here)

"I" word & "the" word

"like" word that neural net expects



Word Representation Resources

Word-level representations:

Word2Vec (Google, 2013)

<https://code.google.com/archive/p/word2vec/>

Glove (Stanford, 2014)

<https://nlp.stanford.edu/projects/glove/>

FastText(Facebook, 2017)

<https://fasttext.cc/>

Sentence-level representations:

ELMO (Allen Institute for AI, 2018)

<https://allennlp.org/elmo>

BERT (Google, 2018)

<https://github.com/google-research/bert>

RoBERTa(Facebook, 2019)

<https://github.com/pytorch/fairseq>

Word representations
are contextualized
using all the words in
the sentence.



More details later
in this lecture

Lexicon-based Word Representation

LIWC: Language Inquiry & Word Count

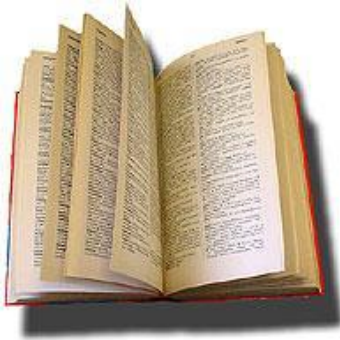
Manually created dictionaries for different topics and categories:

- Function words: *pronouns, preposition, negation...*
- Affect words: *positive, negative emotions*
- Social words: *family, friends, referents*
- Cognitive processes: *Insight, cause, ...*
- Perceptual processes: *Seeing, hearing, feeling*
- Biological processes: *Body, health/illness,...*
- Drives and needs: *Affiliation, achievement, ...*
- Time orientation: *past, present, future*
- Relativity: *motion, space, time*
- Personal concerns: *work, leisure, money, religion ...*
- Informal speech: *swear words, fillers, assent,...*
-

LIWC can encode individual words or full sentences.

<https://www.liwc.app/>

Other Lexicon Resources



Lexicons

- General Inquirer (Stone et al., 1966)
- OpinionFinderlexicon (Wiebe & Riloff, 2005)
- SentiWordNet(Esuli& Sebastiani, 2006)
- LIWC (Pennebaker)



Other Tools

- LightSIDE
- Stanford NLP toolbox
- IBM Watson Tone Analyzer
- Google Cloud Natural Language
- Microsoft Azure Text Analytics

Sentence Modeling and Recurrent Networks

Sentence to Vector Transformation

The movie was fantastic!



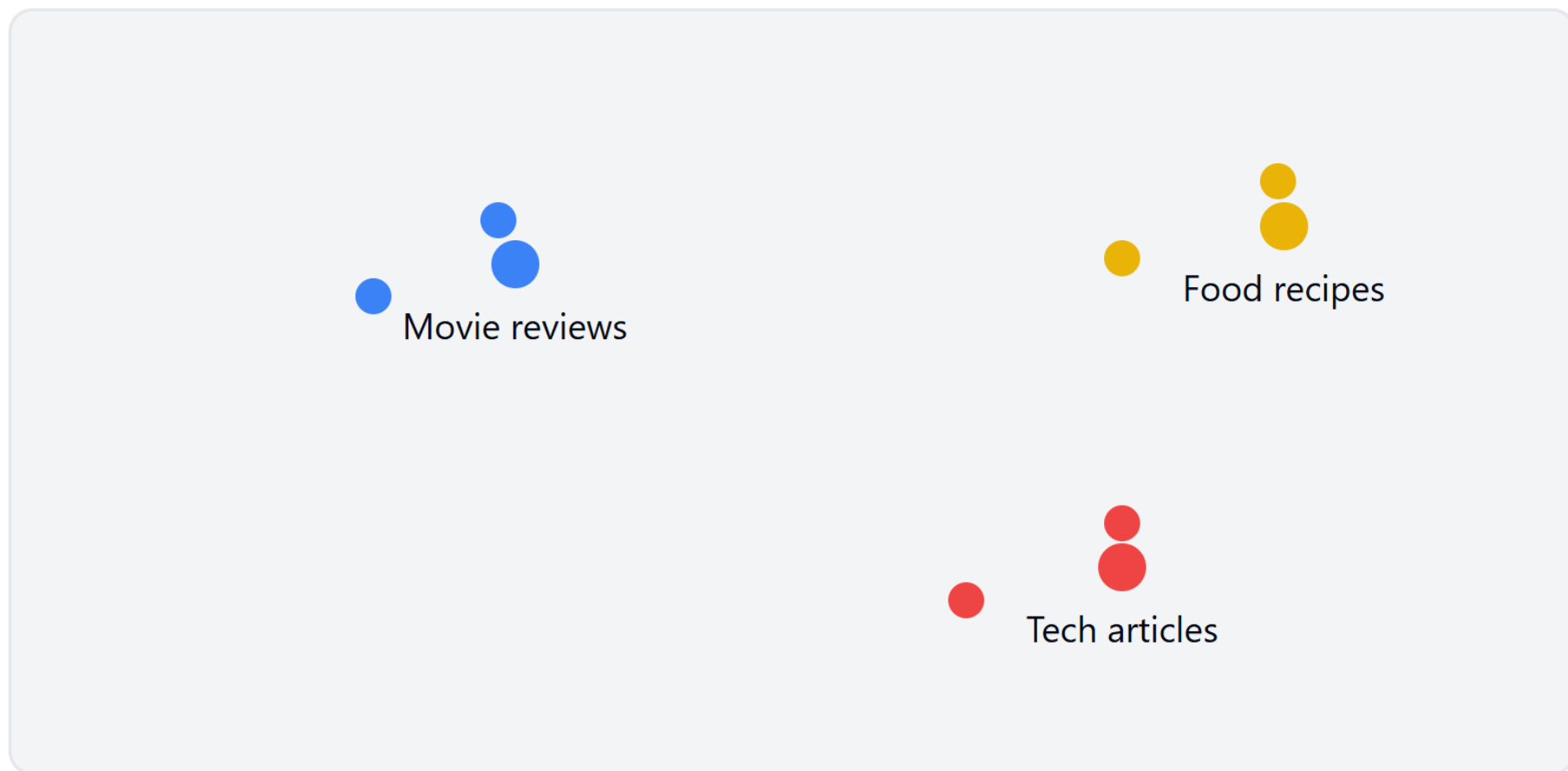
[0.28, -0.61, 0.73, 0.42, ...]

I really enjoyed the film.



[0.3, -0.58, 0.69, 0.45, ...]

Semantic Similarity in Vector Space

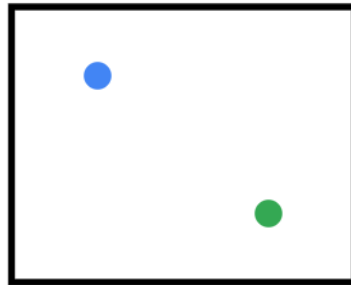


Sentences with similar meanings are clustered together in the embedding space.

Semantic Similarity in Vector Space



1D: Points well distributed



2D: Still good coverage

300+ dimensions:

- Most of space is empty
- Distances less meaningful
- Computation expensive

Embedding Approaches Evolution

Word Vector Averaging

Simple but loses word order and relationships

"The cat sat on the mat" → Average of word vectors

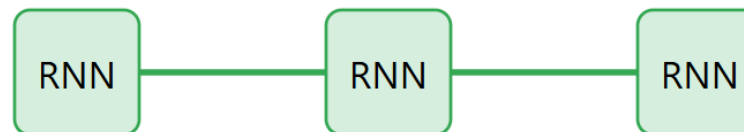
TF-IDF Weighted

Gives more weight to important words

"The ($\times 0.1$) cat ($\times 0.8$) sat ($\times 0.7$) on ($\times 0.1$) the ($\times 0.1$) mat ($\times 0.6$)"

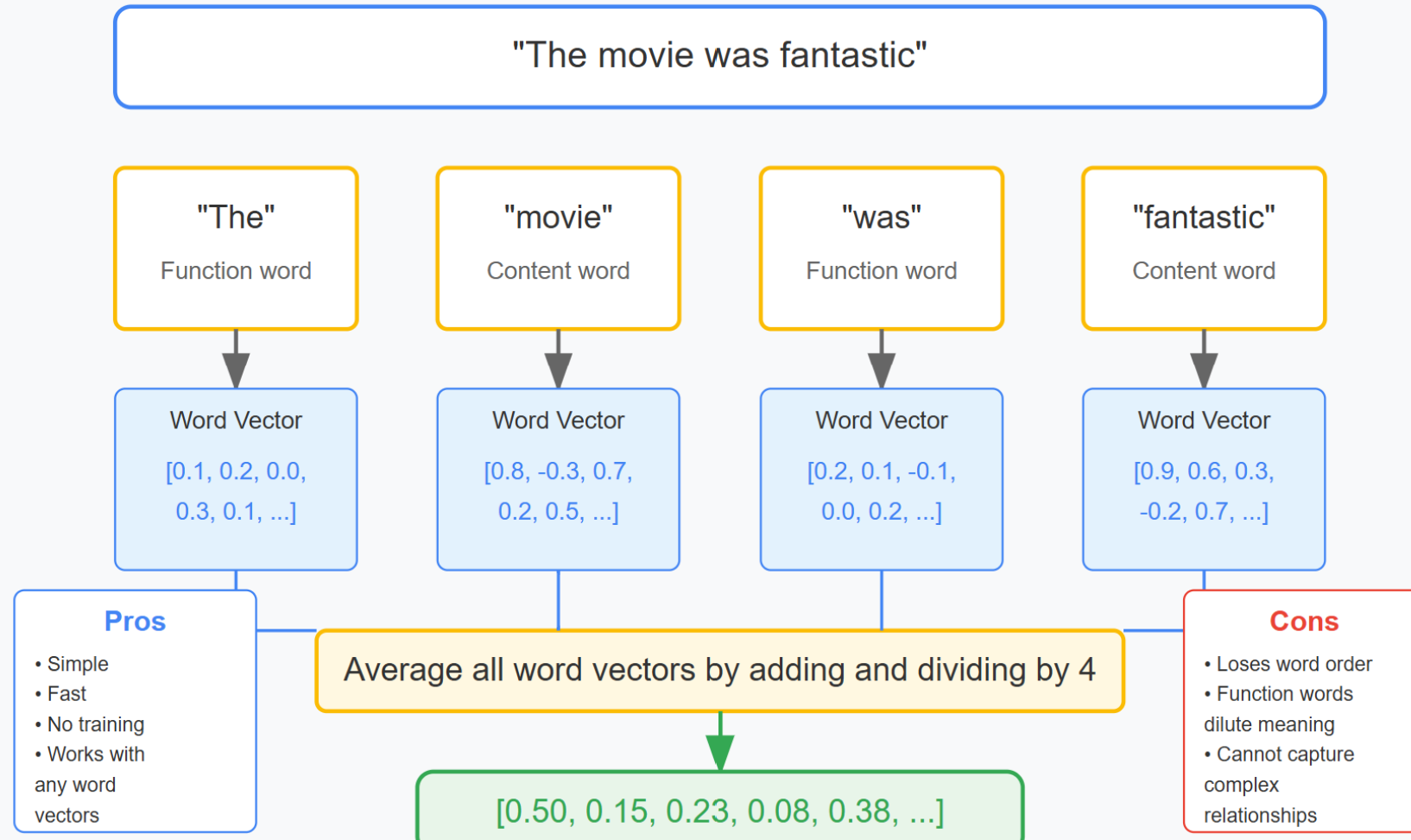
Recurrent Neural Networks

Process text sequentially to maintain order



Embedding Approaches Evolution

Word Vector Averaging for Sentence Embeddings



TF-IDF Weighted Approach for Sentence Embeddings

TF-IDF Weighted Approach for Sentence Embeddings

Sentence: "The quick brown fox jumps over the lazy dog"

Words with TF-IDF Weights:

"the"	"quick"	"brown"	"fox"	"jumps"	"lazy"
0.1	0.7	0.6	0.8	0.6	0.7

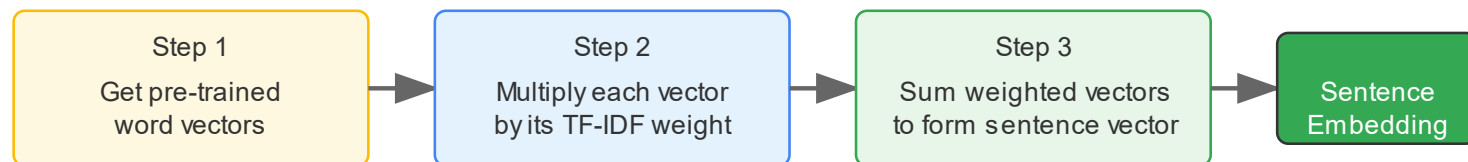
Common words (less important) = lower weights

What is TF-IDF?

- TF (Term Frequency): How often a word appears in a document
- IDF (Inverse Document Frequency): How rare a word is across all documents

$$\text{TF-IDF} = \text{TF} \times \text{IDF}$$

Embedding Generation Process



TF-IDF Weighted Approach for Sentence Embeddings

Understanding the TF-IDF Formula

$$\text{TF-IDF}(t,d) = \text{TF}(t,d) \times \text{IDF}(t)$$

Term Frequency (TF)

How frequently a term appears in a document

$$\text{TF}(t,d) = (\text{Count of } t \text{ in } d) / (\text{Total words in } d)$$

Example:

Document: "The quick fox jumps over the lazy dog"

$$\text{TF}(\text{"fox"}) = 1/8 = 0.125$$

$$\text{TF}(\text{"the"}) = 2/8 = 0.25$$

Lower for common words

Inverse Document Frequency (IDF)

How rare a term is across all documents

$$\text{IDF}(t) = \log(N/\text{df}_t)$$

N = total documents, df_t = documents containing t

Example:

Total documents: 1,000,000

Documents with "fox": 1,000

$$\text{IDF}(\text{"fox"}) = \log(1,000,000/1,000) = \log(1000) \approx 6.9$$

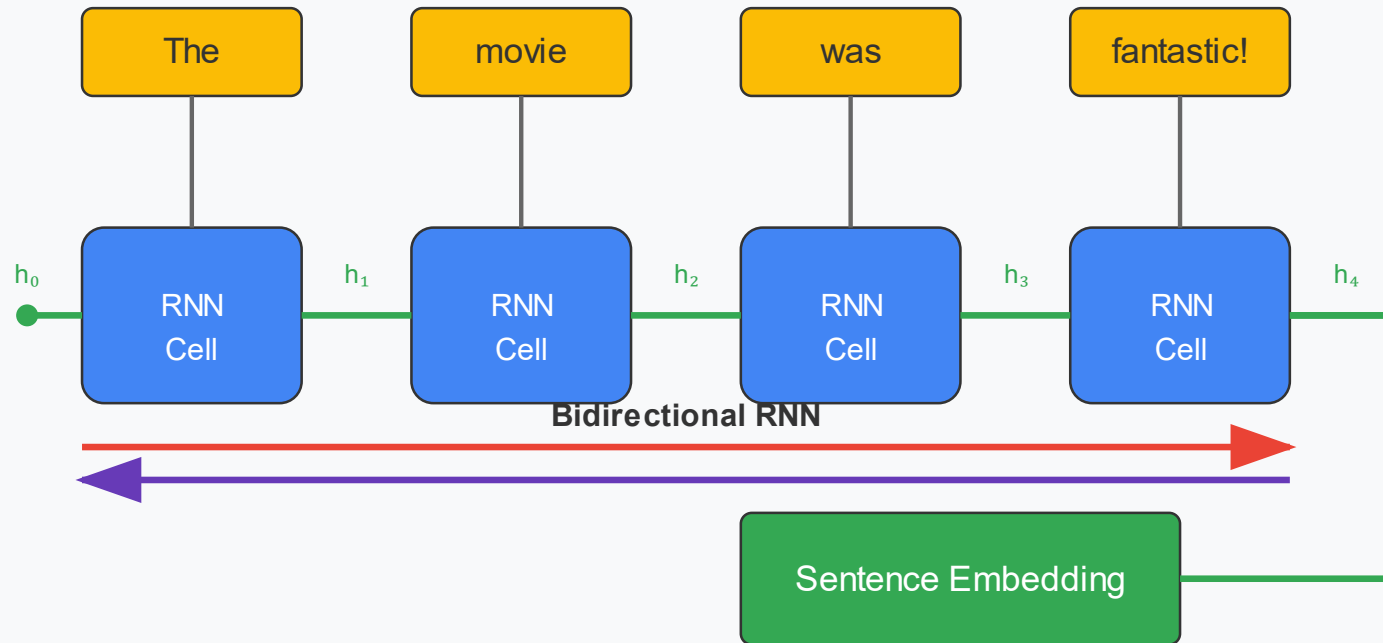
Higher for rare, distinctive words

$$\text{TF-IDF}(\text{"fox"}) = 0.125 \times 6.9 = 0.8625$$

Recurrent Neural Networks for Sentence Embeddings

Recurrent Neural Networks for Sentence Embeddings

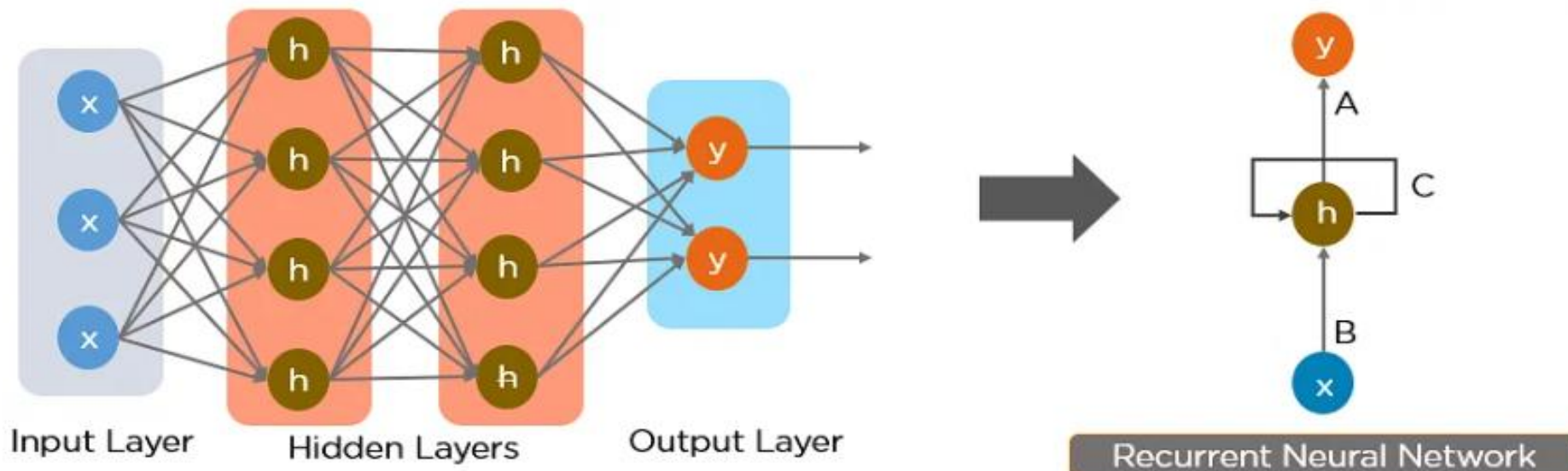
Input Sentence: "The movie was fantastic!"



Recurrent Neural Network

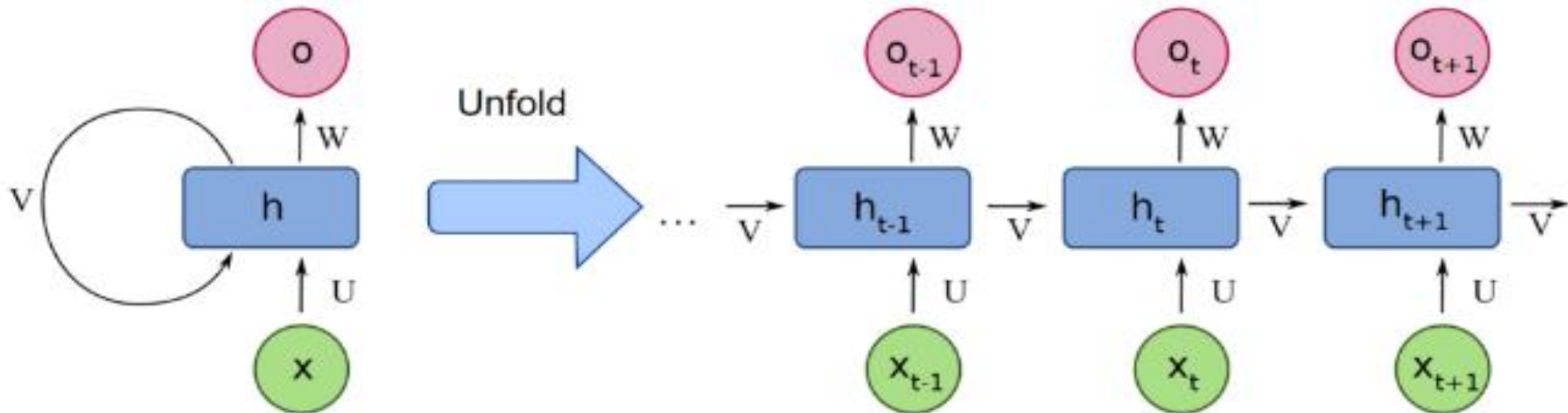
Recurrent Neural Network (RNN)

- An artificial neural network adapted to work for **time series data** or **data that involves sequences**.
- Uses a **Hidden Layer** that remembers specific information about a sequence
- RNN has a **Memory** that stores all information about the calculations.
- Formed from **Feed-forward Networks**



Recurrent Neural Network (RNN)

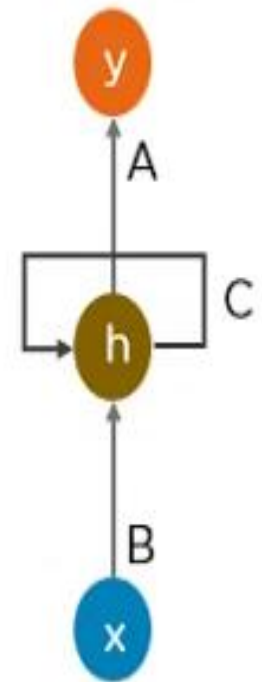
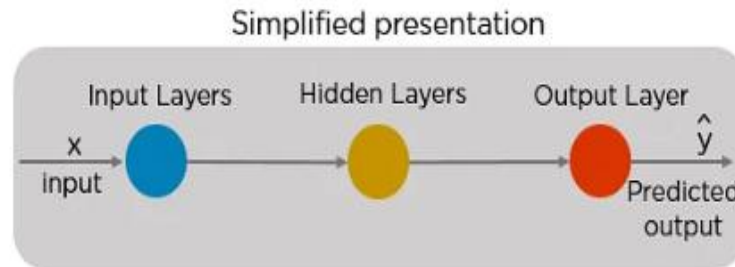
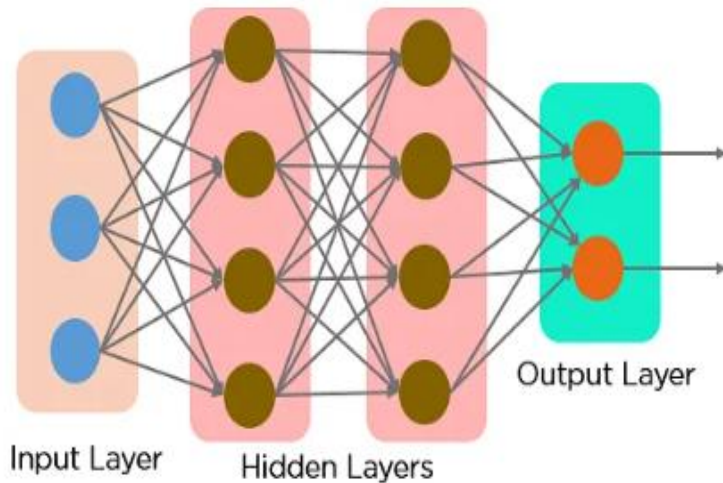
- Uses the **same weights** for each element of the sequence
- Need to **inform about the previous inputs** before evaluating the result
- **Comparing that result to the expected value** will give us an error
- **Propagating the error back through the same path** will adjust the variables.



Why Recurrent Neural Networks?

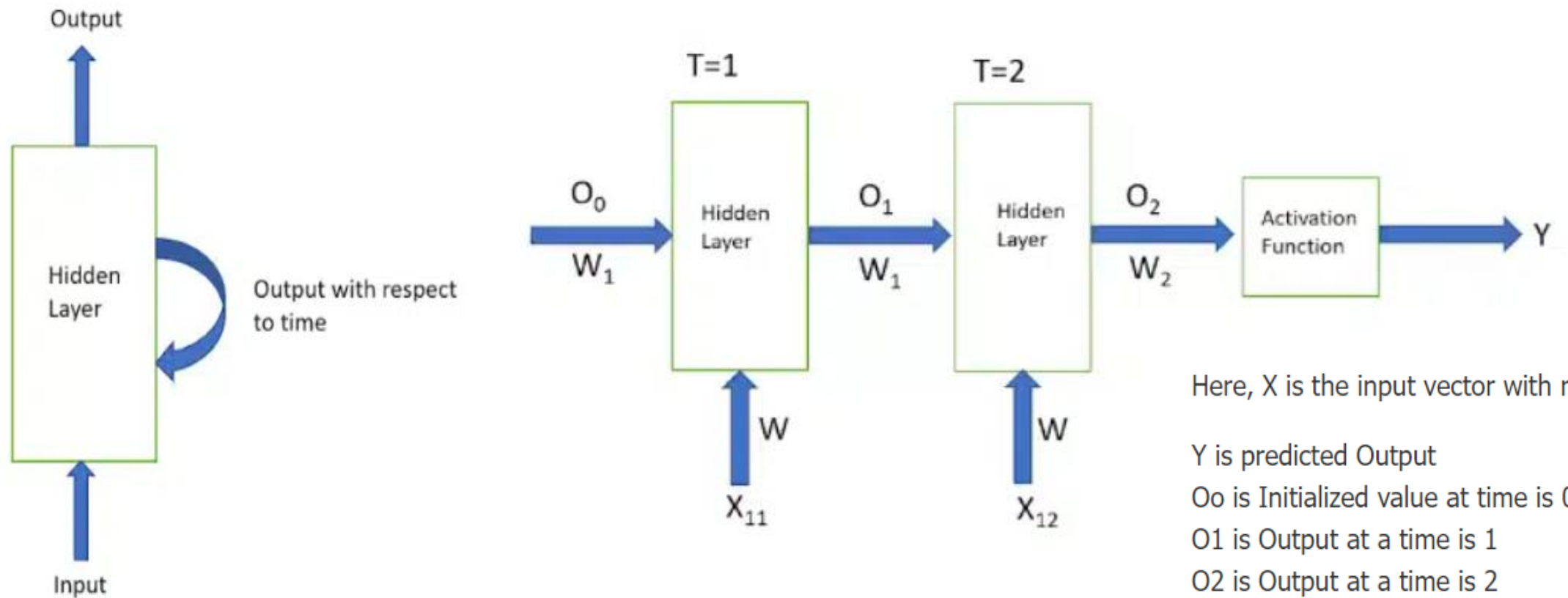
RNN were created because there were a few issues in the feed-forward neural network

- Cannot handle sequential data
- Considers only the current input
- Cannot memorize previous inputs
- Loss of neighborhood information.
- Does not have any loops or circles.



Recurrent Neural Network

Architecture of RNN



Here, X is the input vector with respect to time

Y is predicted Output

O_0 is Initialized value at time is 0

O_1 is Output at a time is 1

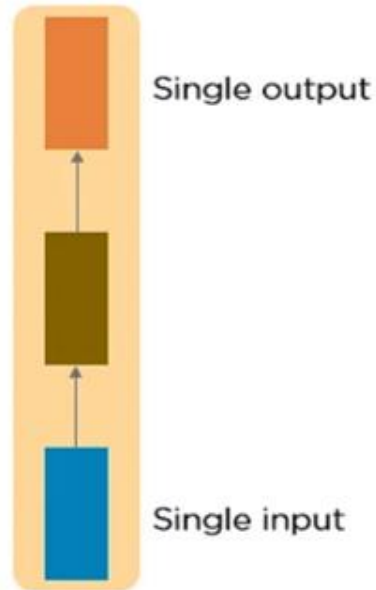
O_2 is Output at a time is 2

T represents the time

W is weight

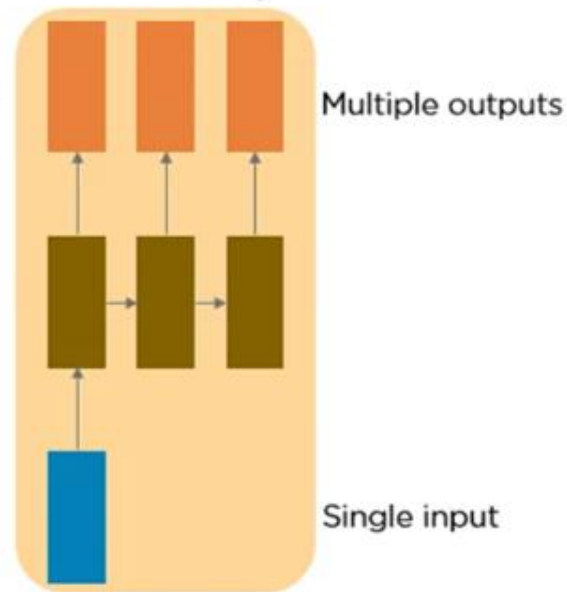
Types of Recurrent Neural Networks

One to One



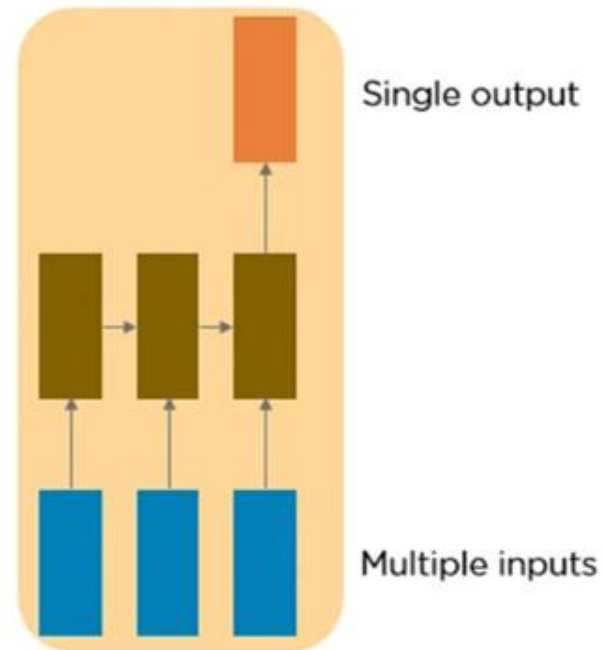
It's used for **GENERAL
MACHINE LEARNING
PROBLEMS**

One to Many



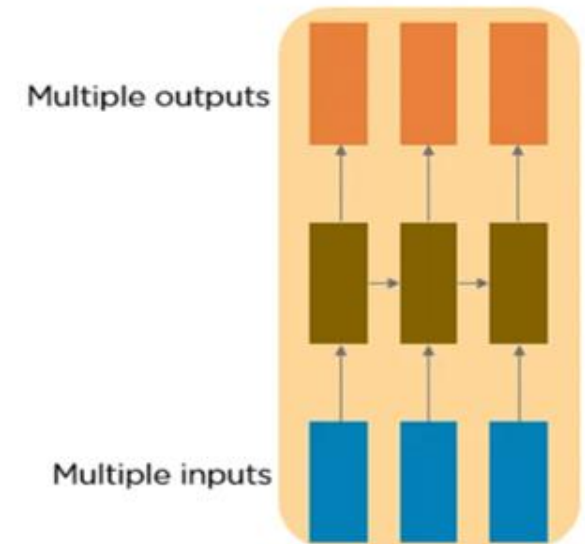
It's used for
IMAGE CAPTION.

Many to One



It's used for
SENTIMENT ANALYSIS.

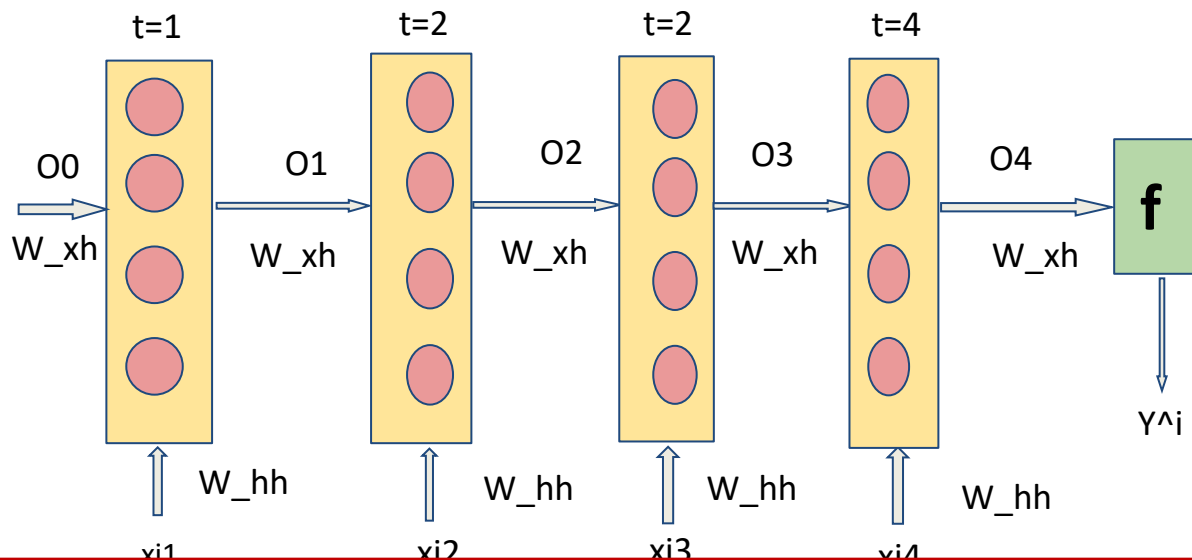
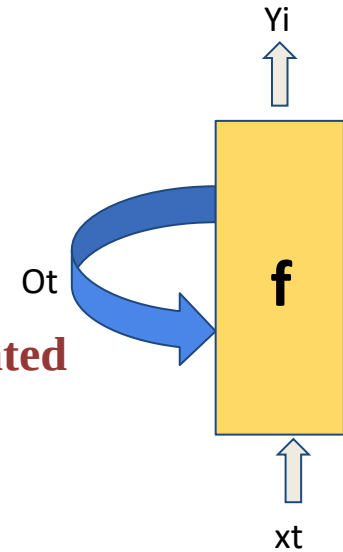
Many to Many



It's used for
MACHINE TRANSLATION.

Steps for training a RNN

- Initial input is sent with same **weight** and **activation function**.
- Current state calculated by using **current input** & **previous state output**
- Current state **X_t becomes X_{t-1}** for second time step.
- Keeps on repeating for all the steps
- Final step calculated by current state of final state and all other previous steps.
- An error is generated by calculating the difference between the **actual output** and **generated output** by RNN model.
- Final step is when the process of back propagation occurs



Recurrance formula

$$ht = fw(ht-1, xt)$$

ht= new hidden state

fw= some functions of parameter w

ht-1= old state

xt= input vector at some time spent

$$O1 = f(Xi1w_{hh} + O0W_{xh})$$

$$O2 = f(Xi2w_{hh} + O1W_{xh})$$

$$O3 = f(Xi3W_{hh} + O2W_{xh})$$

$$O4 = f(Xi4W_{hh} + O3W_{xh})$$

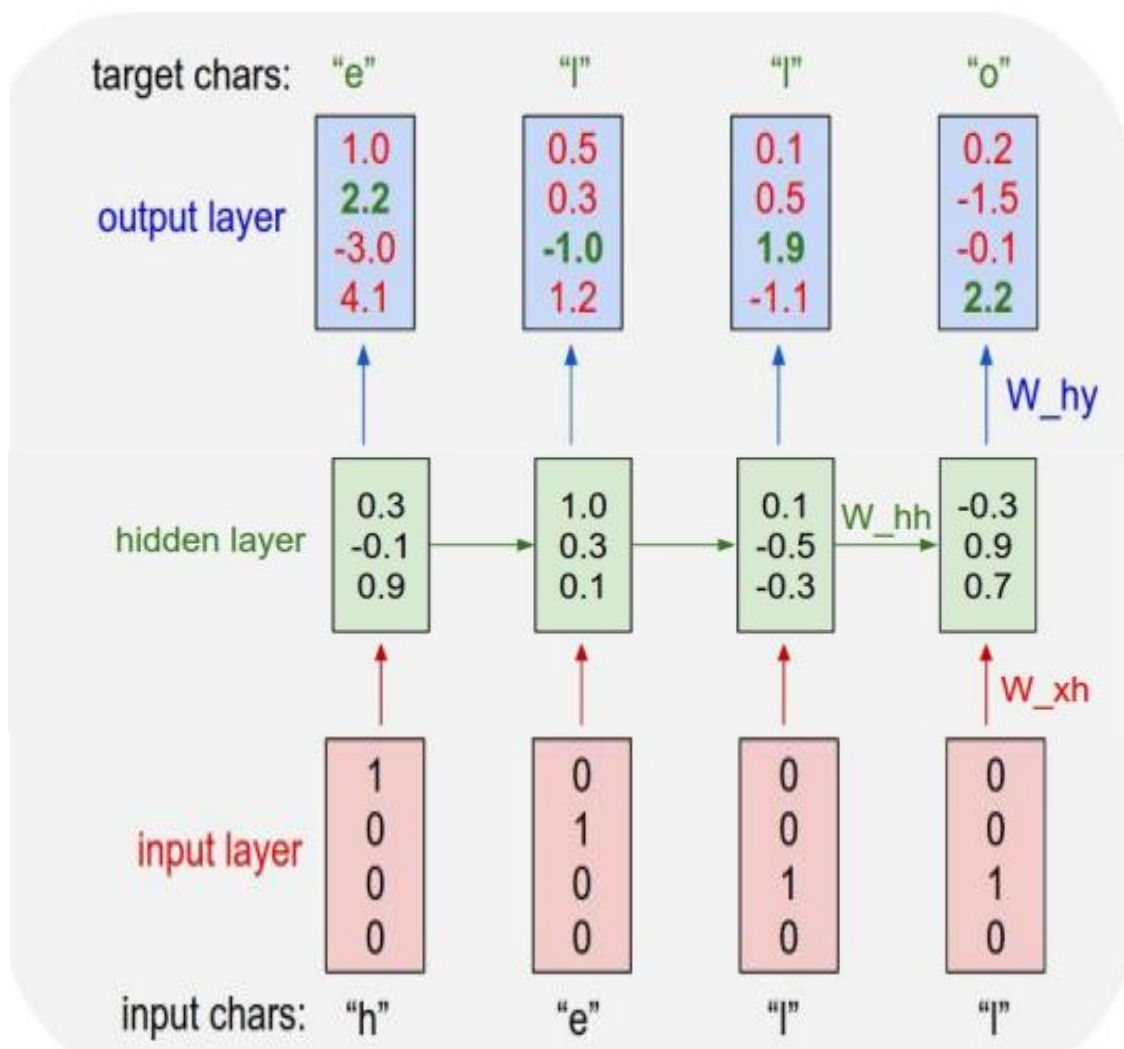
Example: Character-level Language Model

Vocabulary: [h,e,l,o]

Example training sequence: “hello”

$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t + B_h)$$

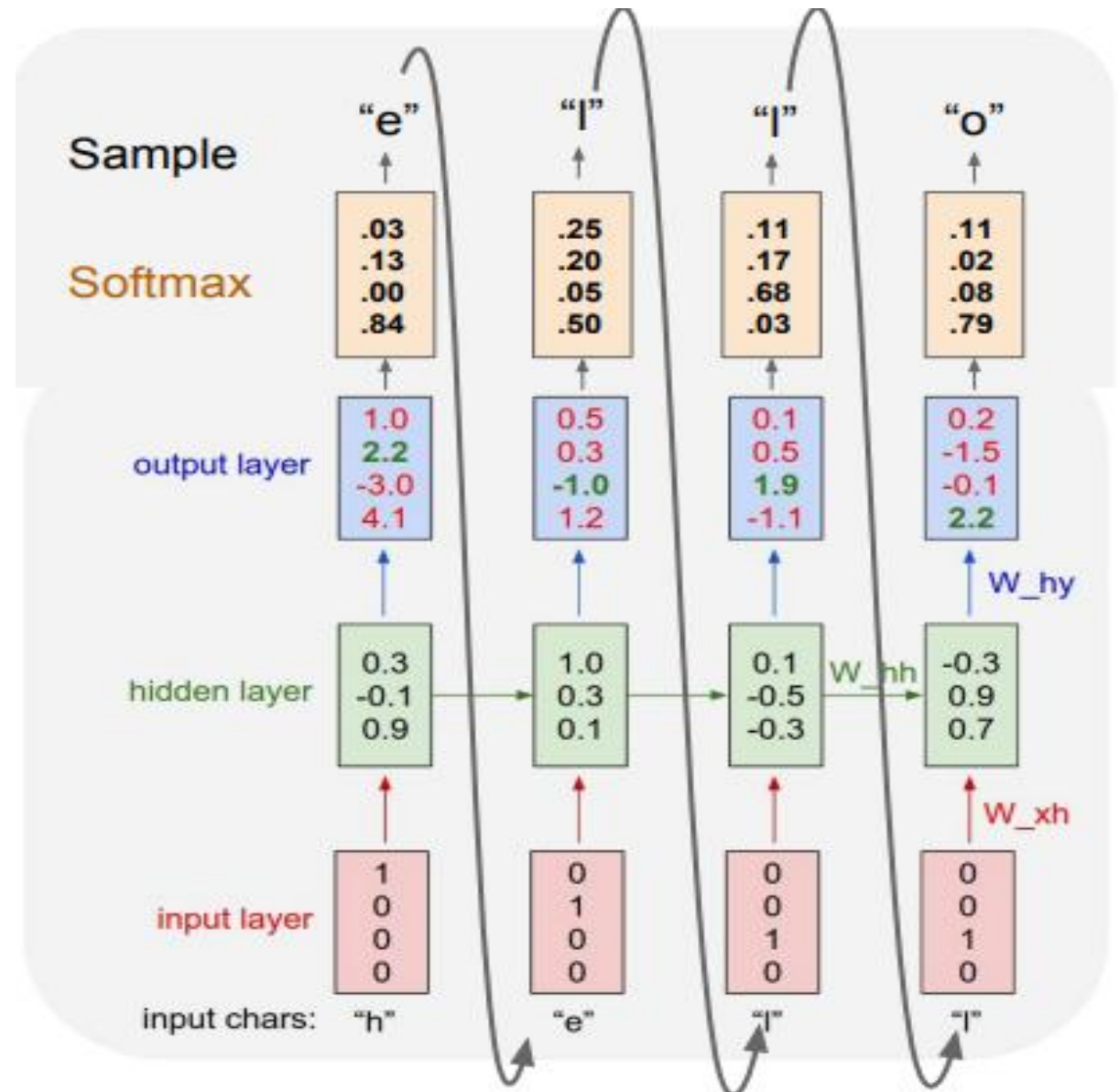
$$y_t = W_{hy}h_t + B_y$$



Continued...

Vocabulary: [h,e,l,o]

At test-time sample characters
one at a time,
feed back to model



Back Propagation

To reduce loss function derivative of y^i

$$\partial L / \partial y^i$$

By Chain rule W_{xh} is dependent on y^i , $\partial L / \partial y^i$

$$\partial L / \partial w_{xh} = (\partial L / \partial y^i * \partial y^i / \partial w_{xh})$$

Weight Updation,

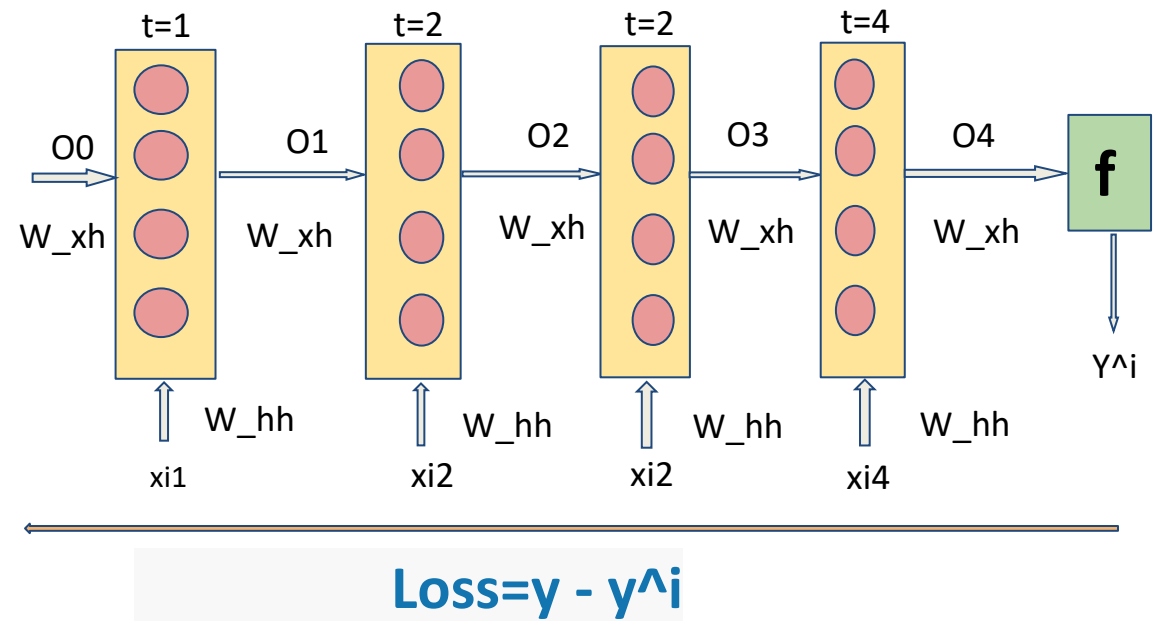
$$W_{hh_new} = W_{xh} - \partial L / \partial w_{xh}$$

Weight Updation W_{xh} w.r.t $O3$ in Backward Propagation at time $t3$

By Chain Rule $O4$ is dependent on W_{hh} , y^i dependent on $O4$, loss is dependent on y^i , $\partial L / \partial y^i$

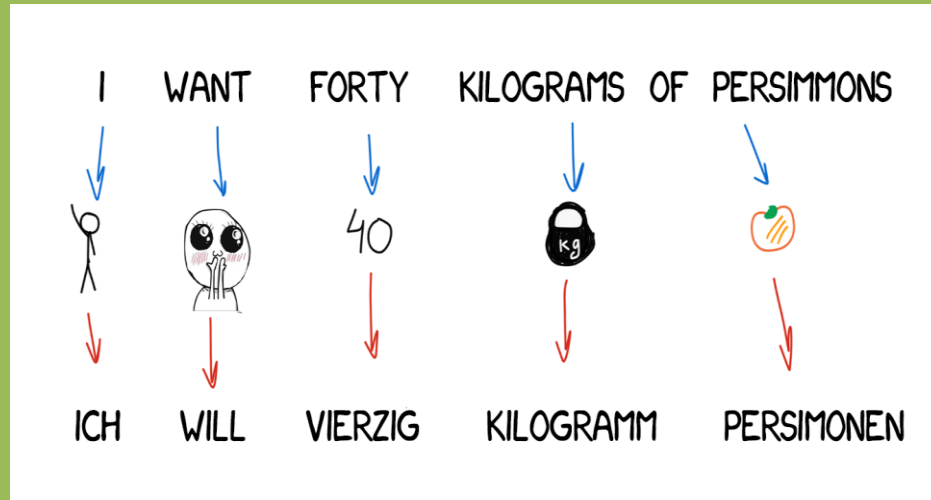
$$\partial L / \partial w_{xh} = (\partial L / \partial y^i * \partial y^i / \partial O4 * \partial O4 / \partial w_{hh})$$

$$W_{new_hh} = W_{xh} - (\partial L / \partial y^i * \partial y^i / \partial O4 * \partial O4 / \partial w_{hh})$$

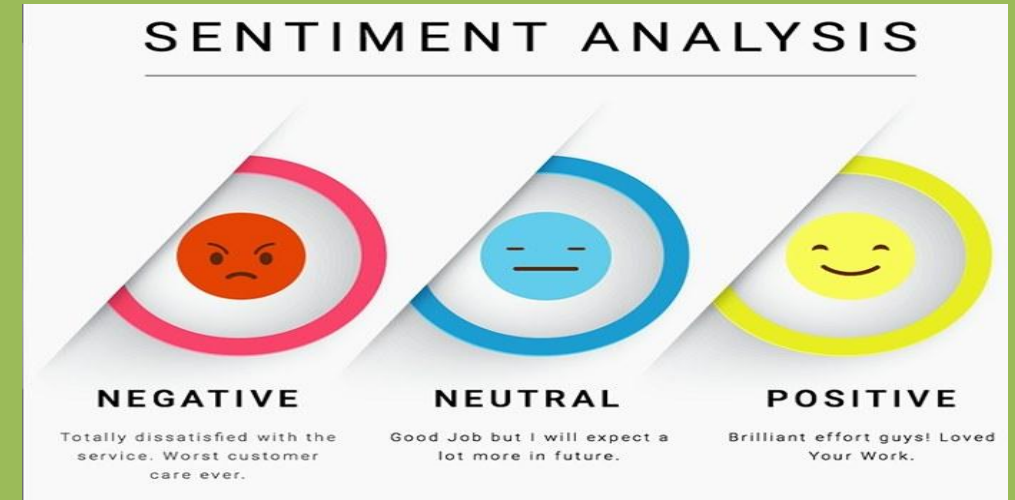


Application

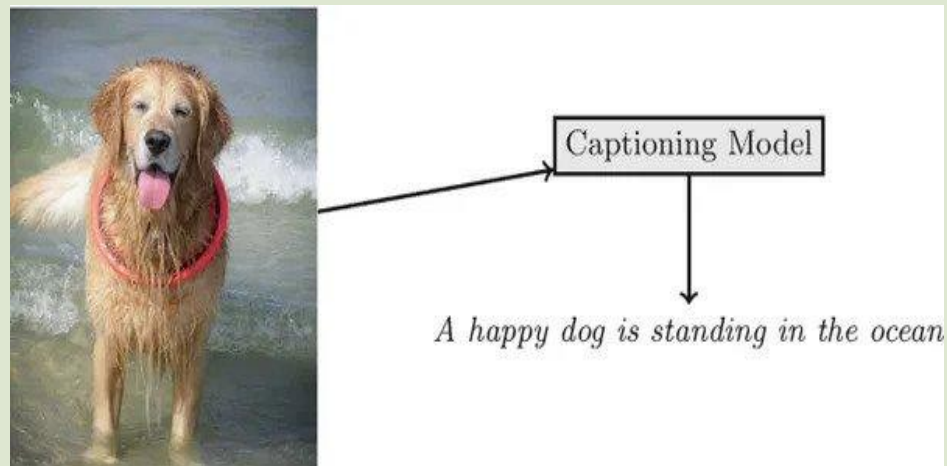
Machine Translation



Text Classification



Captioning Images



Recognition of Speech



Advantage

- ❑ Input of any length.
- ❑ To remember each information throughout the time which is very helpful in any time series predictor.
- ❑ Even if the input size is larger, the model size does not increase.
- ❑ Weights shared across the time steps.

Disadvantage

- ❑ Computation is slow.
- ❑ Training can be difficult.
- ❑ Using of relu or tanh as activation functions can be very difficult to process sequences that are very long.
- ❑ Prone to problems such as exploding and gradient vanishing.

Vanishing & Exploding Gradient



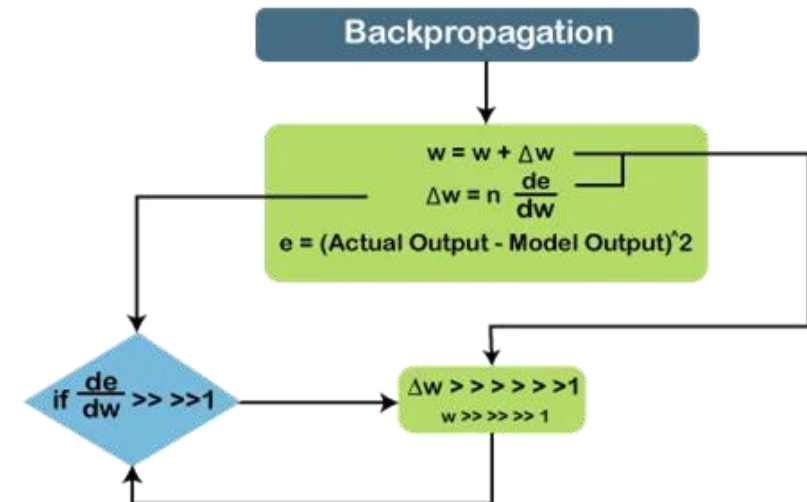
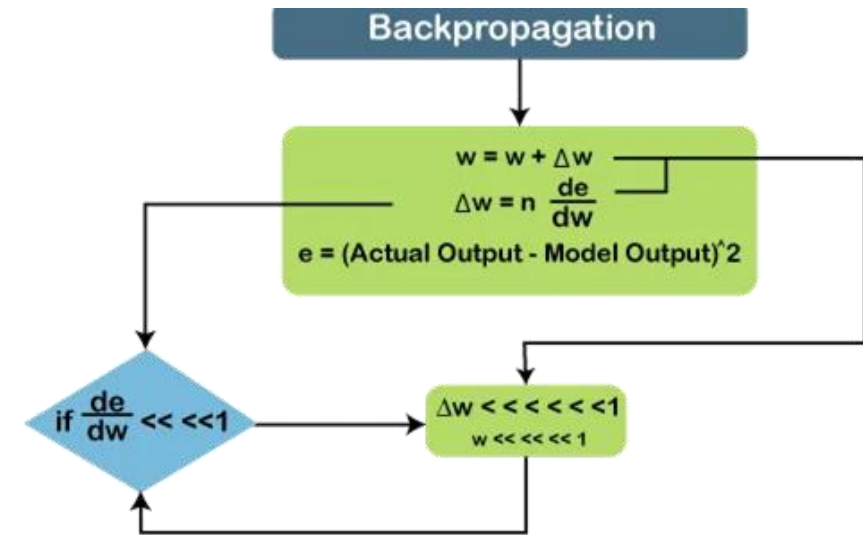
How to identify a vanishing or exploding gradients problem?

Vanishing

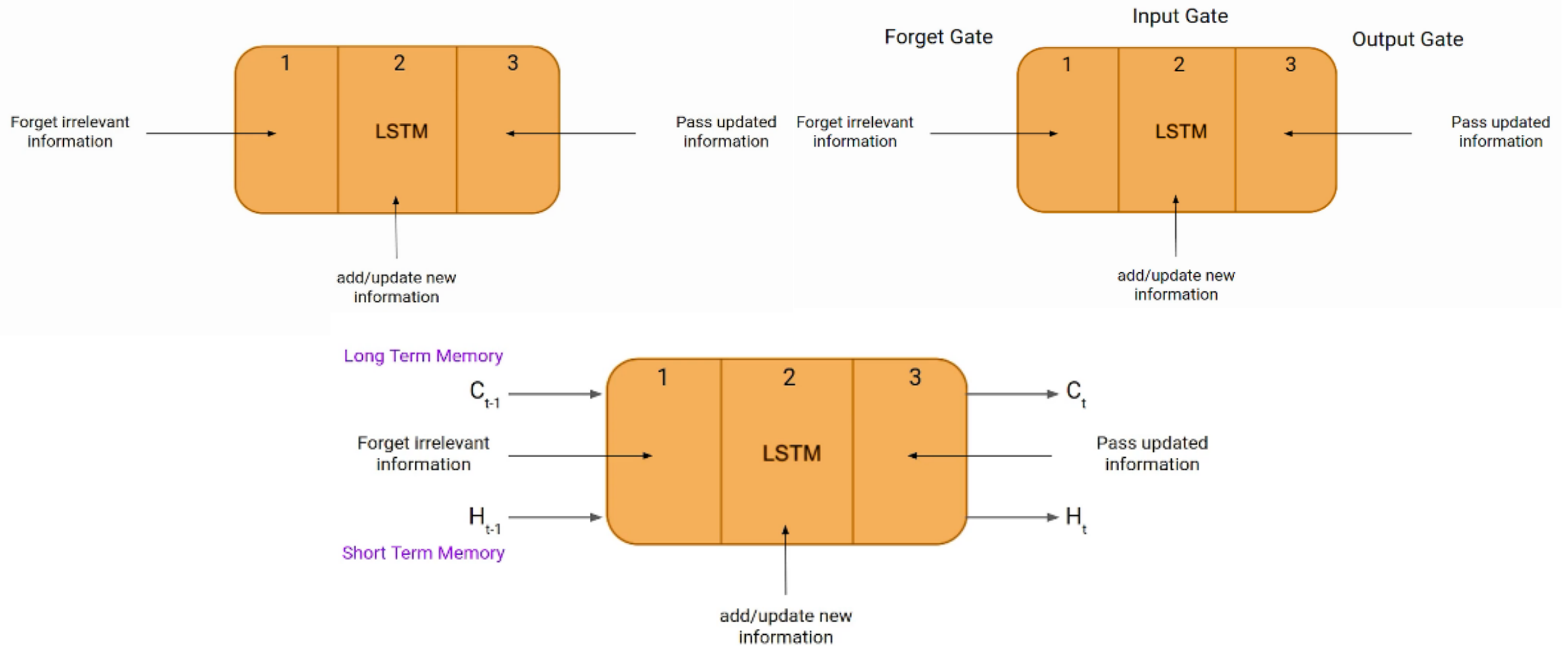
- ❑ **Weights** of earlier layers can become **0**.
- ❑ **Training stops** after a few iterations.

Exploding

- ❑ **Weights** become unexpectedly **large**.
- ❑ **Gradient** value for error persists over **1.0**.



LSTM



Working Process of LSTM

Forget Gate

- X_t : Input to the **current timestamp**
- U_f : **Weight associated** with the input
- H_{t-1} : The Hidden state of the **previous timestamp**
- W_f : It is the **weight matrix associated** with the hidden state



LSTM

Bob is a nice person. Dan on the other hand is evil.

Forget Gate:

- $f_t = \sigma(x_t * U_f + H_{t-1} * W_f)$

$$C_{t-1} * f_t = 0 \quad \dots \text{if } f_t = 0 \text{ (forget everything)}$$

$$C_{t-1} * f_t = C_{t-1} \quad \dots \text{if } f_t = 1 \text{ (forget nothing)}$$

Continued

“Bob knows swimming. He told me over the phone that he had served the **navy** for four long years.”

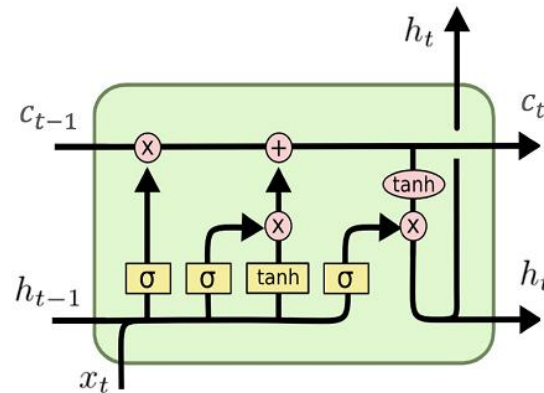
Bob single-handedly fought the enemy and died for his country. For his contributions, **brave**_____.”

Input Gate:

- $i_t = \sigma(x_t * U_i + H_{t-1} * W_i)$
 - $N_t = \tanh(x_t * U_c + H_{t-1} * W_c)$ (new information)
- $$C_t = f_t * C_{t-1} + i_t * N_t$$
- (updating cell state)

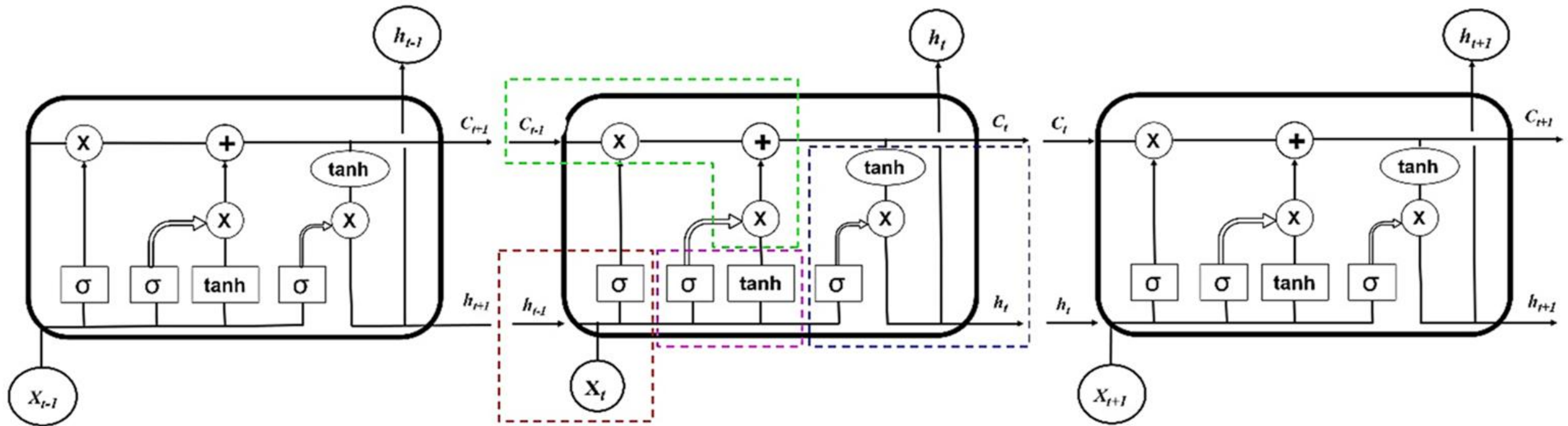
Output Gate:

- $o_t = \sigma(x_t * U_o + H_{t-1} * W_o)$
- $$H_t = o_t * \tanh(C_t)$$
- Output = Softmax(H_t)



LSTM
(Long-Short Term Memory)

Continued...



Gradient Clipping

Clipping – by – value

→ A minimum clip value and a maximum clip value.

- $\mathbf{g} \leftarrow \partial C / \partial \mathbf{W}$
- $\|\mathbf{g}\| \geq \text{max_threshold}$ or $\|\mathbf{g}\| \leq \text{min_threshold}$
- $\mathbf{g} \leftarrow \text{threshold}$ (accordingly)

Clipping – by – norm

Clip the gradients by multiplying the unit vector of the gradients with the threshold.

- $\mathbf{g} \leftarrow \partial C / \partial \mathbf{W}$
- if $\|\mathbf{g}\| \geq \text{threshold}$ then
- $\mathbf{g} \leftarrow \text{threshold} * \mathbf{g} / \|\mathbf{g}\|$

Sentence Modeling: Sequence Prediction



Masterful!

By Antony Witheyman - January 12, 2006

Ideal for anyone with an interest in
disguises who likes to see the subject
tackled in a humorous manner.

0 of 4 people found this review helpful

Prediction

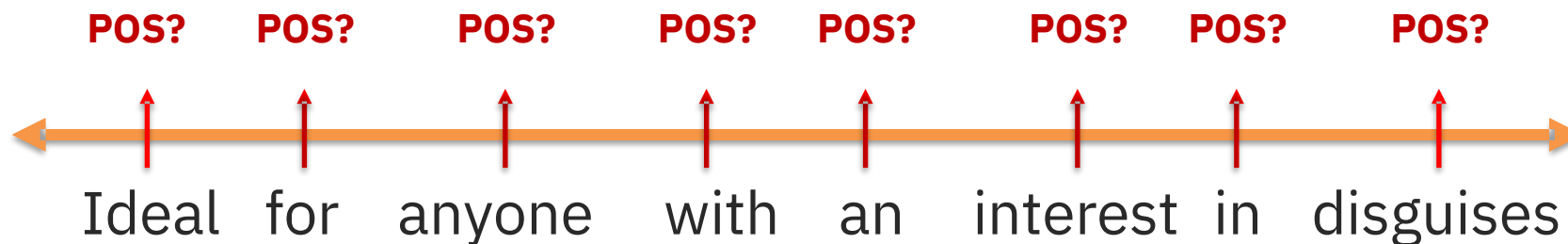


Part-of-speech ?

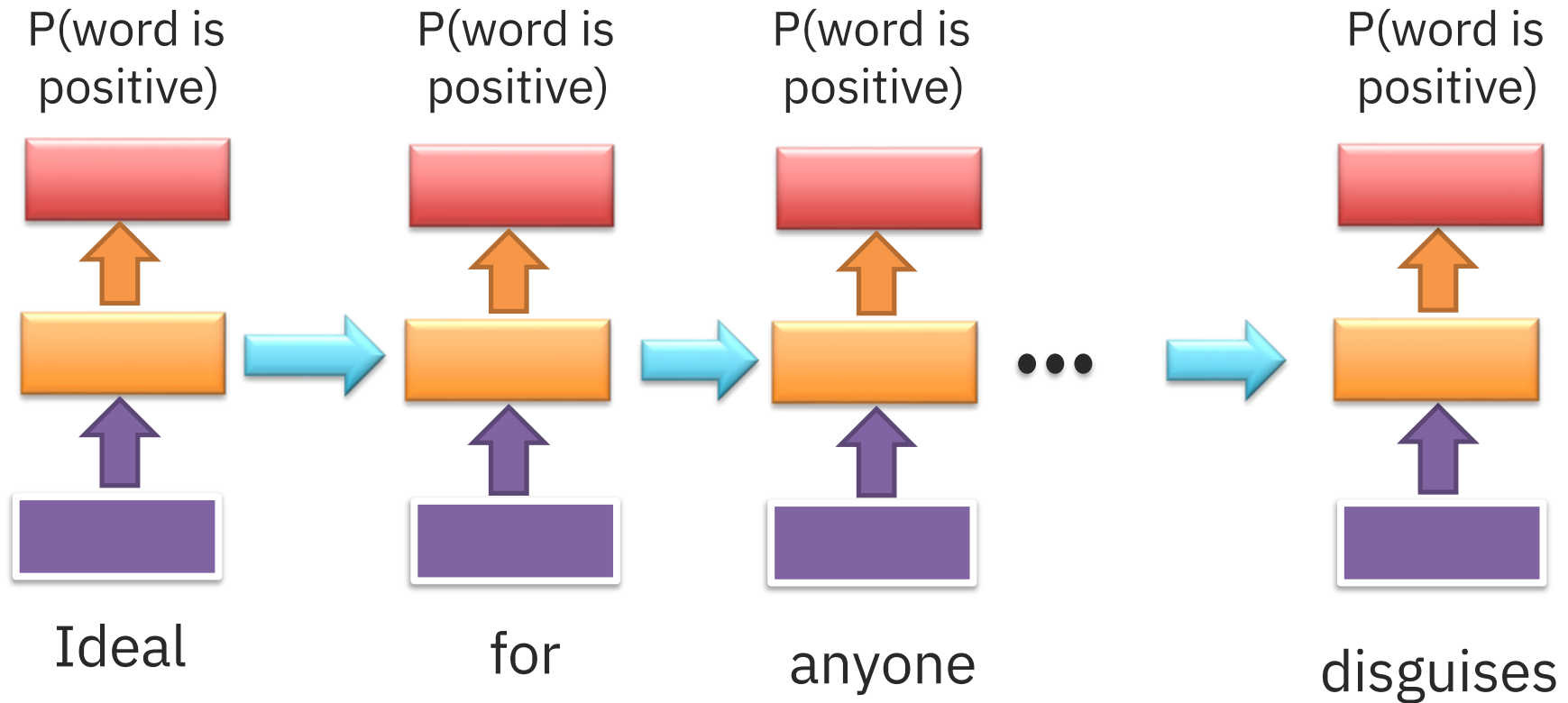
(noun, verb,...)

Sentiment ?

(positive or negative)



RNN for Sequence Prediction



What is the loss?
$$L = \frac{1}{N} \sum_t L^{(t)} = \frac{1}{N} \sum_t -\log P(Y = y^{(t)} | z^{(t)})$$

Sentence Modeling: Sequence Label Prediction



Masterful!

By Antony Witheyman - January 12, 2006

Ideal for anyone with an interest in
disguises who likes to see the subject
tackled in a humorous manner.

0 of 4 people found this review helpful

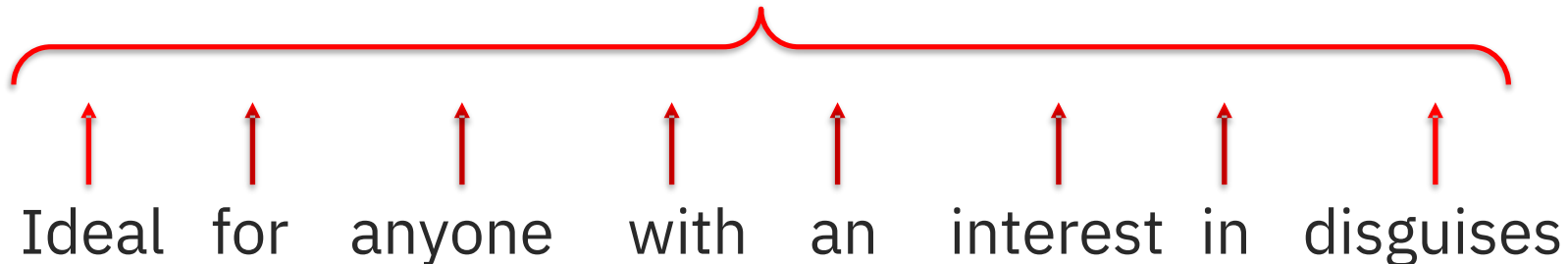
Prediction



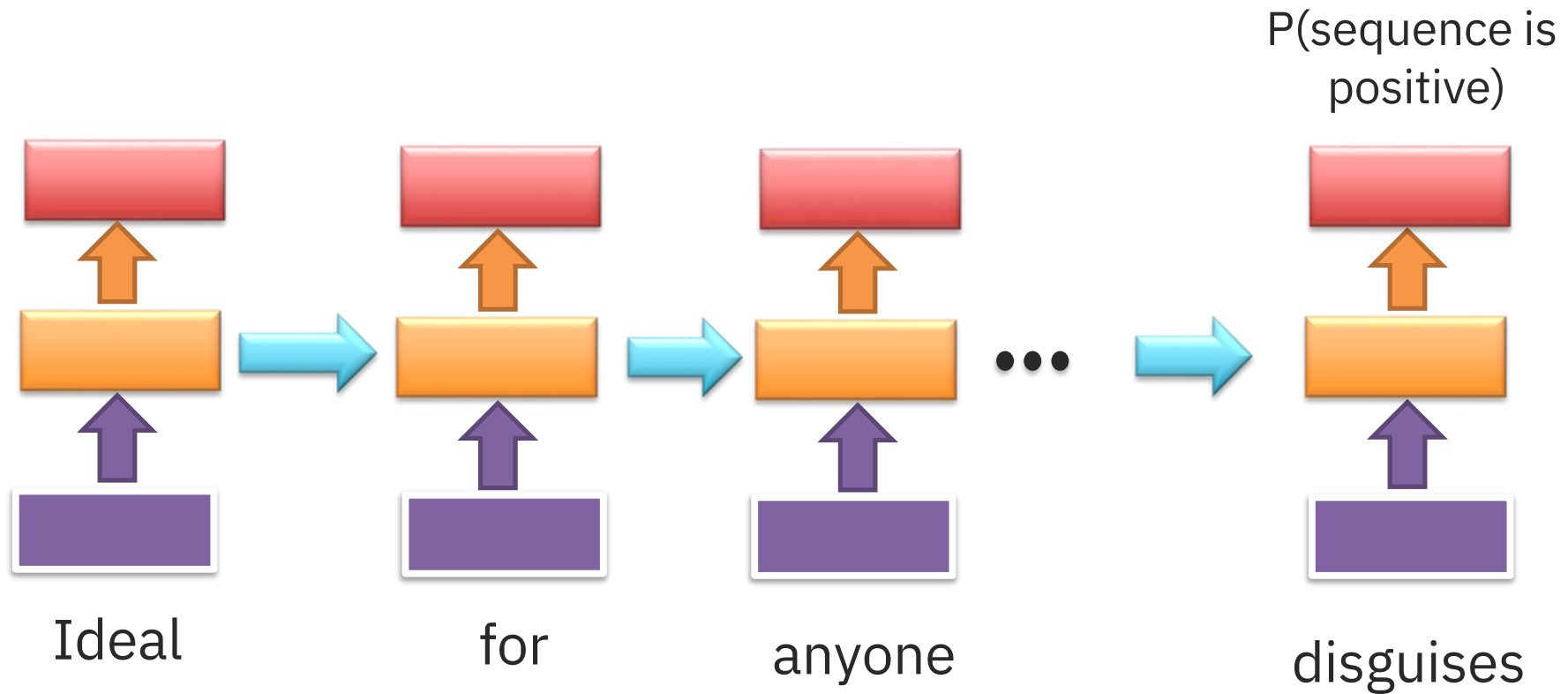
Sentiment ?

(positive or negative)

Sentiment label?



RNN for Sequence Prediction



What is the loss? $L = -\log P(Y = y^{(N)} | z^{(N)})$

Language Models

Sentence Modeling: Language Model



Masterful!

By Antony Witheyman - January 12, 2006

Ideal for anyone with an interest in
disguises who likes to see the subject
tackled in a humourous manner.

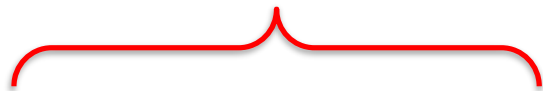
0 of 4 people found this review helpful

Prediction



Next
word

Next word?



Ideal for anyone

with an

interest in disguises

Language Model Application: Speech Recognition

$$\arg \max_{wordsequence} P(wordsequence | acoustics) =$$

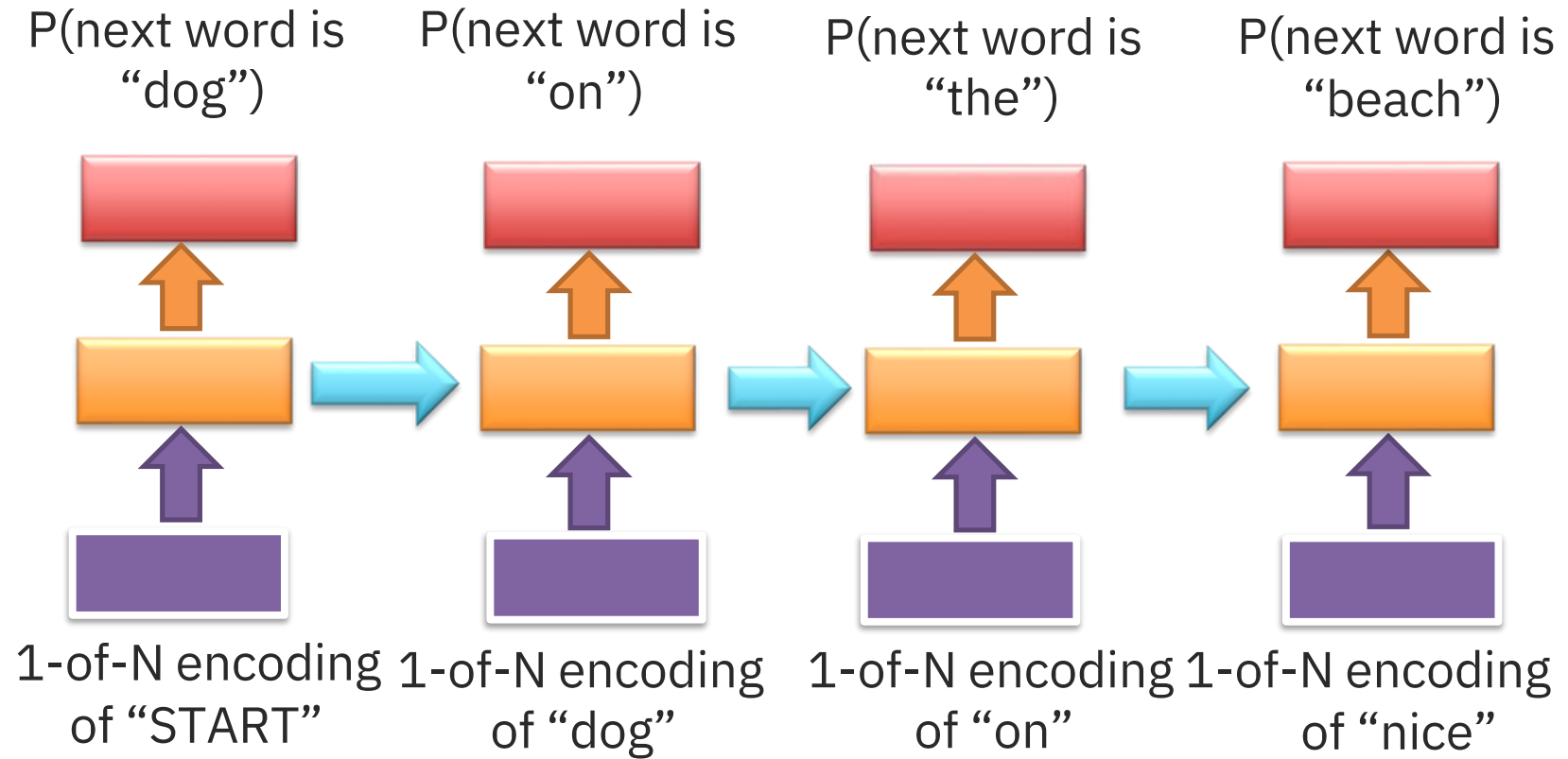
$$\arg \max_{wordsequence} \frac{P(acoustics | wordsequence) \cdot P(wordsequence)}{P(acoustics)}$$

$$\arg \max_{wordsequence} P(acoustics | wordsequence) \cdot P(wordsequence)$$

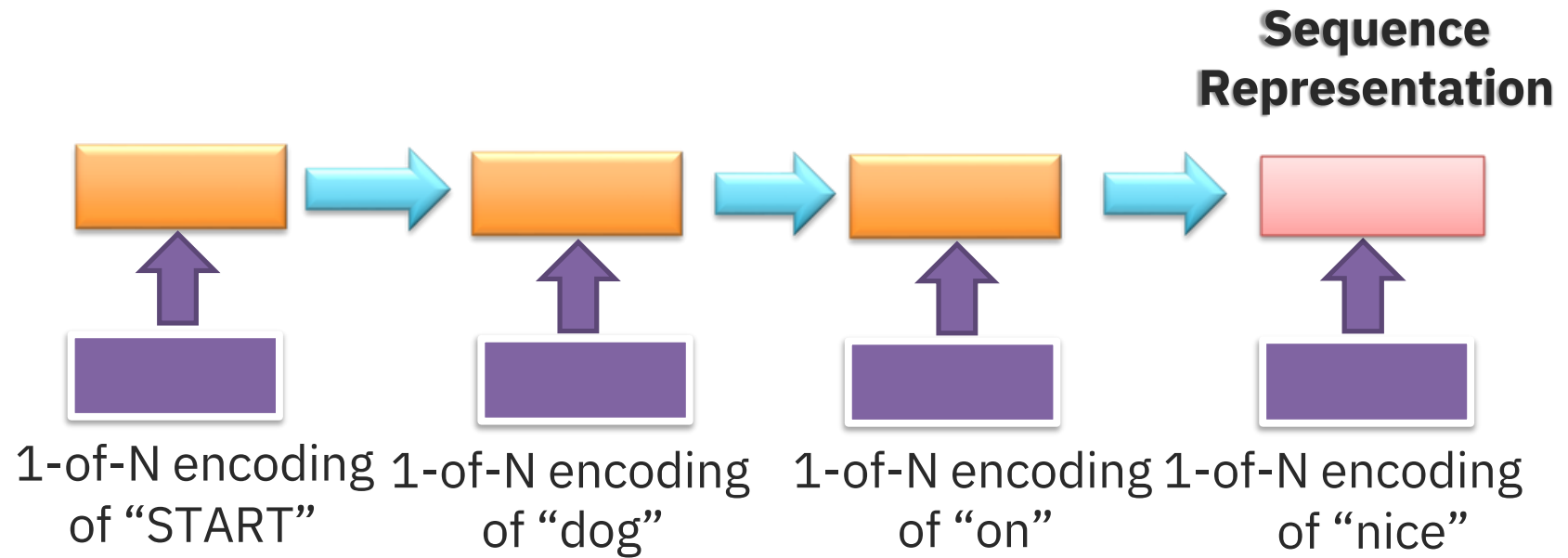


Language model

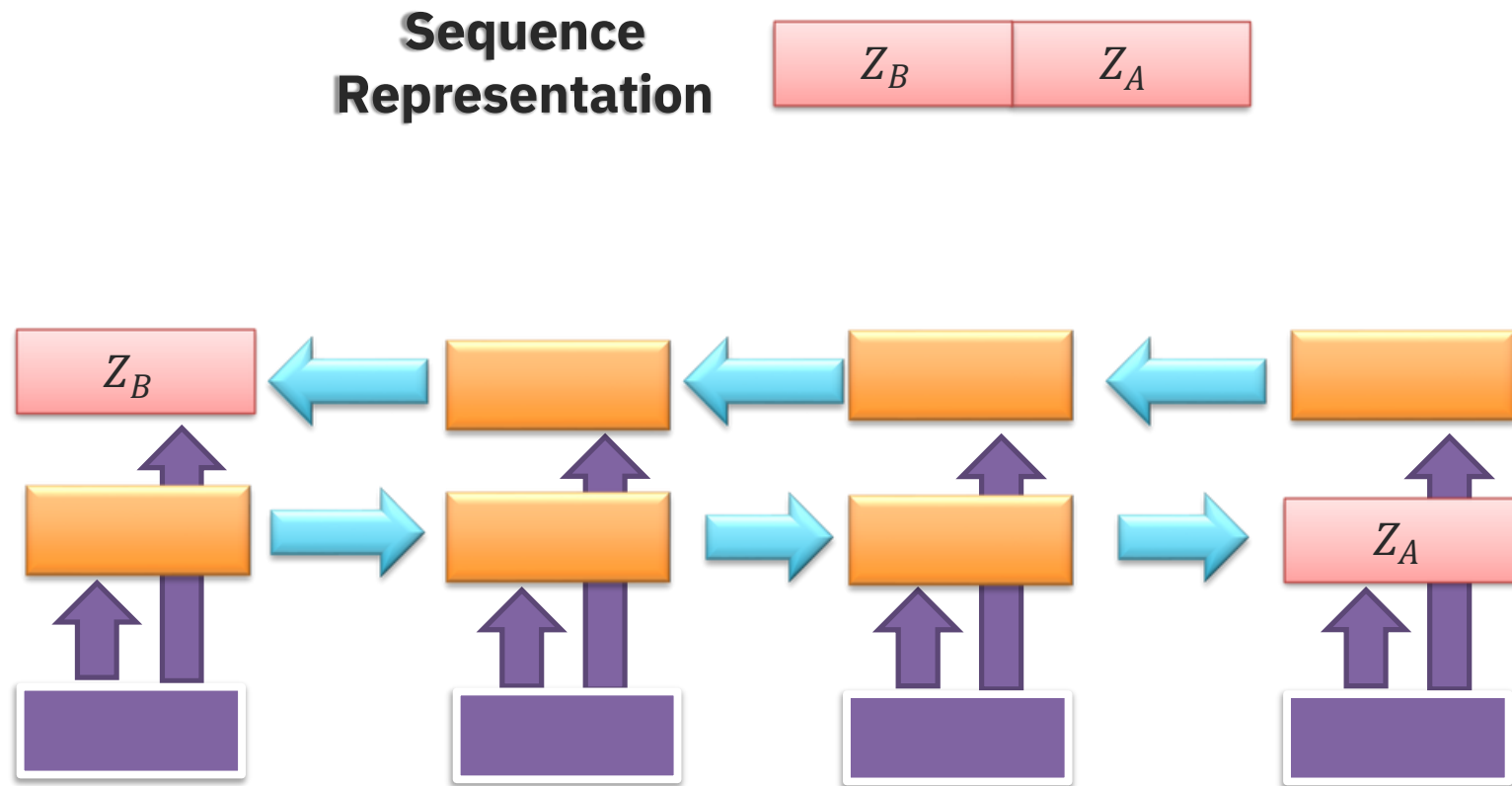
RNN for Language Model



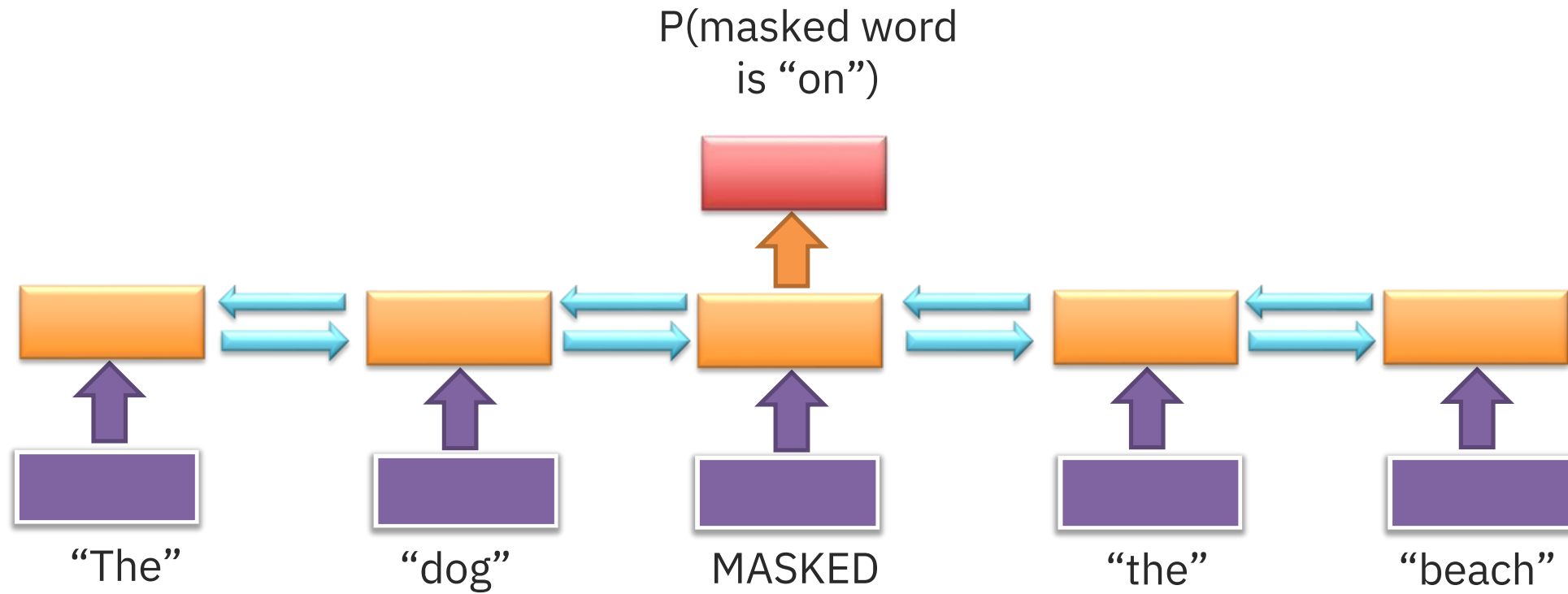
RNN for Sequence Representation (Encoder)



Bi-Directional RNN



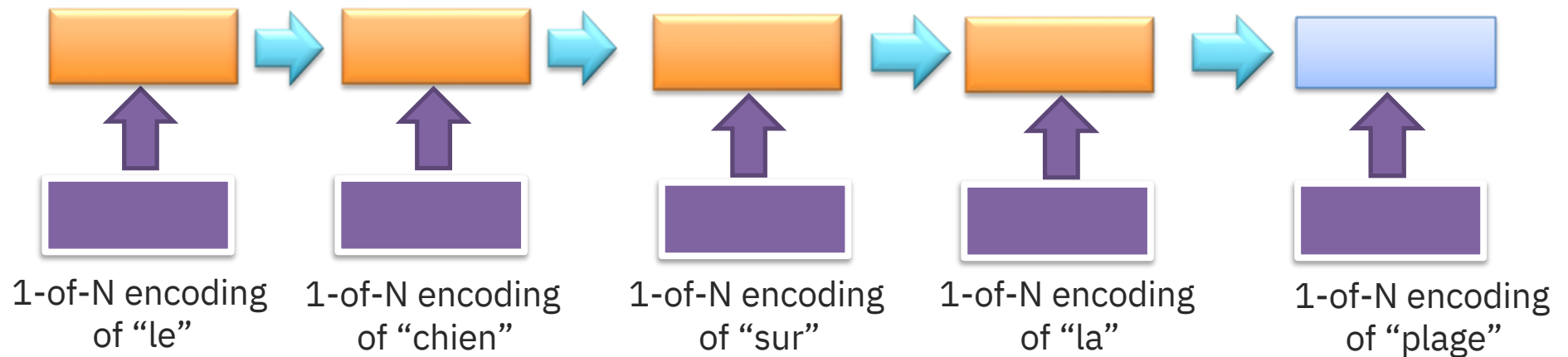
Pre-training and “Masking”



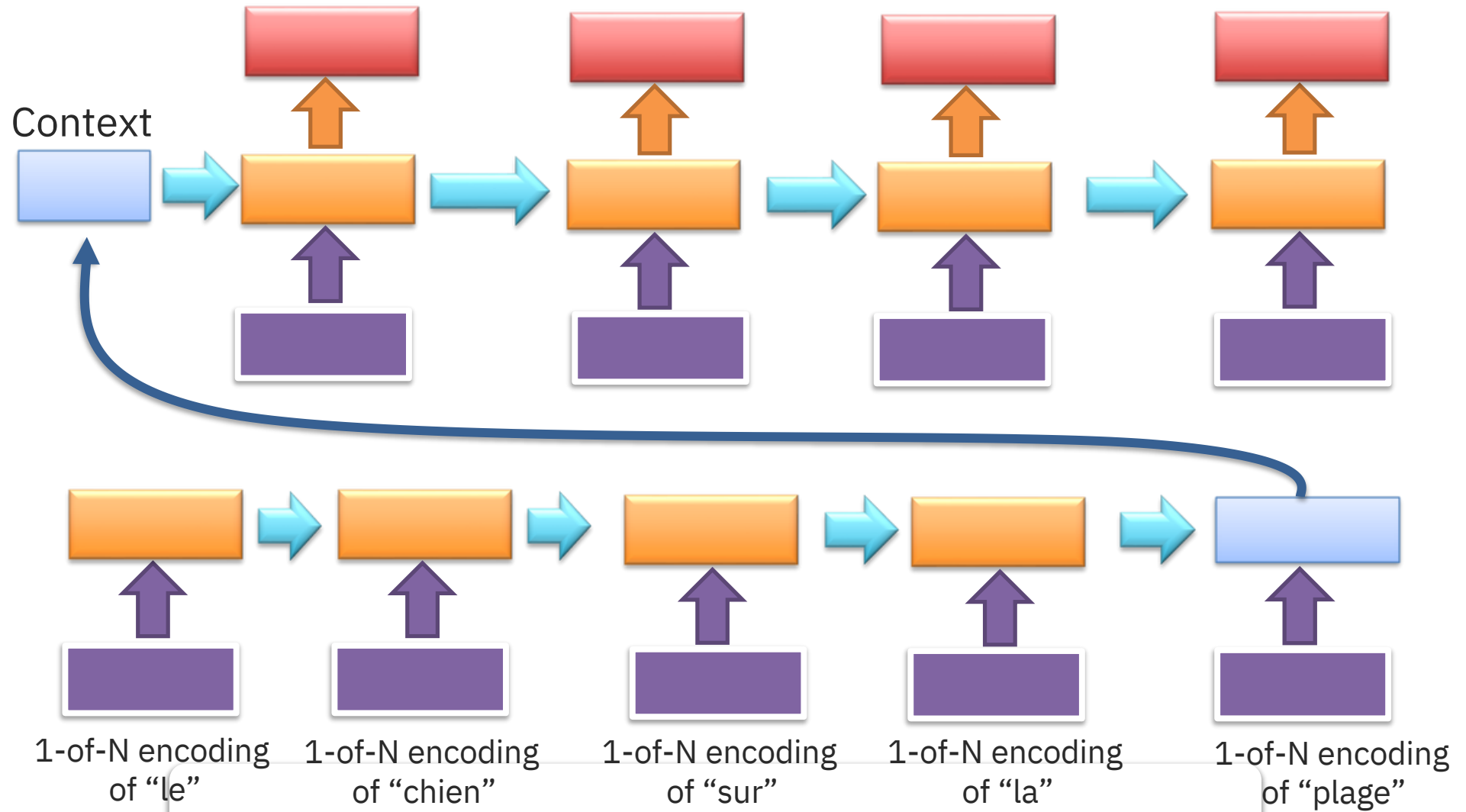
➡ (short-lived) ELMo was a bi-directional pretrained language model

RNN-based for Machine Translation

Le chiensur la plage → The dog on the beach



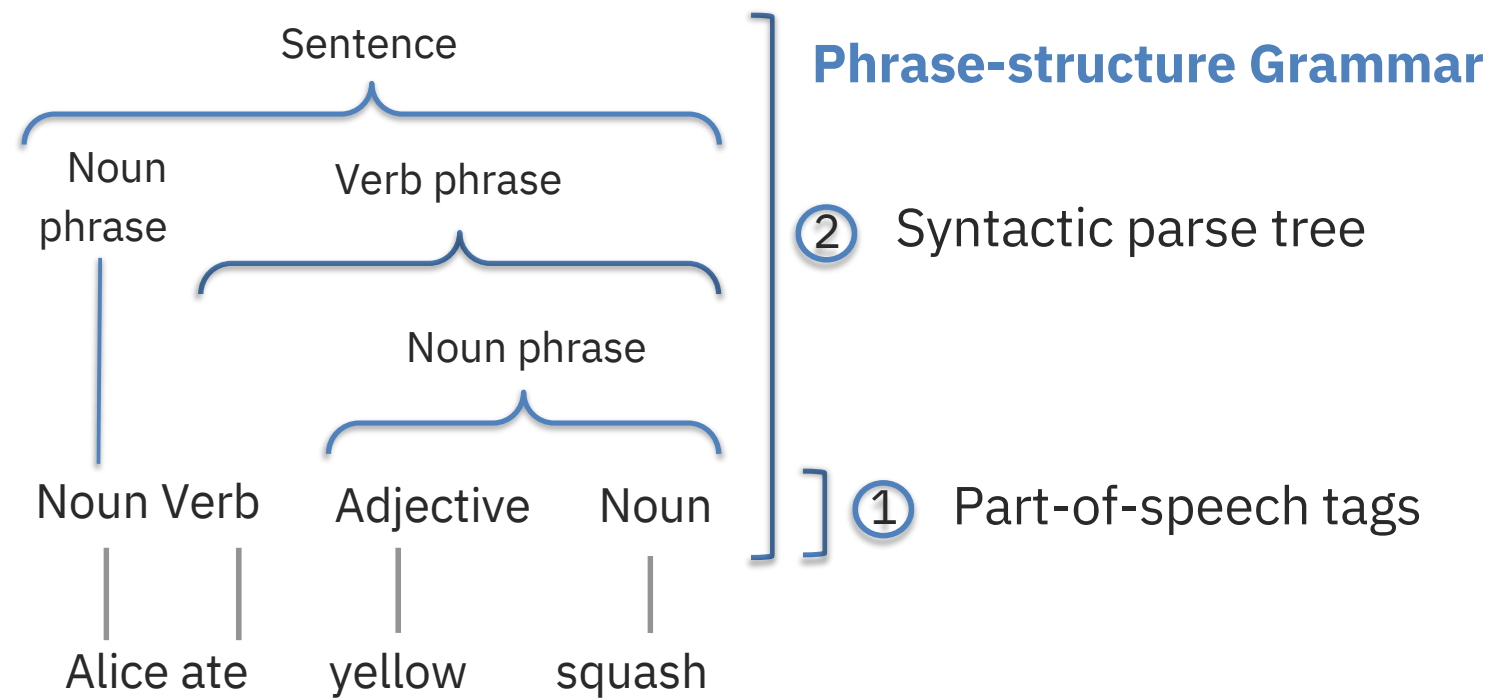
Encoder-Decoder Architecture



Syntax and Language Structure

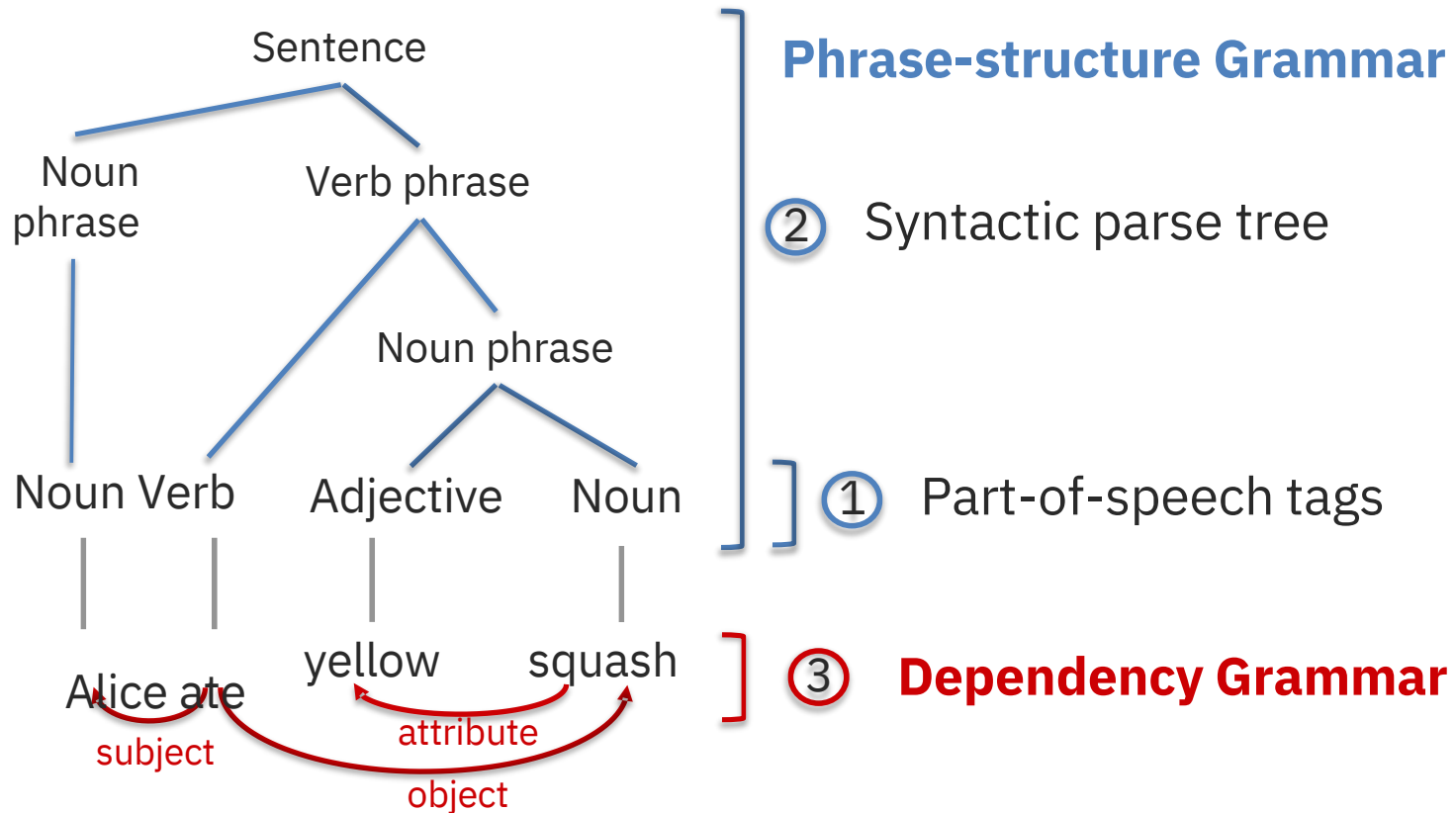
Syntax and Language Structure

What can you tell about this sentence?



Syntax and Language Structure

What can you tell about this sentence?



Ambiguity in Syntactic Parsing

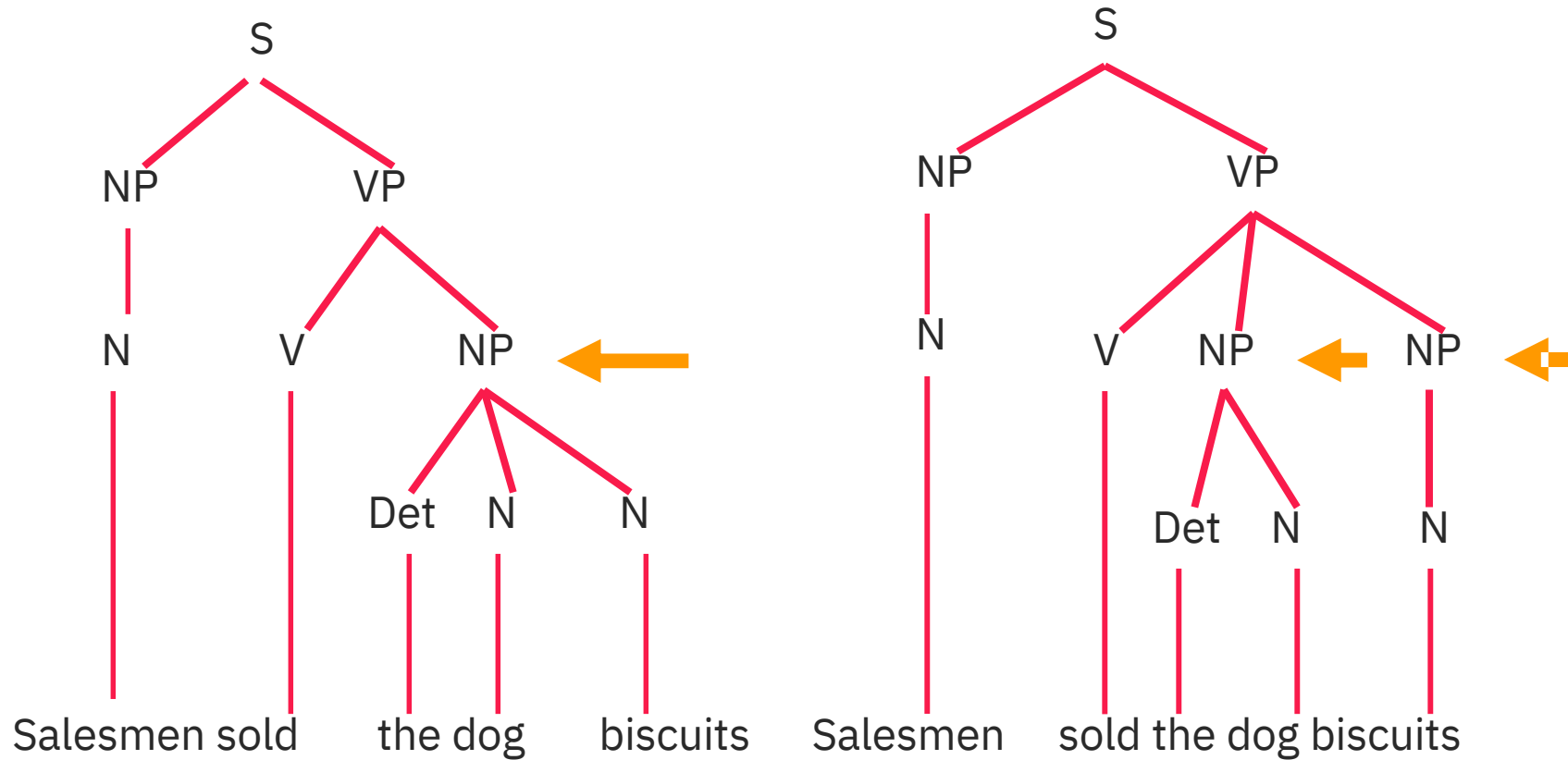
“Like” can be a verb or a preposition

- I like/**VBP** candy.
- Time flies like/**IN** an arrow.

“Around” can be a preposition, particle, or adverb

- I bought it at the shop around/**IN** the corner.
- I never got around/**RP** to getting a car.
- A new Prius costs around/**RB** \$25K.

Language Ambiguity



Language Syntax – Examples

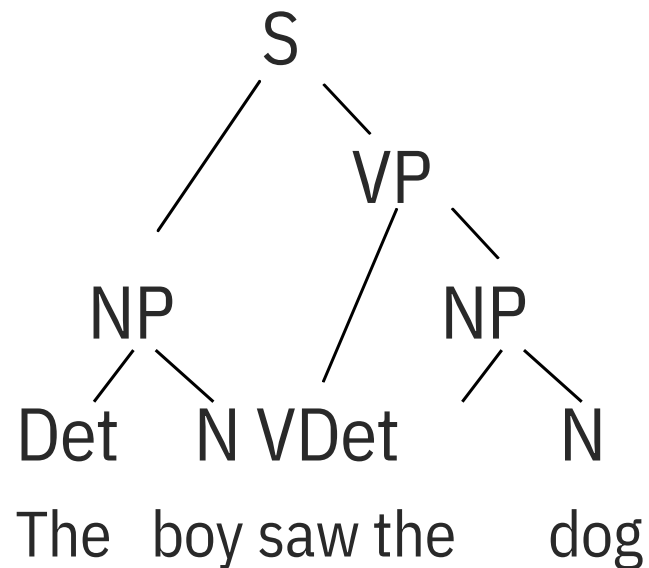
Det Noun Verb Det

The boy saw the

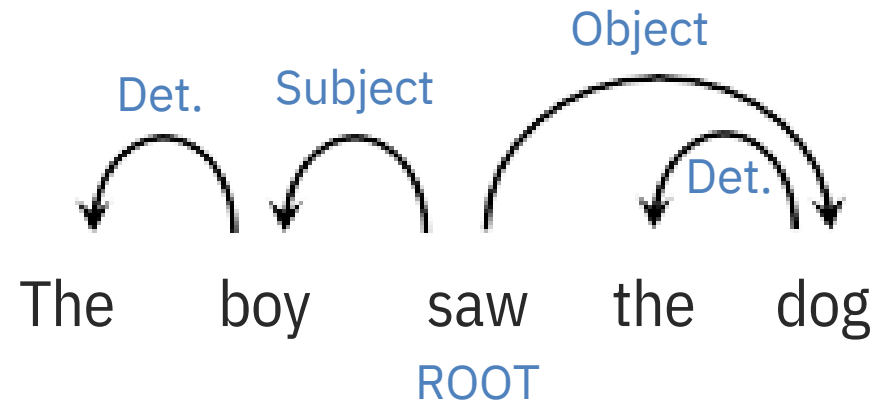
Noun Prep Det Noun

dog in the park

Part of Speech tagging



Constituency Parsing

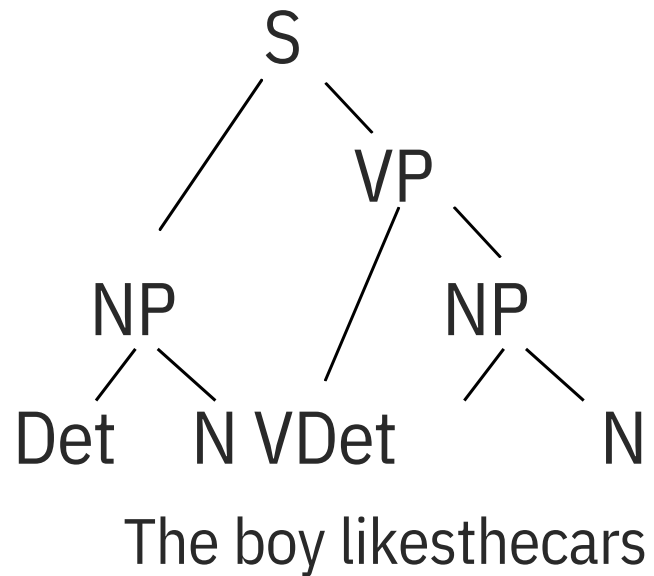


Dependency Parsing

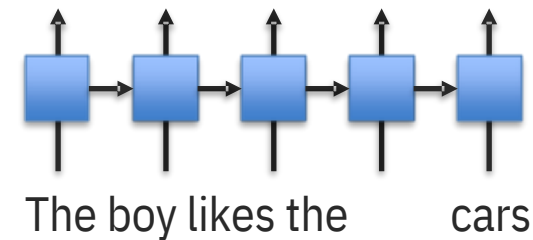
How to take advantage of syntax when modeling language with neural networks?

Syntax with Recursive Neural Network

How to Model Syntax with RNNs?

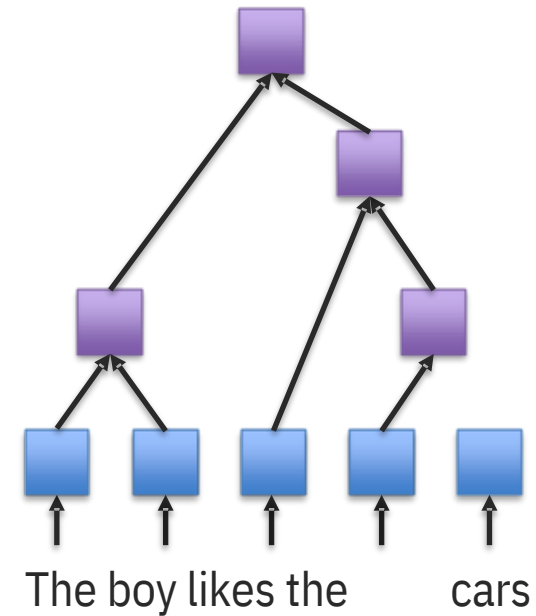
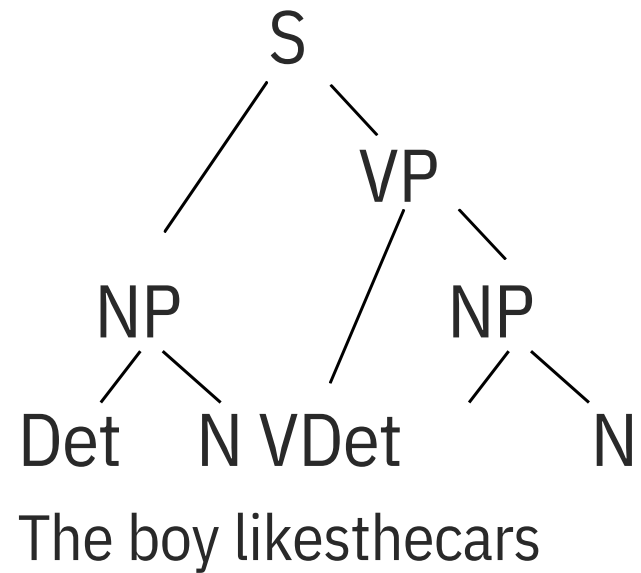


?



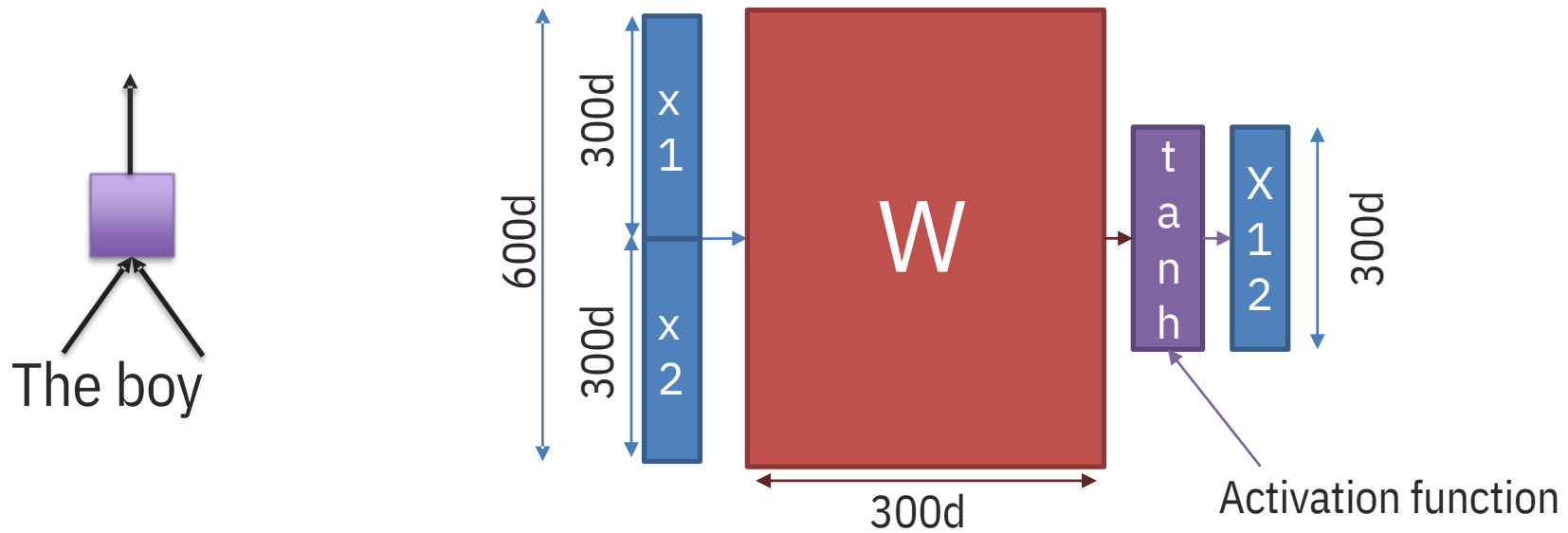
We could use Part-of-Speech tags.

Tree-based RNNs (or Recursive Neural Network)

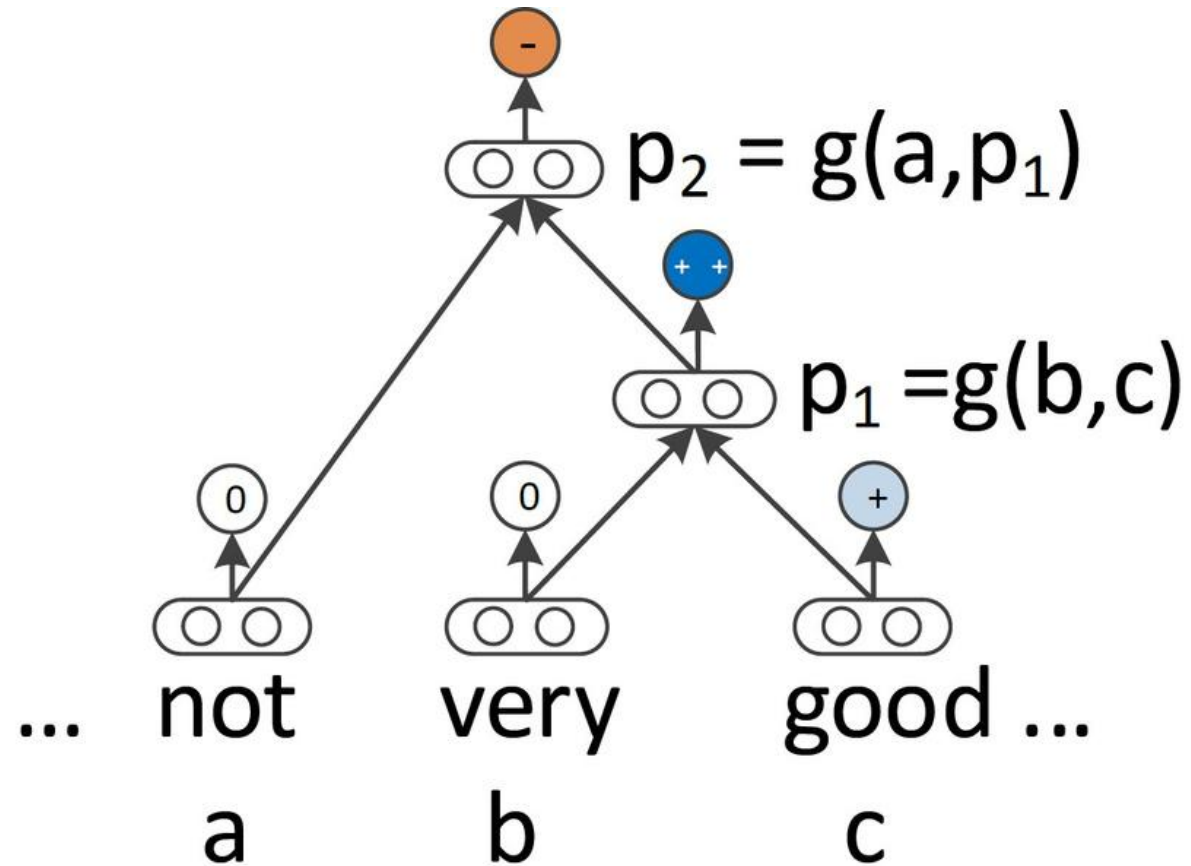


Recursive Neural Unit

➡ Pair-wise combination of two input features



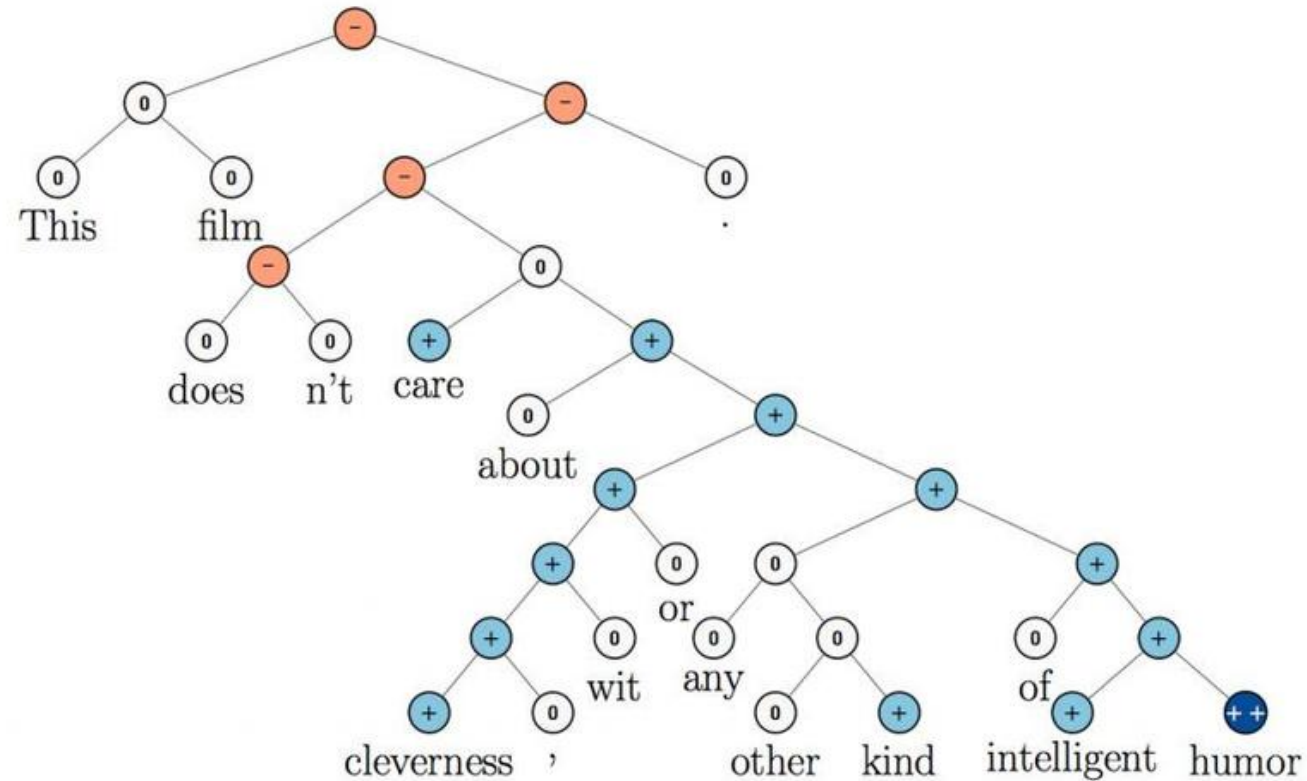
Recursive Neural Network for Sentiment Analysis



Socher et al., Recursive Deep Models for Semantic Compositionality Over a Sentiment Treebank, EMNLP 2013

Recursive Neural Network for Sentiment Analysis

Classification of a sentence using tree-based compositionality of words



Demo: <http://nlp.stanford.edu/sentiment/>

Socher et al., Recursive Deep Models for Semantic Compositionality Over a Sentiment Treebank, EMNLP 2013

RECURSIVE DEEP MODELS FOR SEMANTIC COMPOSITIONALITY OVER A SENTIMENT TREEBANK

**Richard Socher, Alex Perelygin, Jean Y. Wu, Jason Chuang,
Christopher D. Manning, Andrew Y. Ng and Christopher Potts**

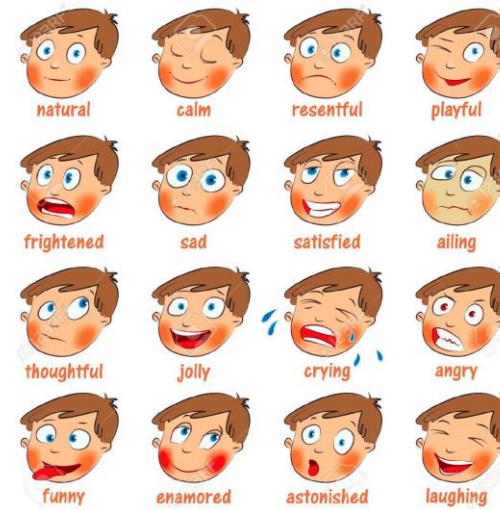
Overview

- Introduction
- Problem
- Stanford Sentiment Treebank
- Models
- Experiments

Introduction | Sentiment

Sentiment ?

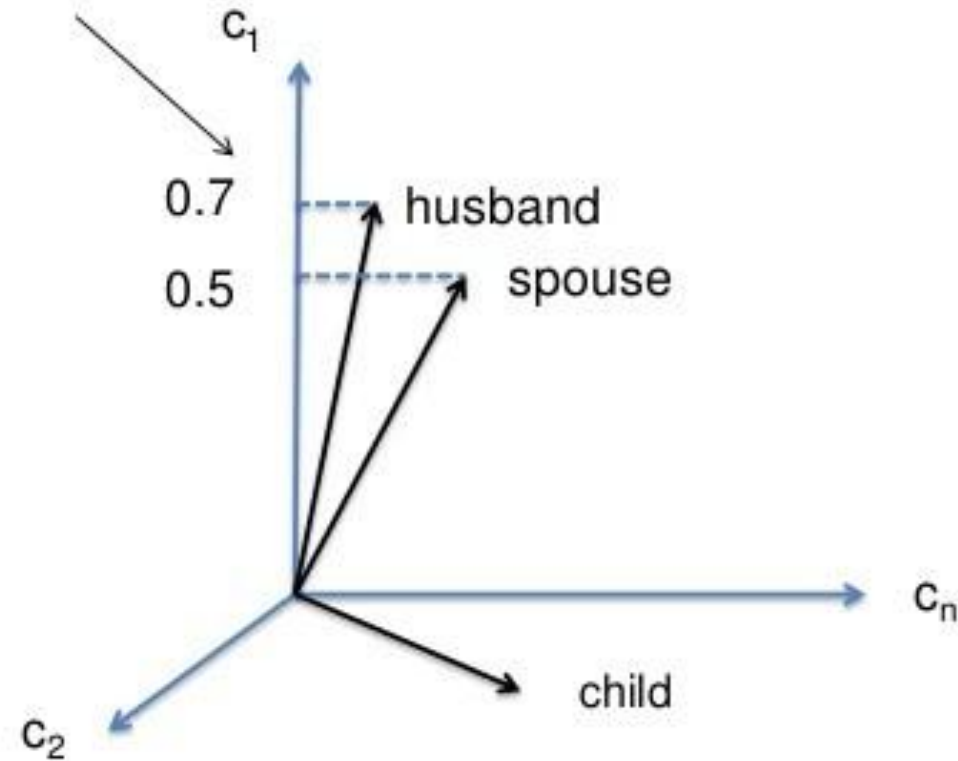
- Attitude
- Emotions
- Opinions



Ex. For/against, good/bad, positive/negative

Introduction | Vector Space Model

function (number of times that the words occur in c_1)



Problem

- Lack of large labeled compositionality corpus and models that can accurately capture the underlying phenomena in such data
- Semantic vector spaces are very useful but cannot express the meaning of longer phrases by themselves

Stanford Sentiment Treebank

- First corpus with fully labeled parse trees
- 10,662 single sentences extracted from movie reviews
- 215,154 unique phrases generated by the Stanford parser
- Each phrase annotated by 3 human judges

Stanford Sentiment Treebank

1. 10,662 sentences were obtained and further parsed into 215,154 phrases using the Stanford Parser
2. Each phrase is annotated by 3 human annotators. Presented with a slider of 25 different values initially set to neutral
3. Phrases were randomly sampled from the set of all phrases



Figure 3: The labeling interface. Random phrases were shown and annotators had a slider for selecting the sentiment and its degree.

- Majority of shorter phrases are neutral. Sentiment often builds up in longer phrases
- Most annotators used 1/5 positions [negative, somewhat negative, neutral, positive or somewhat positive]
- As a result the main experiment is to recover these five labels

Stanford Sentiment Treebank

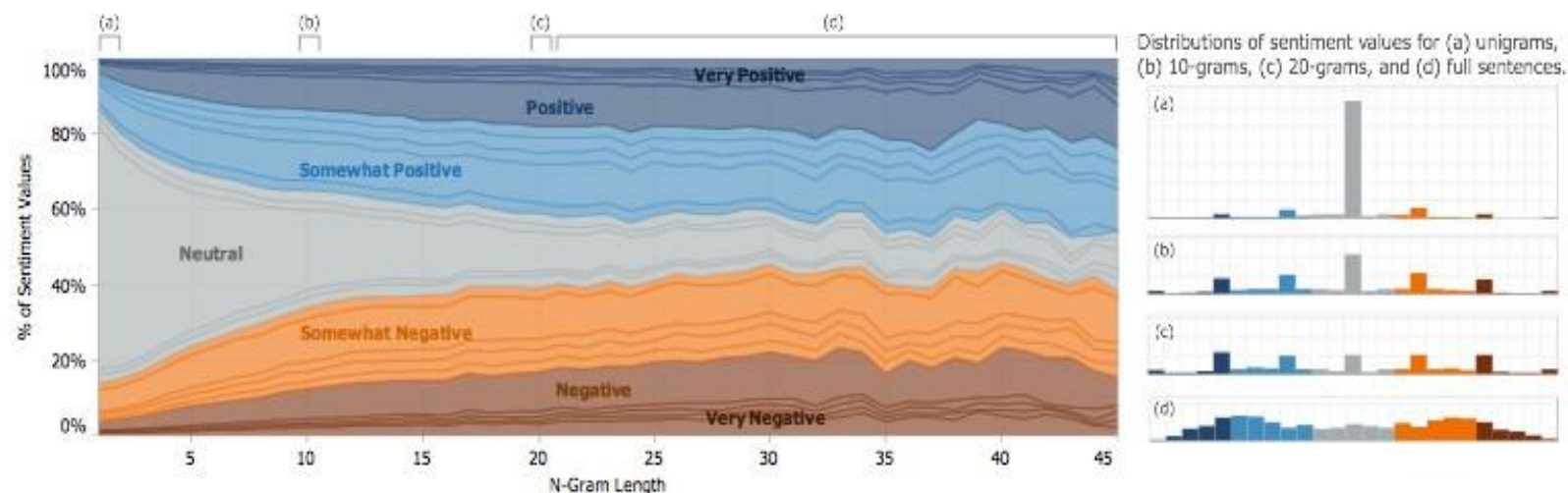


Figure 2: Normalized histogram of sentiment annotations at each n -gram length. Many shorter n -grams are neutral; longer phrases are well distributed. Few annotators used slider positions between ticks or the extreme values. Hence the two strongest labels and intermediate tick positions are merged into 5 classes.

Models - General

All models share the following:

- Compute compositional vector representations for phrases of variable length.
- Use the compositional vector representations derived from above as features to classify each phrase.

1. N-grams passed to compositional models, it is then parsed into a binary tree where each leaf node is represented as a vector.
2. Recursive models then compute parent vectors in a bottom up fashion using different type of

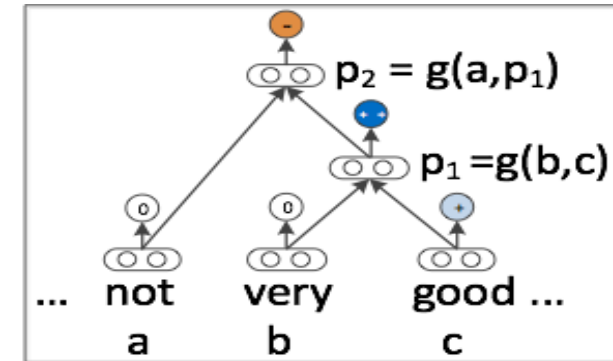


Figure 4: Approach of Recursive Neural Network models for sentiment: Compute parent vectors in a bottom up fashion using a compositionality function g and use node vectors as features for a classifier at that node. This function varies for the different models.

$$y^a = \text{softmax}(W_s a),$$

$$W_s \in \mathbb{R}^{5 \times d}$$

Model

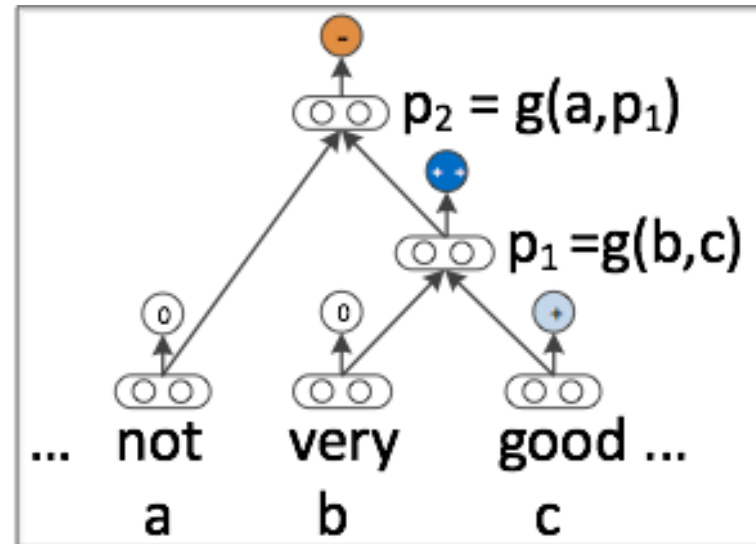


Figure 4: Approach of Recursive Neural Network models for sentiment: Compute parent vectors in a bottom up fashion using a compositionality function g and use node vectors as features for a classifier at that node. This function varies for the different models.

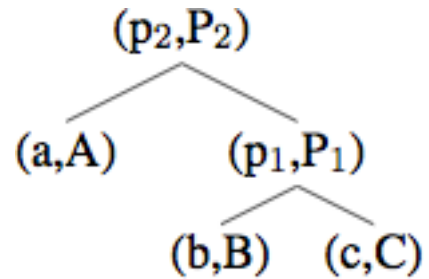
Model – Recursive Neural Network

- Where f is the tanh

$$p_1 = f\left(W \begin{bmatrix} b \\ c \end{bmatrix}\right), p_2 = f\left(W \begin{bmatrix} a \\ p_1 \end{bmatrix}\right),$$

1. It is first determined which parent already has all its children vectors computed.
2. Parent vectors are then computed in a bottom up fashion.
3. Once the parent vectors have been computed they are given to the same softmax classifier to compute its label probability.

Models – MV-RNN



- The main idea of this model is to represent each word as both a vector and a matrix

Disadvantage:

The number of parameters become very large and is dependent on the vocabulary

$$p_1 = f \left(W \begin{bmatrix} Cb \\ Bc \end{bmatrix} \right), P_1 = f \left(W_M \begin{bmatrix} B \\ C \end{bmatrix} \right),$$

Models – RNTN

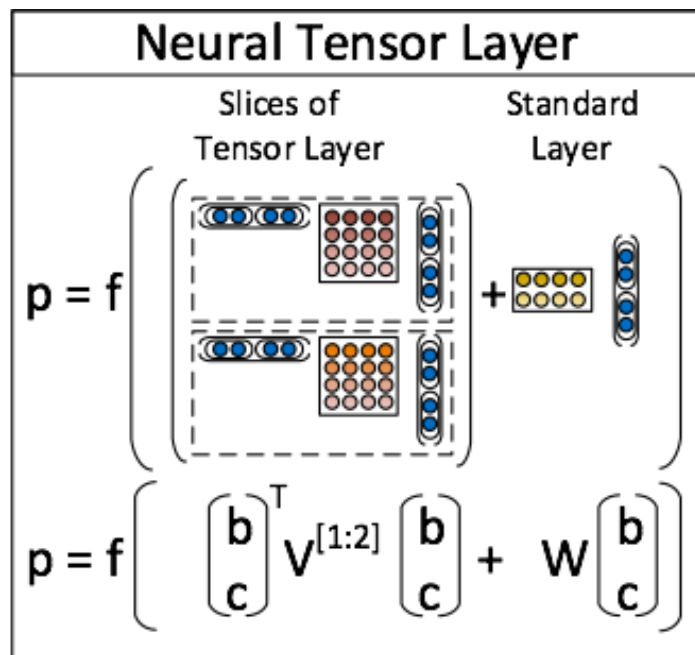


Figure 5: A single layer of the Recursive Neural Tensor Network. Each dashed box represents one of d -many slices and can capture a type of influence a child can have on its parent.

$$h = \begin{bmatrix} b \\ c \end{bmatrix}^T V^{[1:d]} \begin{bmatrix} b \\ c \end{bmatrix}; h_i = \begin{bmatrix} b \\ c \end{bmatrix}^T V^{[i]} \begin{bmatrix} b \\ c \end{bmatrix}.$$

$$p_1 = f \left(\begin{bmatrix} b \\ c \end{bmatrix}^T V^{[1:d]} \begin{bmatrix} b \\ c \end{bmatrix} + W \begin{bmatrix} b \\ c \end{bmatrix} \right),$$

$$p_2 = f \left(\begin{bmatrix} a \\ p_1 \end{bmatrix}^T V^{[1:d]} \begin{bmatrix} a \\ p_1 \end{bmatrix} + W \begin{bmatrix} a \\ p_1 \end{bmatrix} \right).$$

Experiments

Model	Fine-grained		Positive/Negative	
	All	Root	All	Root
NB	67.2	41.0	82.6	81.8
SVM	64.3	40.7	84.6	79.4
BiNB	71.0	41.9	82.7	83.1
VecAvg	73.3	32.7	85.1	80.1
RNN	79.0	43.2	86.1	82.4
MV-RNN	78.7	44.4	86.8	82.9
RNTN	80.7	45.7	87.6	85.4

Table 1: Accuracy for fine grained (5-class) and binary predictions at the sentence level (root) and for all nodes.

- sentence treebank were split into (8544), dev(1101) and test splits(2210).
- dev set was used to cross-validate over regularization of weights, word vector sizes, learning rate and mini batch for AdaGrad.

Optimal performance when:

- word vector sizes between 25-30.
- batch sizes between 20 & 30.

Model Analysis – Contrastive Conjunction

RNTN(41%) , MV-RNN(37%), RNN(36%) & biNB(27%)

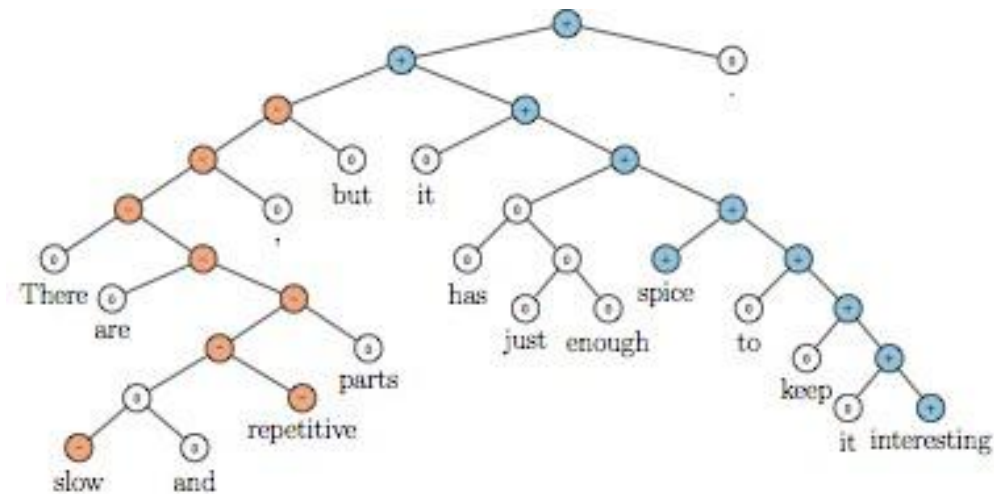


Figure 7: Example of correct prediction for contrastive conjunction *X but Y*.

Model Analysis – High Level Negation

Model	Accuracy	
	Negated Positive	Negated Negative
biNB	19.0	27.3
RNN	33.3	45.5
MV-RNN	52.4	54.6
RNTN	71.4	81.8

Table 2: Accuracy of negation detection. Negated positive is measured as correct sentiment inversions. Negated negative is measured as increases in positive activations.

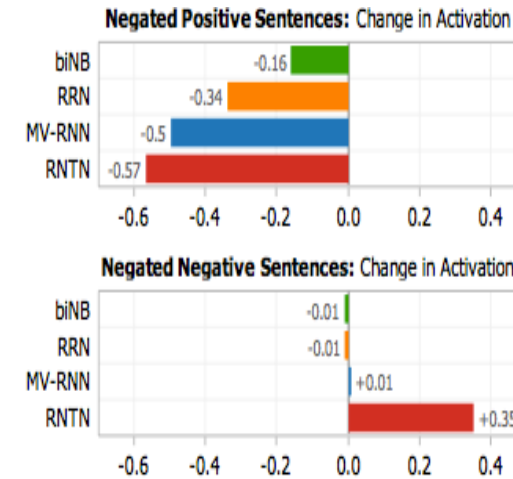


Figure 8: Change in activations for negations. Only the RNTN correctly captures both types. It decreases positive sentiment more when it is negated and learns that negating negative phrases (such as *not terrible*) should increase neutral and positive activations.

Model Analysis – High Level Negation

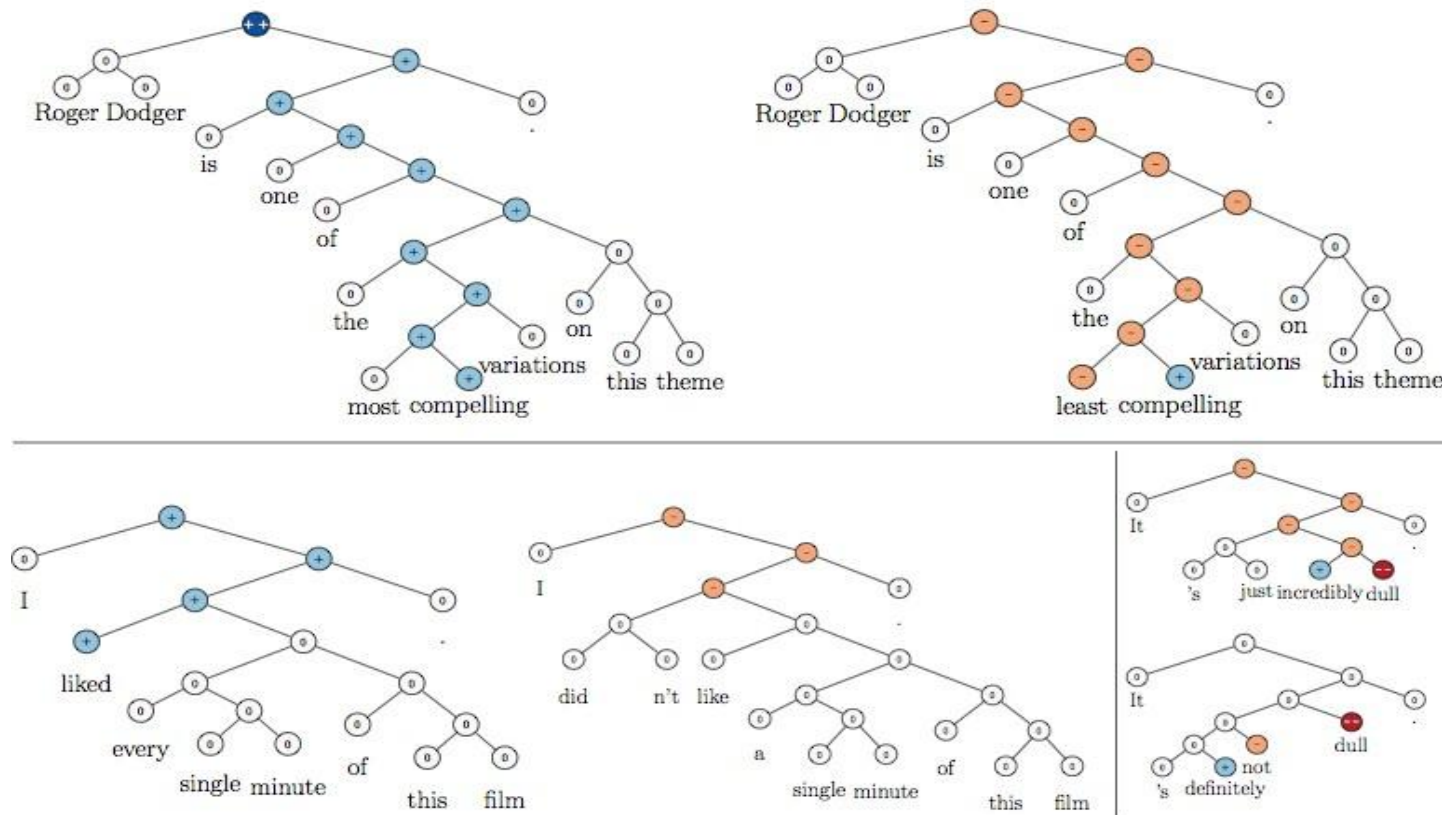


Figure 9: RNTN prediction of positive and negative (bottom right) sentences and their negation.

End

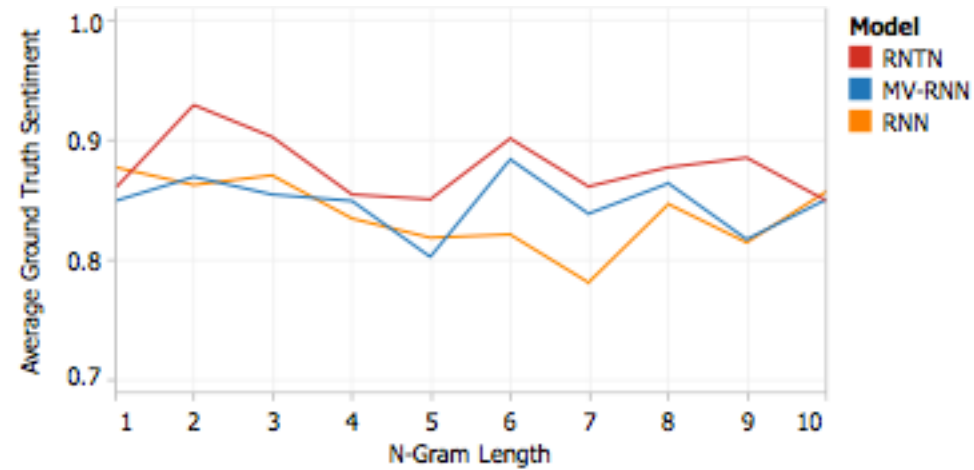


Figure 10: Average ground truth sentiment of top 10 most positive n -grams at various n . The RNTN correctly picks the more negative and positive examples.

End

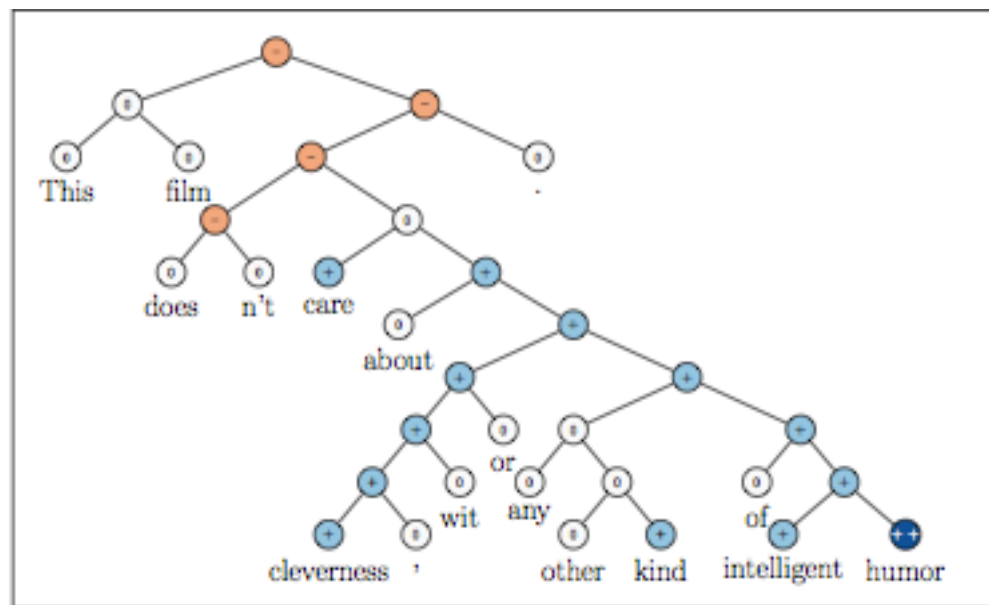
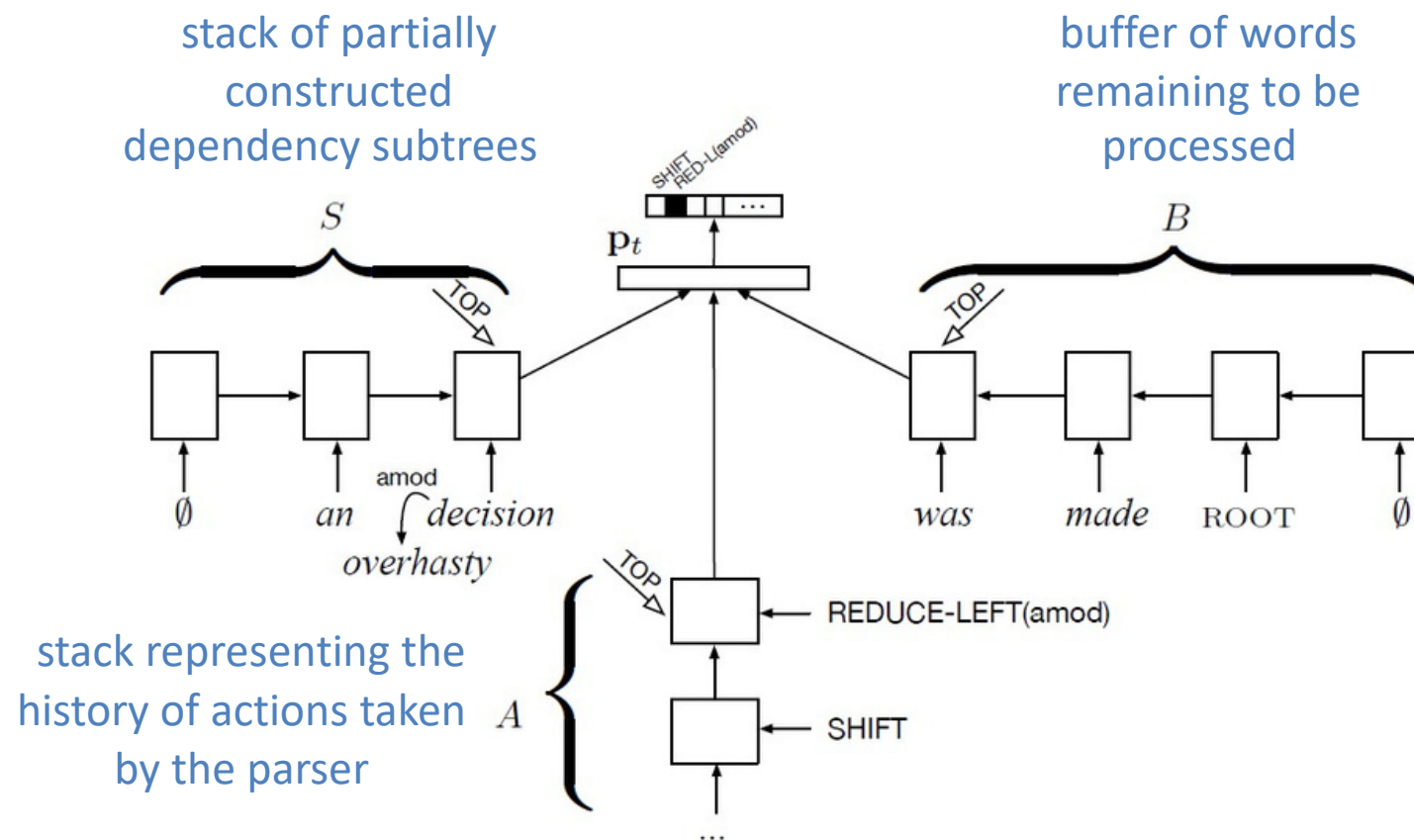


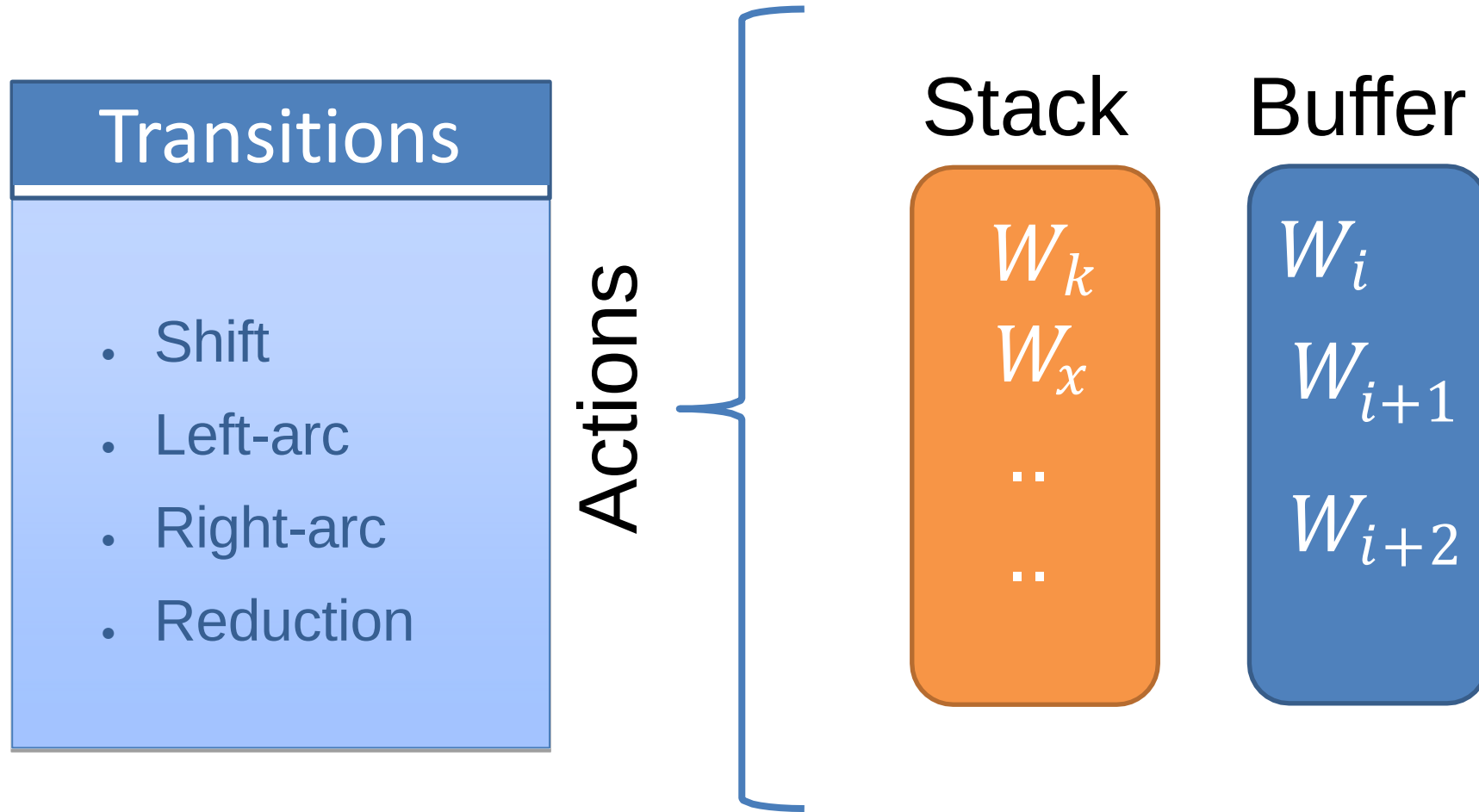
Figure 1: Example of the Recursive Neural Tensor Network accurately predicting 5 sentiment classes, very negative to very positive ($--, -, 0, +, ++$), at every node of a parse tree and capturing the negation and its scope in this sentence.

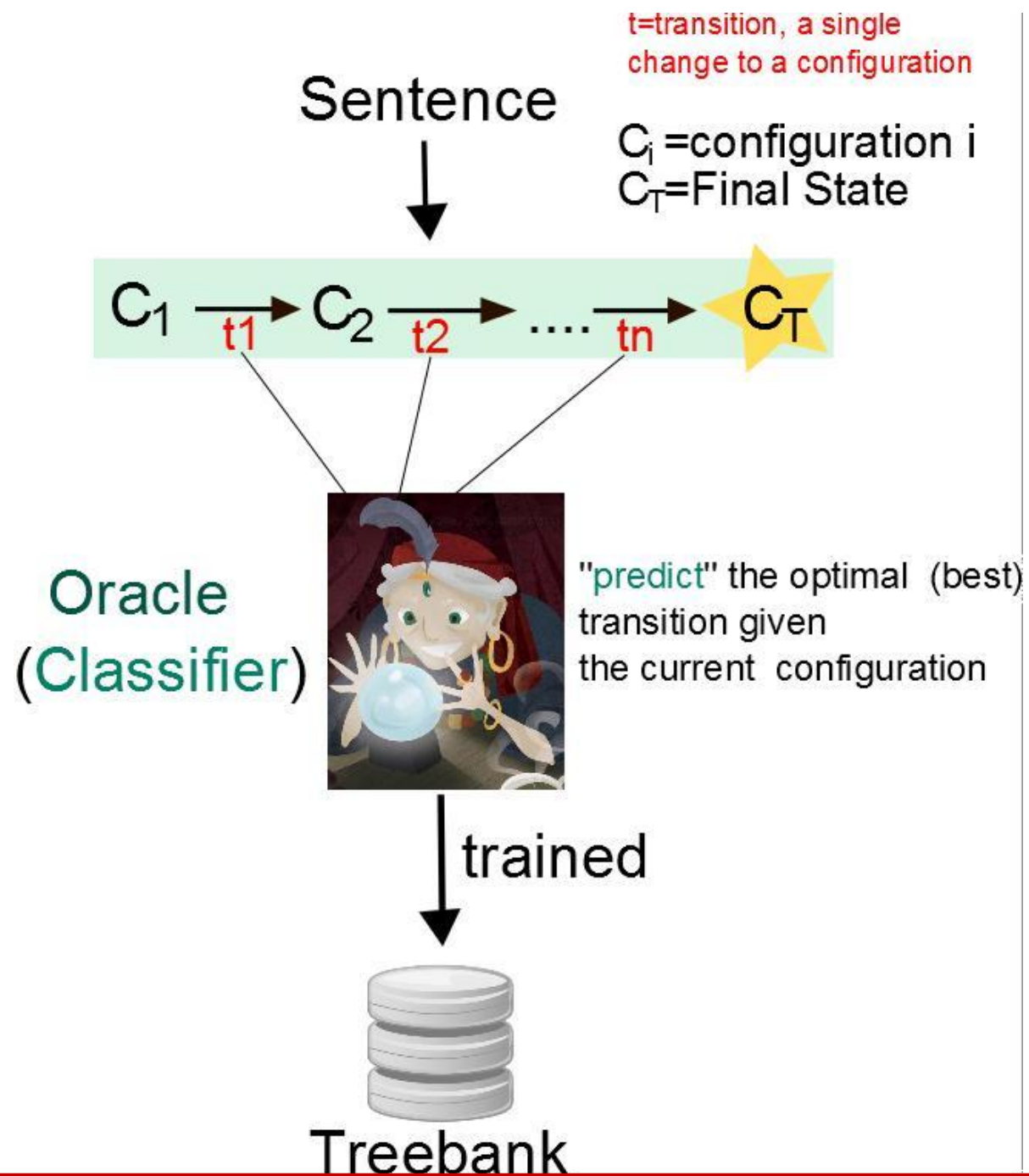
Stack Recurrent Network



Dyer et al., Transition-Based Dependency Parsing with Stack Long Short-Term Memory, 2015

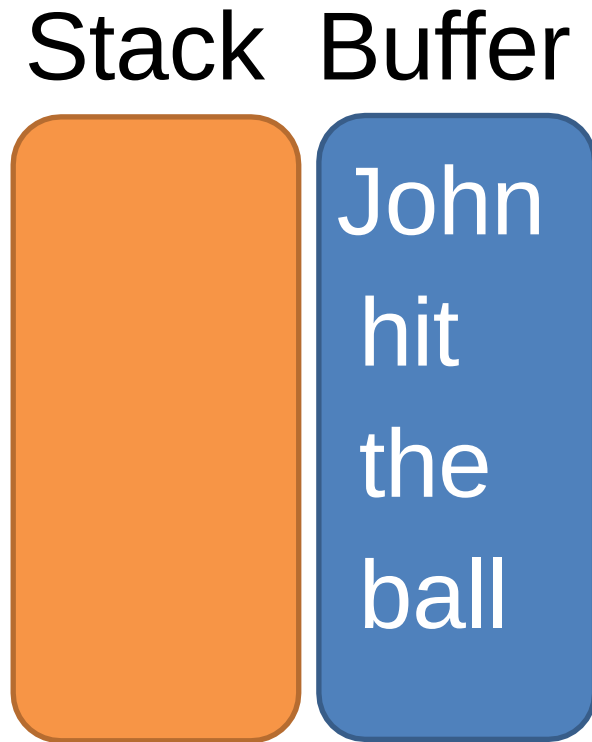
Transition Based Parsing





Example

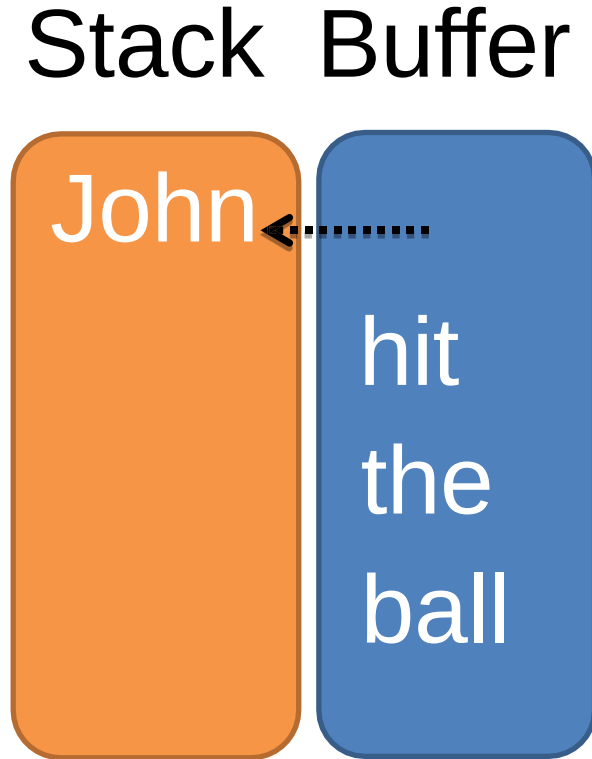
John hit the ball



Example

Transition=Shift

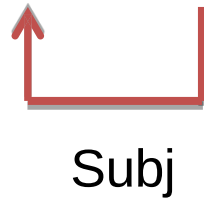
John hit the ball



Example

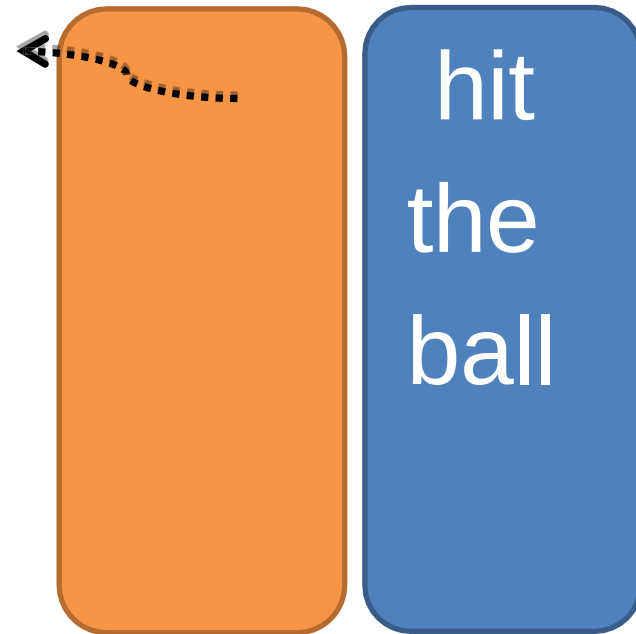
Transition=left Arc

John hit the ball



John

Stack Buffer



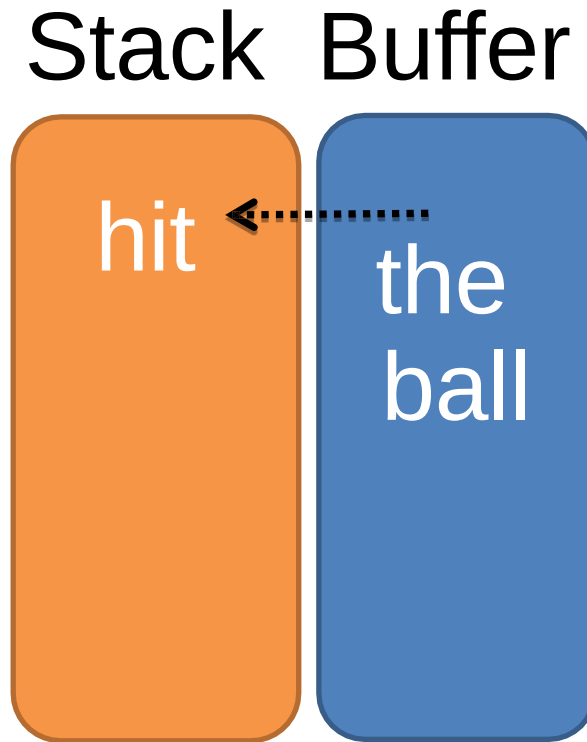
Only if $h(john) = 0$

Example

Transition=Shift

John hit the ball

Subj



Example

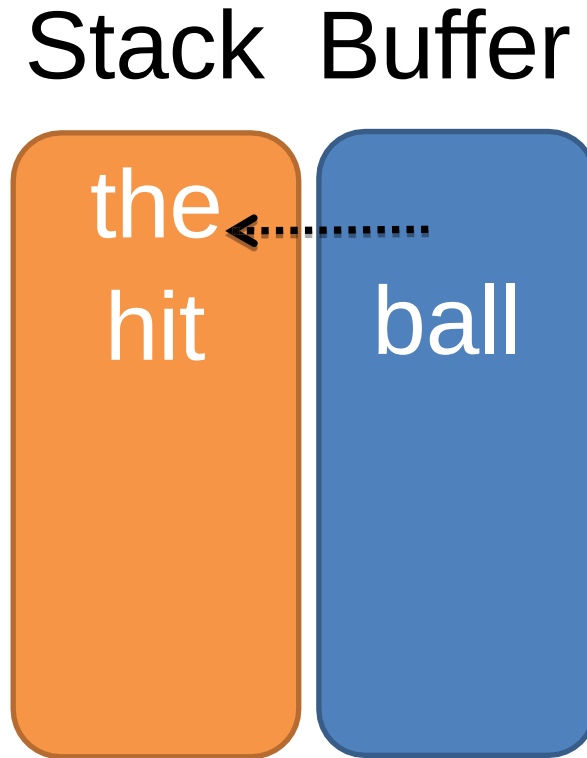
Transition=Shift

John hit the ball

Subj



A red bracket is drawn under the word 'John', with an upward-pointing arrow at its left end and a vertical line at its right end.



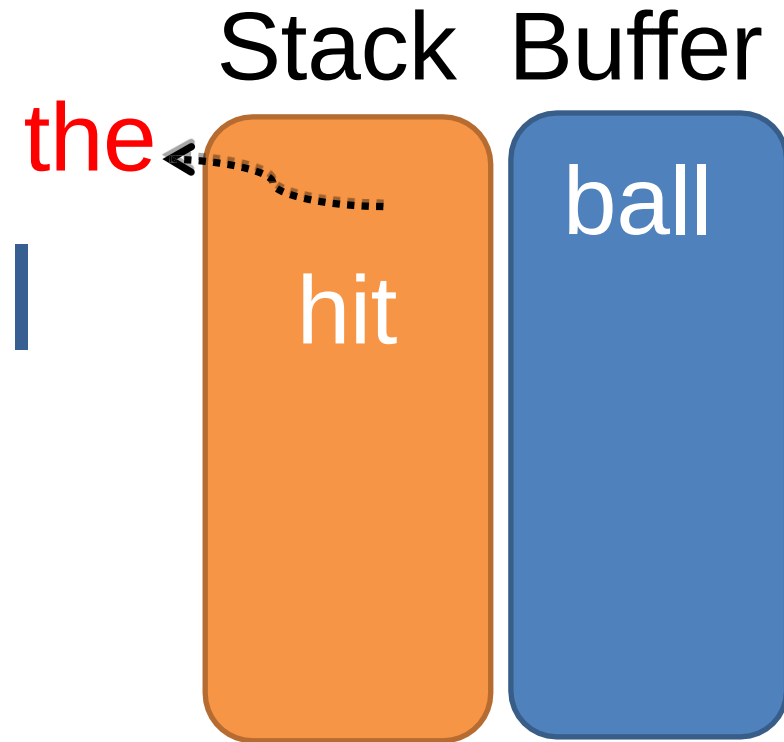
Example

Transition=left Arc

John hit the ball

Subj

Det

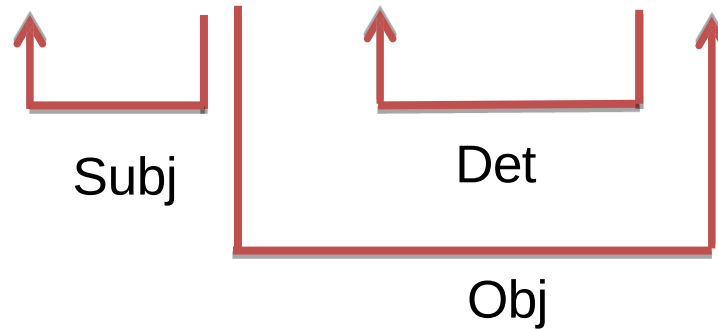


Only if $h(the) = 0$

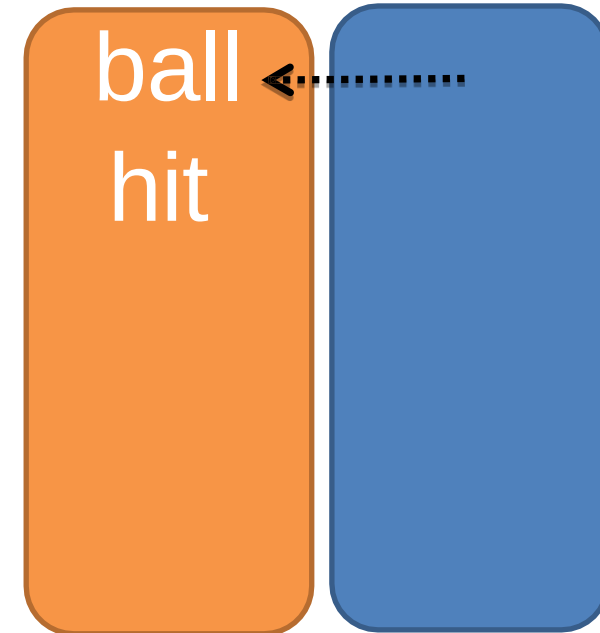
Example

Transition=Right Arc

John hit the ball



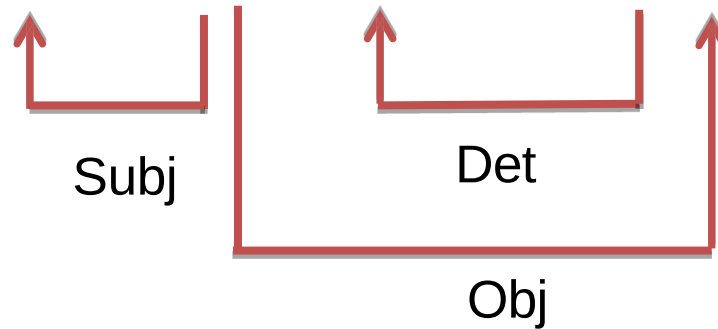
Stack Buffer



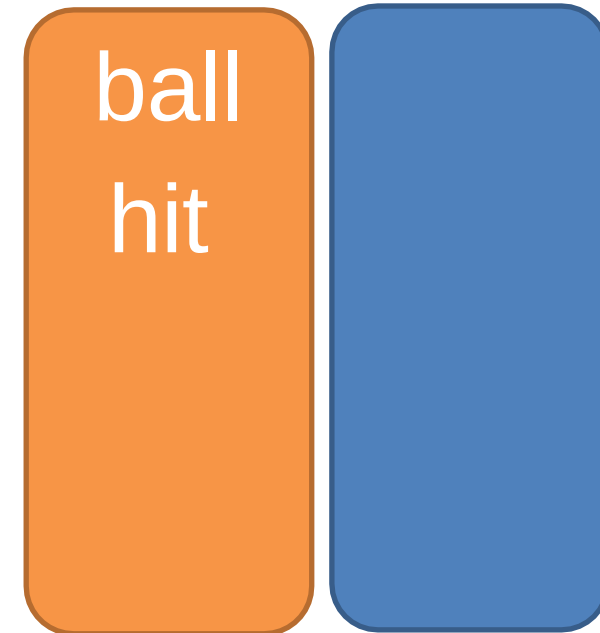
Only if $h(ball) = 0$

Example

John hit the ball

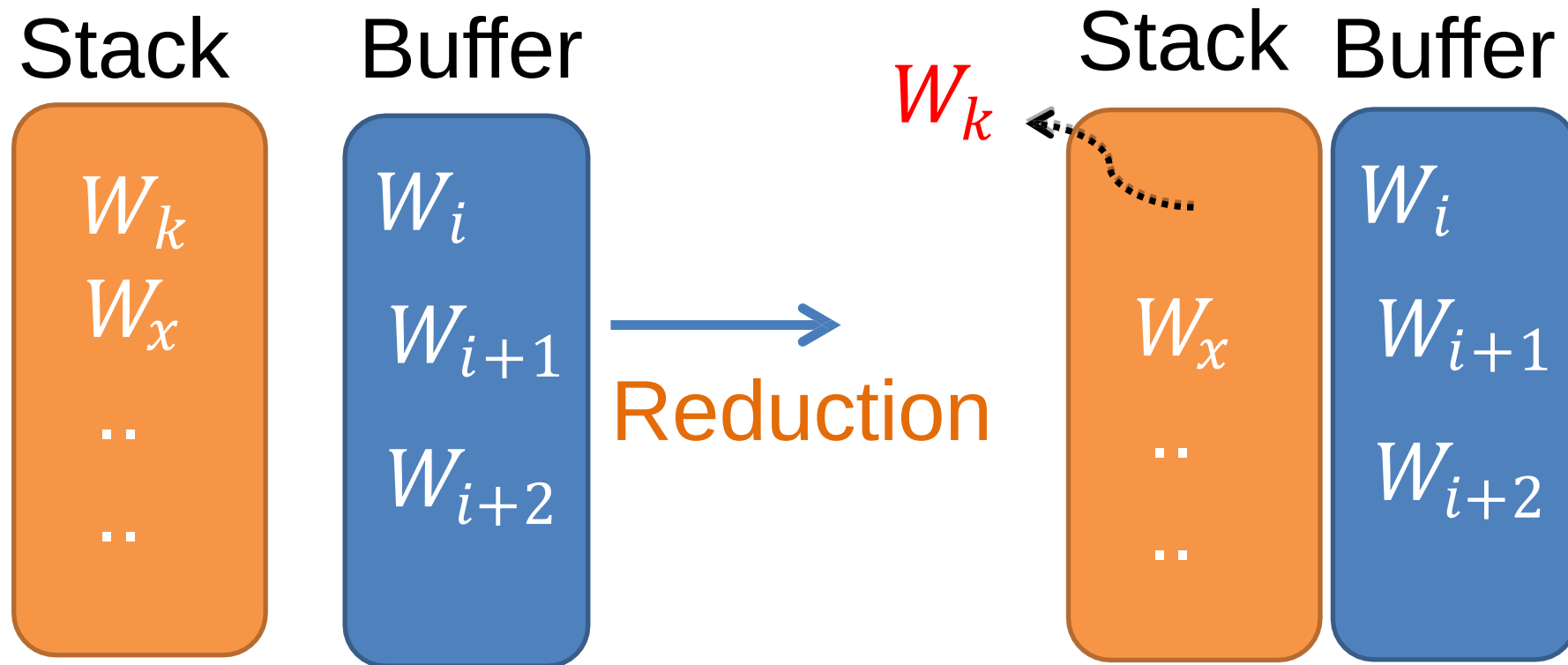


Stack Buffer



Buffer is Empty= Terminal Configuration

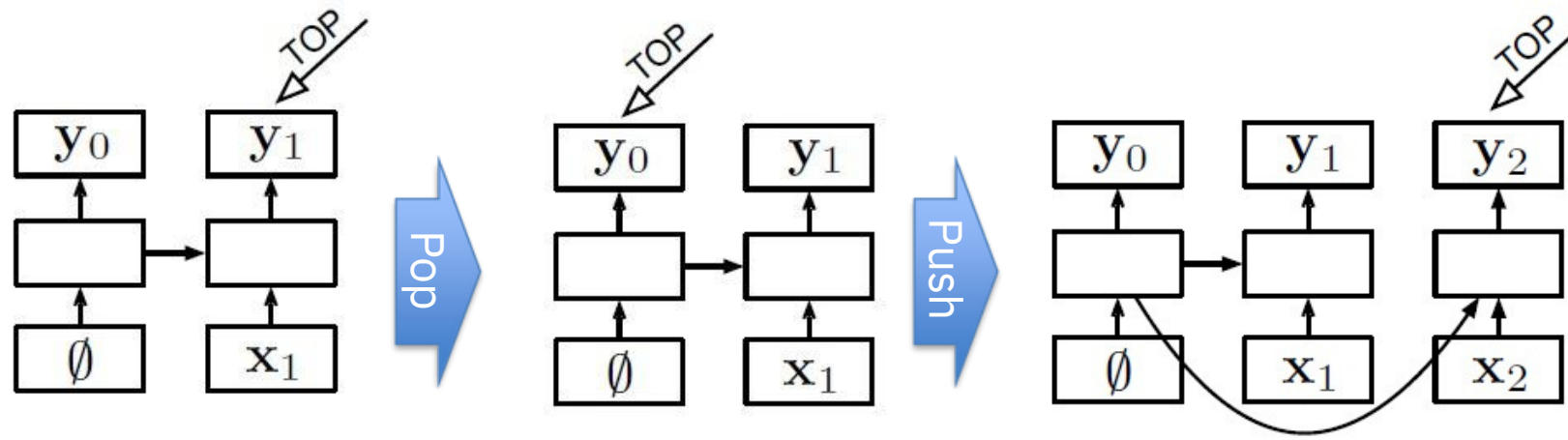
Transition Based Parsing



Sentence: $W_x W_k \dots W_i W_{i+1} \dots$

Only if $h(W_k) \neq 0$

Stack Recurrent Network



Dyer et al., Transition-Based Dependency Parsing with Stack Long Short-Term Memory, 2015